



SQL

Topic 01: Introduction to SQL

1. What is Data?

Data is raw, unprocessed facts and figures. On its own, data has no meaning — it becomes useful only when organized and interpreted.

Examples of Data:

- "Keval", 23, "India", 85000
- 1, 0, 1, 0, 1
- "2024-03-30", "Order #1024"

Types of Data:

Type	Description	Example
Structured	Organized in rows & columns	Excel sheet, SQL table
Semi-structured	Has some structure but not rigid	JSON, XML, CSV
Unstructured	No predefined format	Images, Videos, Text files

Data Science Insight: SQL is primarily used to work with **structured data**. But in modern pipelines, SQL is also used to query semi-structured data like JSON stored in databases like PostgreSQL or BigQuery.

2. What is a Database?

A **Database** is an organized collection of data that is stored electronically and can be easily accessed, managed, and updated.

Think of it like a **digital filing cabinet** where data is stored in a structured and systematic way.

Why do we need Databases?

Before databases existed, data was stored in:

- Flat files (CSV, text files) → Hard to search, no relationships
- Paper records → Impossible to query

Problems with flat file systems:

- Data Redundancy** → Same data stored multiple times
- Data Inconsistency** → Copies of data may differ
- No Security** → Anyone can access/edit any file
- No Concurrency** → Only one person can access data at a time
- No Backup/Recovery** → If file is lost, data is gone forever

Databases solve ALL of these problems.

3. Types of Databases

3.1 Based on Data Model:

Type	Description	Examples
Relational Database	Data stored in tables with rows & columns	MySQL, PostgreSQL, SQL Server
NoSQL Database	Flexible schema; stores JSON, key-value, etc.	MongoDB, Redis, Cassandra
NewSQL Database	SQL + NoSQL scalability	CockroachDB, Google Spanner
Graph Database	Stores data as nodes and edges	Neo4j, Amazon Neptune
Time-Series Database	Optimized for time-stamped data	InfluxDB, TimescaleDB
In-Memory Database	Stored in RAM for lightning-fast access	Redis, Memcached
Cloud Database	Hosted on cloud, serverless	BigQuery, Snowflake, Redshift

3.2 Based on Usage:

Type	Description	Example
OLTP (Transactional)	High volume of short transactions	Bank transactions, e-commerce
OLAP (Analytical)	Complex queries on large datasets	Data Warehouses, BI tools
HTAP	Hybrid of OLTP + OLAP	Snowflake, TiDB

Data Science Insight:

- Data Scientists** mostly work with **OLAP** systems like Snowflake, BigQuery, Redshift
- Data Engineers** design OLTP databases and build ETL pipelines between OLTP → OLAP
- ML Engineers** extract features from both using SQL

4. What is DBMS?

DBMS (Database Management System) is software that allows users to create, read, update, and delete data from a database.

Core Functions of DBMS:

- **Data Definition** → Create/modify database structure
- **Data Manipulation** → Insert, update, delete records
- **Data Retrieval** → Query and fetch data
- **Data Security** → Access control and permissions
- **Data Integrity** → Enforce rules (no duplicates, valid values)
- **Concurrency Control** → Allow multiple users to access data simultaneously
- **Backup & Recovery** → Restore data in case of failure

Examples:

- MySQL, PostgreSQL, SQLite, Oracle, Microsoft SQL Server

5. What is RDBMS?

RDBMS (Relational Database Management System) is a type of DBMS where data is stored in **tables (relations)** and these tables are connected to each other using **relationships**.

Key Features of RDBMS:

Feature	Description
Tables	Data is organized in rows (records) and columns (fields)
Primary Key	Uniquely identifies each row in a table
Foreign Key	Links one table to another
Relationships	Tables are related (one-to-one, one-to-many, many-to-many)
ACID Properties	Guarantees reliable transactions
Normalization	Reduces data redundancy

ACID Properties (Critical for Interviews):

Property	Meaning
Atomicity	Transaction is all-or-nothing (either fully completes or fully rolls back)
Consistency	Database remains in valid state before and after transaction
Isolation	Transactions don't interfere with each other
Durability	Once committed, data is permanently saved

Example RDBMS Tables:

Customers Table:

customer_id	name	city	email
1	Alice Kumar	Mumbai	alice@email.com
2	Bob Singh	Delhi	bob@email.com
3	Carol Rao	Bangalore	carol@email.com

Orders Table:

order_id	customer_id	product	amount	order_date
101	1	Laptop	75000	2024-01-10
102	2	Phone	30000	2024-01-15
103	1	Headphones	5000	2024-02-01

Here, `customer_id` in Orders is a **Foreign Key** that links to `customer_id` in Customers. This is the "relational" part of RDBMS.

6. What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases.

- Pronounced: **"S-Q-L"** or **"Sequel"** (both are acceptable)
- Created by: IBM researchers **Donald D. Chamberlin** and **Raymond F. Boyce** in the **1970s**
- Standardized by **ANSI** and **ISO**
- First called **SEQUEL** (Structured English Query Language)

What can SQL do?

- **Create** databases and tables
- **Insert** data into tables
- **Update** existing data
- **Delete** data
- **Query** (retrieve) data with filtering, sorting, grouping
- **Join** multiple tables
- **Control** who can access data
- **Define** constraints and rules on data

SQL is Declarative:

You tell SQL **WHAT** you want, not **HOW** to get it.

```
-- You say WHAT you want:
SELECT name, salary FROM employees WHERE department = 'Sales';
-- SQL figures out HOW to retrieve it (query execution plan)
```

Compare this to Python (procedural):

```
# You say HOW to do it step by step:
result = []
for emp in employees:
    if emp['department'] == 'Sales':
        result.append({'name': emp['name'], 'salary': emp['salary']})
```

SQL is cleaner and much faster for large datasets!

7. Why SQL for Data Science & AI/ML?

The Hard Truth:

- Most beginners think Python/pandas replaces SQL. **It does NOT.**
- In every real-world data job — Data Analyst, Data Scientist, ML Engineer — **SQL is used daily.**

Where SQL fits in the Data Science Pipeline:

```
Raw Data (in Database)
  ↓
[SQL] Data Extraction & Cleaning
  ↓
[SQL/Python] Feature Engineering
  ↓
[Python/sklearn/TF] Model Training
  ↓
[SQL] Predictions stored back in DB
  ↓
[SQL/BI Tool] Reporting & Monitoring
```

Specific Use Cases:

Use Case	How SQL is Used
Data Cleaning	Remove nulls, fix formats, deduplicate
Exploratory Data Analysis (EDA)	Aggregations, distributions, outlier detection
Feature Engineering	Create new columns, ratios, lags, rolling averages
Data Extraction for ML	Pull training/test datasets from databases
A/B Testing Analysis	Compare metrics between experiment groups
Cohort Analysis	Track user behavior over time
Churn Prediction Data Prep	Build feature tables for churn models
Recommendation System	Extract user-item interaction matrices
Business Metrics (KPIs)	Revenue, retention, conversion rates

Industry Fact:

- 90%+ of data jobs require SQL
- SQL is used by **data engineers, data analysts, data scientists, ML engineers, and even software engineers**
- Tools like **Pandas, Spark, BigQuery, Redshift, Snowflake** all have SQL interfaces

8. SQL vs NoSQL

Feature	SQL (Relational)	NoSQL
Data Structure	Tables (rows & columns)	Documents, Key-Value, Graph, Column
Schema	Fixed schema (strict)	Flexible / Schema-less
Query Language	SQL	Depends on DB (not standardized)
Relationships	Strong (JOINS)	Usually weak/no native JOINS
Scalability	Vertical (scale up hardware)	Horizontal (add more servers)
ACID Compliance	Full ACID	Eventual consistency (mostly)
Best For	Structured data, complex queries	Unstructured data, high speed/scale
Examples	MySQL, PostgreSQL, SQL Server	MongoDB, Redis, Cassandra, DynamoDB

When to use SQL?

- Your data is structured and has clear relationships
- You need complex queries, joins, aggregations
- Consistency and integrity are critical (banking, e-commerce)

When to use NoSQL?

- You need to store unstructured data (JSON, images, logs)
- You need massive horizontal scalability
- Speed is more important than strict consistency

9. Popular SQL Databases

Database	Company	Best For	Used In
MySQL	Oracle	Web apps, general purpose	Facebook, Twitter (historically)
PostgreSQL	Open Source	Advanced features, Data Science	Instagram, Spotify, Uber
SQLite	Open Source	Lightweight, embedded apps	Android, iOS apps
Microsoft SQL Server	Microsoft	Enterprise, Windows ecosystem	Banks, large enterprises
Oracle Database	Oracle	Enterprise, very large scale	Fortune 500 companies
Google BigQuery	Google	Cloud analytics, big data	Data warehouses
Amazon Redshift	Amazon	Cloud data warehousing	Cloud-based analytics
Snowflake	Snowflake	Cloud, data science, OLAP	Leading data companies
DuckDB	Open Source	In-process analytics (like SQLite for analytics)	Data Science workflows

For this course: We will use **standard SQL syntax** that works across all databases. Differences between databases will be highlighted when they arise.

10. SQL Sublanguages

SQL is divided into 5 sublanguages based on the type of operation:

10.1 DDL — Data Definition Language

Purpose: Define and modify the structure of the database (tables, schemas)

Command	Description
CREATE	Create a new table/database
ALTER	Modify an existing table
DROP	Delete a table/database
TRUNCATE	Remove all records from a table
RENAME	Rename a table or column

```
-- Example:
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  name VARCHAR(100),
  salary DECIMAL(10, 2),
  dept VARCHAR(50)
);
```

10.2 DML — Data Manipulation Language

Purpose: Insert, update, and delete data in tables

Command	Description
INSERT	Add new records
UPDATE	Modify existing records
DELETE	Remove specific records

```
-- Example:
INSERT INTO employees VALUES (1, 'Alice Kumar', 85000, 'Sales');
UPDATE employees SET salary = 90000 WHERE emp_id = 1;
DELETE FROM employees WHERE emp_id = 1;
```

10.3 DQL — Data Query Language

Purpose: Retrieve data from tables

Command	Description
SELECT	Query/fetch data

```
-- Example:
SELECT name, salary FROM employees WHERE dept = 'Sales' ORDER BY salary DESC;
```

Note: Some references classify SELECT under DML. In this course, we treat it separately as DQL since it's the most important command for Data Science.

10.4 DCL — Data Control Language

Purpose: Control access and permissions

Command	Description
GRANT	Give access/permissions to a user
REVOKE	Remove access/permissions from a user

```
-- Example:
GRANT SELECT ON employees TO analyst_user;
REVOKE INSERT ON employees FROM analyst_user;
```

10.5 TCL — Transaction Control Language

Purpose: Manage database transactions

Command	Description
COMMIT	Save all changes made in the current transaction
ROLLBACK	Undo changes made in the current transaction
SAVEPOINT	Set a point to rollback to within a transaction

```
-- Example:
BEGIN;
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;
COMMIT; -- Both updates saved together
-- If error: ROLLBACK; -- Both updates undone
```

Summary Table of All Sublanguages:

Sublanguage	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE	Structure of database
DML	INSERT, UPDATE, DELETE	Manipulate data
DQL	SELECT	Query/retrieve data
DCL	GRANT, REVOKE	Access control
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transaction management

11. How SQL Works Internally

When you write a SQL query, here's what happens behind the scenes:

```
You write:
SELECT name FROM employees WHERE salary > 50000;

Step 1: PARSING
  → SQL engine checks syntax and grammar
  → Builds a parse tree

Step 2: SEMANTIC ANALYSIS
  → Checks if table "employees" exists
  → Checks if column "name" and "salary" exist
  → Validates data types

Step 3: QUERY OPTIMIZATION
  → Query optimizer decides the best execution plan
  → Should it scan the full table or use an index?
  → Chooses the most efficient path

Step 4: EXECUTION
  → Database engine executes the plan
  → Retrieves data from storage (disk or memory)

Step 5: RESULT RETURNED
  → Result set is sent back to you
```

Why does this matter for Data Science?

Understanding this helps you write **optimized queries** — choosing the right indexes, avoiding full table scans, making big data queries 10x–100x faster.

12. Real-World Analogy

Think of a database like a **large library**:

Library Concept	Database Equivalent
The Library	The Database
A Section (e.g., Science)	A Schema
A Shelf	A Table
A Book	A Row (record)
Book's Attributes (Title, Author, ISBN)	Columns (fields)
The Catalog System	Indexes
Librarian	DBMS
Library Rules	Constraints & Permissions

SQL is the **language you use to ask the librarian** to find, add, or modify books.

"Give me all books by authors from India, published after 2010, sorted by year."

```
SELECT title, author, year
FROM books
WHERE country = 'India' AND year > 2010
ORDER BY year;
```

13. Practice Questions

These are conceptual/theoretical questions. Answer them in your own words.

Easy Level:

1. What is data? Give 3 examples of structured data.
2. What is the difference between a database and a DBMS?
3. What does RDBMS stand for? Name 3 examples.
4. What is SQL and what does it stand for?
5. Name the 5 sublanguages of SQL and give one command from each.
6. What is the difference between DDL and DML?

Medium Level:

1. What are the 4 ACID properties? Explain each with a real-world banking example.
2. What is the difference between OLTP and OLAP databases? When would a Data Scientist use each?
3. Explain the difference between SQL and NoSQL databases. When would you choose one over the other?
4. What is a Primary Key? What is a Foreign Key? How are they related?
5. In the library analogy, what would a "query" represent?
6. Why is SQL considered a declarative language? How is it different from Python's procedural approach?

Hard / Interview Level:

1. A company stores customer transactions in MySQL and their product catalog in MongoDB. As a Data Scientist, you need to build a recommendation model. How would you approach the data extraction? What are the challenges?
2. Explain what happens step-by-step when you run `SELECT * FROM orders WHERE amount > 10000;` on a database with 100 million rows.
3. Why do large companies like Netflix, Uber, and Airbnb still use SQL despite having massive data volumes and also using NoSQL? What problem does SQL solve that NoSQL doesn't?
4. What is the difference between `DROP`, `DELETE`, and `TRUNCATE` ? When would you use each?
5. A banking transaction involves: (a) deducting ₹5,000 from Account A and (b) adding ₹5,000 to Account B. If the server crashes after step (a) but before step (b), what happens? Which ACID property prevents data loss here, and how?

📌 Key Takeaways

- **SQL** is the universal language for working with structured data stored in relational databases.
- **RDBMS** tables store data in rows and columns with defined relationships.
- SQL has **5 sublanguages**: DDL, DML, DQL, DCL, TCL — each serving a different purpose.
- **ACID properties** guarantee reliable and consistent transactions.
- SQL is **not optional** for Data Science — it's a core, daily-used skill.
- SQL is used for **data extraction, cleaning, feature engineering, EDA, and reporting** in real-world ML pipelines.
- In the Data Science world, you'll encounter SQL on platforms like **BigQuery, Snowflake, Redshift, and PostgreSQL**.

📘 Phase 1: Core Basics

📊 STANDARD DATASET (Used Throughout This Course)

📌 All examples in this course use the following tables. Memorize the structure.

Table: employees

emp_id	first_name	last_name	age	gender	department	salary	hire_date	manager_id	city
1	Alice	Kumar	30	F	Sales	85000	2020-01-15	5	Mumbai
2	Bob	Singh	35	M	IT	95000	2018-06-01	6	Delhi
3	Carol	Rao	28	F	HR	60000	2021-03-20	5	Bangalore
4	David	Mehta	40	M	Sales	75000	2017-09-10	5	Mumbai
5	Eva	Sharma	45	F	Management	130000	2015-01-01	NULL	Delhi
6	Frank	Gupta	38	M	IT	110000	2016-07-15	NULL	Bangalore
7	Grace	Nair	26	F	HR	55000	2022-08-01	5	Chennai
8	Henry	Joshi	33	M	IT	88000	2019-04-22	6	Delhi
9	Isla	Patel	29	F	Sales	72000	2021-11-30	5	Mumbai
10	Jake	Verma	31	M	Finance	82000	2020-05-14	NULL	Pune

Table: departments

dept_id	dept_name	budget	location
1	Sales	500000	Mumbai
2	IT	800000	Bangalore
3	HR	300000	Delhi
4	Finance	600000	Pune
5	Management	1000000	Delhi

Table: customers

customer_id	name	email	city	country	age	signup_date
1	Ravi Desai	ravi@email.com	Mumbai	India	32	2022-01-10
2	Priya Menon	priya@email.com	Delhi	India	27	2022-03-15
3	John Smith	john@email.com	New York	USA	45	2021-11-20
4	Sara Ali	sara@email.com	London	UK	35	2023-02-28
5	Chen Wei	chen@email.com	Shanghai	China	29	2022-07-05
6	Meera Iyer	meera@email.com	Bangalore	India	31	2021-09-18
7	Carlos Ruiz	carlos@email.com	Madrid	Spain	38	2023-05-12

Table: orders

order_id	customer_id	product_id	quantity	amount	order_date	status
101	1	1	2	150000	2024-01-05	Delivered
102	2	3	1	30000	2024-01-10	Pending
103	1	2	1	75000	2024-01-15	Delivered
104	3	4	3	9000	2024-02-01	Cancelled
105	4	1	1	75000	2024-02-10	Delivered
106	5	2	2	150000	2024-02-15	Pending
107	6	3	1	30000	2024-03-01	Delivered
108	2	4	5	15000	2024-03-10	Delivered

Table: products

product_id	name	category	price	stock
1	Laptop	Electronics	75000	50
2	Smartphone	Electronics	30000	200
3	Headphones	Electronics	5000	500
4	Desk Chair	Furniture	3000	100
5	Monitor	Electronics	15000	80

TOPIC 1: What is a Database, Table, Row, Column

Concept Explanation

Database

A **database** is an organized collection of related data stored electronically. It is managed by a **DBMS** (Database Management System) like MySQL, PostgreSQL, or SQL Server.

Think of a database as an entire **Excel Workbook**.

Table

A **table** is a structured grid that stores data in **rows and columns**, just like a single **Excel Sheet** inside a workbook.

- A database can have many tables
- Each table stores data about one specific entity (e.g., employees, orders)

Row (Record / Tuple)

A **row** represents a **single data entry** — one complete record of an entity.

- In the employees table: one row = one employee's full information

Column (Field / Attribute)

A **column** represents a **specific attribute** or property shared by all rows.

- In the employees table: salary is a column that stores the salary of every employee

Visual Representation:

```

DATABASE: company_db
|
├─ TABLE: employees
|   ├── COLUMNS: emp_id, first_name, last_name, salary, department...
|   ├── ROW 1: 1, 'Alice', 'Kumar', 85000, 'Sales'
|   ├── ROW 2: 2, 'Bob', 'Singh', 95000, 'IT'
|   └── ROW 3: 3, 'Carol', 'Rao', 60000, 'HR'
|
├─ TABLE: orders
|   └── ...
|
└─ TABLE: customers
    └── ...
```

Key Terminology:

Term	Also Called	Description
Table	Relation, Entity	Grid of rows and columns
Row	Record, Tuple	One complete data entry

Term	Also Called	Description
Column	Field, Attribute	One property shared across all rows
Cell	Value	Intersection of a row and a column
Primary Key	PK	Unique identifier for each row
Foreign Key	FK	Column linking to another table's PK

TOPIC 2: SQL Data Types

Concept Explanation

Every column in a SQL table must have a **data type** — it defines what kind of data can be stored in that column. Choosing the right data type is critical for:

- **Data integrity** (no junk values stored)
- **Storage efficiency** (use less disk space)
- **Query performance** (faster searches)

Main Categories of Data Types

2.1 Numeric Data Types

Data Type	Description	Range / Notes	Example
INT / INTEGER	Whole numbers	-2,147,483,648 to 2,147,483,647	1, 42, -99
SMALLINT	Small whole numbers	-32,768 to 32,767	Age, quantity
BIGINT	Very large whole numbers	-9.2 × 10^18 to 9.2 × 10^18	User counts
DECIMAL(p, s)	Exact decimal (precision, scale)	p = total digits, s = decimals	75000.50
NUMERIC(p, s)	Same as DECIMAL	Same as DECIMAL	1234.56
FLOAT	Approximate decimal (floating point)	Large range, less precision	3.14159
REAL	Smaller floating point	Less precise than FLOAT	2.71

Rule: Use DECIMAL for money/financial data. Never use FLOAT for currency — floating point errors can cause rounding issues!

2.2 String (Character) Data Types

Data Type	Description	Notes	Example
CHAR(n)	Fixed-length string	Always uses n characters	CHAR(2) → 'IN'
VARCHAR(n)	Variable-length string	Uses only what's needed, up to n	VARCHAR(100)
TEXT	Large variable-length text	No length limit (up to 65KB+)	Blog posts
NCHAR(n)	Fixed Unicode string	For multilingual data	Chinese names
NVARCHAR(n)	Variable Unicode string	For multilingual data	Emoji support

Rule: Use VARCHAR for names, emails, descriptions. Use CHAR for fixed codes like country codes ('IN', 'US').

2.3 Date and Time Data Types

Data Type	Description	Format	Example
DATE	Date only	YYYY-MM-DD	2024-01-15
TIME	Time only	HH:MM:SS	14:30:00
DATETIME	Date and time combined	YYYY-MM-DD HH:MM:SS	2024-01-15 14:30:00
TIMESTAMP	Date + time + timezone	YYYY-MM-DD HH:MM:SS UTC	2024-01-15 09:00:00 UTC
YEAR	Year only (MySQL)	YYYY	2024

2.4 Boolean Data Type

Data Type	Description	Values
BOOLEAN	True/False values	TRUE, FALSE, 1, 0

Note: MySQL stores BOOLEAN as TINYINT(1). PostgreSQL has a native BOOLEAN type.

2.5 Other Data Types

Data Type	Description	Example
ENUM	One value from a defined list	ENUM('M','F') or ENUM('Active','Inactive')
JSON	JSON structured data	Storing nested objects
ARRAY	Array of values (PostgreSQL)	{1, 2, 3}
UUID	Universally unique identifier	550e8400-e29b-41d4-a716-...
BLOB	Binary large object (images)	Storing files/images in DB

Choosing the Right Data Type — Rules


```
employee salary      → DECIMAL(10, 2)  -- exact decimal
employee age         → SMALLINT         -- small whole number
employee name        → VARCHAR(100)     -- variable length text
employee id          → INT              -- whole number
country code         → CHAR(2)          -- always exactly 2 chars
hire date            → DATE             -- date only
is_active flag       → BOOLEAN          -- true/false
product description  → TEXT             -- long text
transaction amount   → DECIMAL(15, 2)   -- precise financial data
```

Examples

Example 1: Checking data types in a realistic table definition

```
CREATE TABLE employees (
  emp_id      INT,
  first_name  VARCHAR(50),
  last_name   VARCHAR(50),
  age         SMALLINT,
  gender      CHAR(1),
  department  VARCHAR(50),
  salary      DECIMAL(10, 2),
  hire_date   DATE,
  is_active   BOOLEAN
);
```

Example 2: Data type mismatches (Common Mistakes)

```
-- ❌ Wrong: Using FLOAT for money (rounding errors)
price FLOAT

-- ✅ Correct: Using DECIMAL for money
price DECIMAL(10, 2)

-- ❌ Wrong: Using TEXT for short codes
country_code TEXT

-- ✅ Correct: Using CHAR for fixed-length codes
country_code CHAR(2)

-- ❌ Wrong: Storing dates as VARCHAR
hire_date VARCHAR(20)  -- '2024-01-15' stored as string

-- ✅ Correct: Using DATE type
hire_date DATE         -- Allows date math, comparisons
```

TOPIC 3: Creating a Table - CREATE TABLE

Concept Explanation

CREATE TABLE is a **DDL (Data Definition Language)** command used to create a new table in a database. You define the table name, columns, data types, and constraints.

Why this matters for Data Science:

- You need to create tables to store raw data before importing CSV files
- Schema design affects query performance
- Constraints ensure data quality (no nulls, no duplicates)

Syntax

```
CREATE TABLE table_name (
  column1 datatype [constraints],
  column2 datatype [constraints],
  column3 datatype [constraints],
  ...
  [table-level constraints]
);
```

Key Constraints:

Constraint	Description
PRIMARY KEY	Uniquely identifies each row; cannot be NULL
NOT NULL	Column must have a value; cannot be empty
UNIQUE	All values in column must be different

Constraint	Description
DEFAULT	Provides a default value if none is specified
CHECK	Ensures values satisfy a condition
FOREIGN KEY	Links column to another table's primary key
AUTO_INCREMENT	Automatically increments value (MySQL)
SERIAL	Auto-incrementing integer (PostgreSQL)

Examples

Example 1: Simple Table (No Constraints)

```
CREATE TABLE products (  
  product_id  INT,  
  name        VARCHAR(100),  
  price       DECIMAL(10, 2),  
  stock       INT  
);
```

Example 2: Table with PRIMARY KEY

```
CREATE TABLE departments (  
  dept_id     INT PRIMARY KEY,  
  dept_name   VARCHAR(50),  
  budget      DECIMAL(15, 2),  
  location    VARCHAR(100)  
);
```

Example 3: Table with Multiple Constraints

```
CREATE TABLE employees (  
  emp_id      INT           PRIMARY KEY,  
  first_name  VARCHAR(50)   NOT NULL,  
  last_name   VARCHAR(50)   NOT NULL,  
  age         SMALLINT      CHECK (age >= 18 AND age <= 70),  
  gender      CHAR(1)        CHECK (gender IN ('M', 'F', 'O')),  
  department  VARCHAR(50),  
  salary      DECIMAL(10,2) NOT NULL DEFAULT 30000.00,  
  hire_date   DATE,  
  is_active   BOOLEAN        DEFAULT TRUE  
);
```

Example 4: AUTO_INCREMENT Primary Key (MySQL)

```
CREATE TABLE customers (  
  customer_id INT           AUTO_INCREMENT PRIMARY KEY,  
  name        VARCHAR(100)  NOT NULL,  
  email       VARCHAR(150)  UNIQUE NOT NULL,  
  city        VARCHAR(50),  
  signup_date DATE          DEFAULT CURRENT_DATE  
);
```

Example 5: PostgreSQL SERIAL (auto-increment)

```
CREATE TABLE customers (  
  customer_id SERIAL          PRIMARY KEY,  
  name        VARCHAR(100)  NOT NULL,  
  email       VARCHAR(150)  UNIQUE NOT NULL,  
  city        VARCHAR(50),  
  signup_date DATE          DEFAULT CURRENT_DATE  
);
```

Example 6: FOREIGN KEY (Linking Tables)

```
CREATE TABLE orders (  
  order_id     INT           PRIMARY KEY,  
  customer_id  INT           NOT NULL,  
  product_id   INT           NOT NULL,  
  quantity     INT           DEFAULT 1,  
  amount       DECIMAL(12,2) NOT NULL,  
  order_date   DATE          DEFAULT CURRENT_DATE,  
  status       VARCHAR(20)   DEFAULT 'Pending',
```

```
FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Example 7: CREATE TABLE with COMPOSITE PRIMARY KEY

```
-- When two columns together uniquely identify a row
CREATE TABLE order_items (
  order_id    INT,
  product_id  INT,
  quantity    INT,
  unit_price  DECIMAL(10,2),

  PRIMARY KEY (order_id, product_id) -- Composite PK
);
```

Example 8: CREATE TABLE IF NOT EXISTS

```
-- Safe creation – won't throw error if table already exists
CREATE TABLE IF NOT EXISTS products (
  product_id INT    PRIMARY KEY,
  name       VARCHAR(100),
  price      DECIMAL(10,2)
);
```

Example 9: CREATE TABLE AS SELECT (Copy table structure + data)

```
-- Creates a new table from query results
CREATE TABLE high_salary_employees AS
SELECT * FROM employees WHERE salary > 80000;
```

Example 10: Modifying a Table — ALTER TABLE

```
-- Add a new column
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);

-- Modify column data type
ALTER TABLE employees MODIFY COLUMN phone VARCHAR(20);

-- Drop a column
ALTER TABLE employees DROP COLUMN phone;

-- Rename a column (MySQL 8+)
ALTER TABLE employees RENAME COLUMN dept TO department;

-- Add a constraint after creation
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary > 0);
```

Example 11: Drop a Table

```
DROP TABLE employees;           -- Permanently deletes table and all data
DROP TABLE IF EXISTS employees; -- Safe drop (no error if not exists)
TRUNCATE TABLE employees;       -- Deletes all rows but keeps structure
```

TOPIC 4: Inserting Data — INSERT INTO

Concept Explanation

INSERT INTO is a **DML (Data Manipulation Language)** command used to add new rows of data into a table.

Why this matters:

- Loading raw data into tables for analysis
- Adding test data for development
- Populating lookup/reference tables

Syntax

```
-- Method 1: Specify column names (Recommended)
INSERT INTO table_name (column1, column2, column3, ...)
```

```
VALUES (value1, value2, value3, ...);

-- Method 2: Without column names (All columns in order)
INSERT INTO table_name
VALUES (value1, value2, value3, ...);

-- Method 3: Insert Multiple Rows at Once
INSERT INTO table_name (column1, column2, ...)
VALUES
    (val1a, val2a, ...),
    (val1b, val2b, ...),
    (val1c, val2c, ...);

-- Method 4: Insert from Another Table
INSERT INTO table_name (column1, column2, ...)
SELECT column1, column2, ...
FROM other_table
WHERE condition;
```

Examples

Example 1: Basic INSERT

```
INSERT INTO departments (dept_id, dept_name, budget, location)
VALUES (1, 'Sales', 500000, 'Mumbai');
```

Example 2: INSERT Without Column Names

```
-- Must provide values for ALL columns in exact order
INSERT INTO departments
VALUES (2, 'IT', 800000, 'Bangalore');
```

Example 3: INSERT with NULL for Optional Fields

```
INSERT INTO employees (emp_id, first_name, last_name, salary, hire_date)
VALUES (11, 'Kavya', 'Reddy', 70000, '2024-01-01');
-- department, age, gender, city will be NULL
```

Example 4: INSERT Multiple Rows (Efficient)

```
INSERT INTO employees (emp_id, first_name, last_name, age, gender, department, salary,
hire_date, city)
VALUES
    (1, 'Alice', 'Kumar', 30, 'F', 'Sales', 85000, '2020-01-15', 'Mumbai'),
    (2, 'Bob', 'Singh', 35, 'M', 'IT', 95000, '2018-06-01', 'Delhi'),
    (3, 'Carol', 'Rao', 28, 'F', 'HR', 60000, '2021-03-20', 'Bangalore'),
    (4, 'David', 'Mehta', 40, 'M', 'Sales', 75000, '2017-09-10', 'Mumbai'),
    (5, 'Eva', 'Sharma', 45, 'F', 'Management', 130000, '2015-01-01', 'Delhi');
```

Example 5: INSERT with DEFAULT Values

```
-- If column has DEFAULT, you can skip it or explicitly use DEFAULT keyword
INSERT INTO customers (customer_id, name, email, city)
VALUES (1, 'Ravi Desai', 'ravi@email.com', 'Mumbai');
-- signup_date will use DEFAULT CURRENT_DATE automatically
```

Example 6: INSERT FROM SELECT (Copy filtered data)

```
-- Create archive table for high-salary employees
INSERT INTO high_salary_employees (emp_id, first_name, last_name, salary)
SELECT emp_id, first_name, last_name, salary
FROM employees
WHERE salary > 90000;
```

Example 7: UPDATE — Modify Existing Data

```
-- Update single row
UPDATE employees
SET salary = 90000
WHERE emp_id = 1;
```

```
-- Update multiple columns
UPDATE employees
SET salary = 70000, city = 'Hyderabad'
WHERE emp_id = 3;

-- Update multiple rows matching condition
UPDATE employees
SET salary = salary * 1.10 -- 10% raise
WHERE department = 'Sales';

-- ⚠ Update ALL rows (no WHERE – dangerous!)
UPDATE employees
SET is_active = TRUE;
```

Example 8: DELETE — Remove Rows

```
-- Delete specific row
DELETE FROM employees WHERE emp_id = 11;

-- Delete multiple rows matching condition
DELETE FROM employees WHERE department = 'Temp';

-- ⚠ Delete ALL rows (no WHERE – very dangerous!)
DELETE FROM employees;
-- Use TRUNCATE instead for clearing all rows – it's faster
```

TOPIC 5: SELECT Statement Basics

📘 Concept Explanation

SELECT is the most used SQL command — it retrieves data from one or more tables. It is part of **DQL (Data Query Language)**. As a Data Scientist, you will use SELECT in 90%+ of your SQL work.

📋 Syntax

```
SELECT column1, column2, ... -- Which columns to show
FROM table_name              -- Which table
WHERE condition              -- Filter rows (optional)
GROUP BY column              -- Group rows (optional)
HAVING condition             -- Filter groups (optional)
ORDER BY column ASC/DESC     -- Sort results (optional)
LIMIT n;                     -- Limit number of results (optional)
```

SQL Clause Execution Order (CRITICAL — Interview Topic!):

ORDER OF WRITING:	ORDER OF EXECUTION:
1. SELECT	1. FROM
2. FROM	2. WHERE
3. WHERE	3. GROUP BY
4. GROUP BY	4. HAVING
5. HAVING	5. SELECT
6. ORDER BY	6. ORDER BY
7. LIMIT	7. LIMIT

⚠ This order matters when debugging: you can use an alias from SELECT in ORDER BY but NOT in WHERE — because WHERE runs before SELECT!

🔍 Examples

Example 1: Select Everything (All Columns)

```
SELECT * FROM employees;
```

Output:

emp_id	first_name	last_name	age	gender	department	salary	hire_date	manager_id	city
1	Alice	Kumar	30	F	Sales	85000	2020-01-15	5	Mumbai
2	Bob	Singh	35	M	IT	95000	2018-06-01	6	Delhi
3	Carol	Rao	28	F	HR	60000	2021-03-20	5	Bangalore
...	...	...	...	...	...	...	...	...	...

Note: SELECT * retrieves all columns. In production, always specify columns — SELECT * can be very slow on large tables!

Example 2: Select All Customers

```
SELECT * FROM customers;
```

customer_id	name	email	city	country	age	signup_date
1	Ravi Desai	ravi@email.com	Mumbai	India	32	2022-01-10
2	Priya Menon	priya@email.com	Delhi	India	27	2022-03-15
3	John Smith	john@email.com	New York	USA	45	2021-11-20
...	...	...	...	...	...	...

Example 3: Select All Orders

```
SELECT * FROM orders;
```

order_id	customer_id	product_id	quantity	amount	order_date	status
101	1	1	2	150000	2024-01-05	Delivered
102	2	3	1	30000	2024-01-10	Pending
...	...	...	...	...	...	...

TOPIC 6: Selecting Specific Columns

Concept Explanation

Instead of `SELECT *`, you select only the columns you need. This is:

- **Faster** — less data transferred from database
- **Cleaner** — output is easier to read
- **Safer** — avoids exposing sensitive columns (e.g., passwords)

Examples

Example 1: Select Two Columns

```
SELECT first_name, salary
FROM employees;
```

Output:

first_name	salary
Alice	85000
Bob	95000
Carol	60000
David	75000
Eva	130000
Frank	110000
Grace	55000
Henry	88000
Isla	72000
Jake	82000

Example 2: Select Name, Department, City

```
SELECT first_name, last_name, department, city
FROM employees;
```

Output:

first_name	last_name	department	city
Alice	Kumar	Sales	Mumbai
Bob	Singh	IT	Delhi
Carol	Rao	HR	Bangalore
David	Mehta	Sales	Mumbai
Eva	Sharma	Management	Delhi

Example 3: Select Customer Names and Countries

```
SELECT name, country, city
FROM customers;
```

Output:

name	country	city
Ravi Desai	India	Mumbai
Priya Menon	India	Delhi
John Smith	USA	New York
Sara Ali	UK	London

name	country	city
Chen Wei	China	Shanghai

Example 4: Select with Simple Arithmetic

```
-- Calculate annual salary from monthly salary
SELECT first_name, salary, salary * 12 AS annual_salary
FROM employees;
```

Output:

first_name	salary	annual_salary
Alice	85000	1020000
Bob	95000	1140000
Carol	60000	720000

Example 5: Concatenate Columns in SELECT

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name, department
FROM employees;
```

Output:

full_name	department
Alice Kumar	Sales
Bob Singh	IT
Carol Rao	HR

TOPIC 7: Using Aliases — AS

Concept Explanation

An **alias** gives a column or table a **temporary name** in the output. It makes results more readable and is used heavily when:

- Renaming calculated columns
- Shortening long table names in JOINS
- Making output column names meaningful

Key Rules:

- Alias exists only for the duration of the query
- Use double quotes for aliases with spaces (PostgreSQL) or backticks (MySQL)
- Cannot use a column alias in the WHERE clause (runs before SELECT)
- CAN use a column alias in ORDER BY (runs after SELECT)

Syntax

```
-- Column Alias
SELECT column_name AS alias_name
FROM table_name;

-- Table Alias (very useful in JOINS)
SELECT t.column1, t.column2
FROM table_name AS t;

-- AS keyword is optional (but use it for clarity)
SELECT salary * 12 annual_salary FROM employees;    -- Works
SELECT salary * 12 AS annual_salary FROM employees; -- Better, more readable
```

Examples

Example 1: Rename a Column

```
SELECT first_name AS "Employee Name", salary AS "Monthly Salary"
FROM employees;
```

Output:

Employee Name	Monthly Salary
Alice	85000
Bob	95000
Carol	60000

Example 2: Alias for Calculated Column

```
SELECT
    first_name,
    salary,
```



```
salary * 12          AS annual_salary,
salary * 0.10        AS annual_bonus,
salary + salary*0.10 AS total_compensation
FROM employees;
```

Output:

first_name	salary	annual_salary	annual_bonus	total_compensation
Alice	85000	1020000	8500	93500
Bob	95000	1140000	9500	104500
Carol	60000	720000	6000	66000

Example 3: Table Alias (Shortening Table Name)

```
SELECT e.first_name, e.salary, e.department
FROM employees AS e;

-- Especially useful when same table appears multiple times
```

Example 4: Alias for CONCAT

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name,
    CONCAT(city, ', ', 'India')      AS location
FROM employees;
```

Output:

full_name	location
Alice Kumar	Mumbai, India
Bob Singh	Delhi, India

Example 5: Alias Used in ORDER BY

```
SELECT first_name, salary * 12 AS annual_salary
FROM employees
ORDER BY annual_salary DESC;  -- Using alias in ORDER BY ✔️

-- This works because ORDER BY runs AFTER SELECT
```

Output:

first_name	annual_salary
Eva	1560000
Frank	1320000
Bob	1140000

Example 6: Alias NOT Working in WHERE (Common Mistake!)

```
-- ❌ This will FAIL – WHERE runs before SELECT
SELECT first_name, salary * 12 AS annual_salary
FROM employees
WHERE annual_salary > 1000000;  -- ERROR: alias not recognized here

-- ✔️ Correct way – repeat the expression
SELECT first_name, salary * 12 AS annual_salary
FROM employees
WHERE salary * 12 > 1000000;
```

TOPIC 8: DISTINCT Keyword

Concept Explanation

DISTINCT removes **duplicate values** from the result set, returning only unique values.

Why this matters for Data Science:

- Count how many unique categories exist in a column
- Find unique customers who placed orders
- Identify all distinct values before encoding a categorical variable
- Part of EDA (Exploratory Data Analysis)

Syntax

```
-- Distinct values in one column
SELECT DISTINCT column_name
```

```
FROM table_name;

-- Distinct combinations of multiple columns
SELECT DISTINCT column1, column2
FROM table_name;

-- Distinct with other clauses
SELECT DISTINCT column_name
FROM table_name
WHERE condition
ORDER BY column_name;
```

Examples

Example 1: All Unique Departments

```
SELECT DISTINCT department
FROM employees;
```

Output:

department
Sales
IT
HR
Management
Finance

Without DISTINCT: Sales would appear 3 times (Alice, David, Isla), IT three times, etc.

Example 2: All Unique Cities

```
SELECT DISTINCT city
FROM employees;
```

Output:

city
Mumbai
Delhi
Bangalore
Chennai
Pune

Example 3: Unique Gender Values

```
SELECT DISTINCT gender
FROM employees;
```

Output:

gender
F
M

Example 4: Unique Combinations (Department + City)

```
SELECT DISTINCT department, city
FROM employees;
```

Output:

department	city
Sales	Mumbai
IT	Delhi
HR	Bangalore
Management	Delhi
HR	Chennai
Finance	Pune
IT	Bangalore

Note: Each unique combination of department + city is shown once.

Example 5: Unique Order Statuses

```
SELECT DISTINCT status
FROM orders;
```

Output:

status
Delivered
Pending
Cancelled

Example 6: Unique Countries from Customers

```
SELECT DISTINCT country
FROM customers
ORDER BY country;
```

Output:

country
China
India
Spain
UK
USA

Example 7: Count of Unique Departments (using COUNT + DISTINCT)

```
SELECT COUNT(DISTINCT department) AS total_departments
FROM employees;
```

Output:

total_departments
5

Example 8: Distinct Customers Who Have Placed Orders

```
SELECT DISTINCT customer_id
FROM orders
ORDER BY customer_id;
```

Output:

customer_id
1
2
3
4
5
6

Example 9: DISTINCT vs No DISTINCT Comparison

```
-- Without DISTINCT (shows duplicates)
SELECT department FROM employees;
-- Output: Sales, IT, HR, Sales, Management, IT, HR, IT, Sales, Finance
-- (10 rows, with duplicates)

-- With DISTINCT (only unique values)
SELECT DISTINCT department FROM employees;
-- Output: Sales, IT, HR, Management, Finance
-- (5 rows, no duplicates)
```

Example 10: DISTINCT for Data Exploration (EDA Use Case)

```
-- Before building an ML model, check unique values in categorical columns
SELECT DISTINCT gender      FROM employees; -- How many gender categories?
SELECT DISTINCT department FROM employees; -- How many departments?
SELECT DISTINCT status      FROM orders;    -- How many order states?
SELECT DISTINCT country     FROM customers;  -- How many countries?
```

This is how Data Scientists check cardinality of categorical columns before encoding them for ML.

Practice Questions

Questions cover all topics in Phase 1. Solutions are in the next section.

● Easy Level

- Q1.** Write a query to view all data in the products table.
- Q2.** Write a query to show only name, price, and stock from the products table.
- Q3.** Write a query to show all unique values in the category column of the products table.
- Q4.** Write a query to select customer name and country, giving the column name an alias of "Customer Name" and country an alias of "Country".
- Q5.** Write a query to show all distinct status values from the orders table.
- Q6.** What data type would you use for:
- A product ID (whole number, up to 9999)
 - A product price (up to ₹99,999.99)
 - A product description (long text)
 - A "is_available" flag (yes/no)
 - An order timestamp (date and time)
- Q7.** Write the SQL to create a products table with: product_id (PK), name (not null), category, price (decimal, not null), stock (default 0).
- Q8.** Write an INSERT to add this product: Laptop, Electronics, 75000.00, stock = 50.

● Medium Level

- Q9.** Write a query to display first_name, last_name, salary, and salary * 1.15 aliased as "Salary After Raise".
- Q10.** Write a query to show unique department, city combinations from the employees table.
- Q11.** How many unique departments are there in the employees table? Write the query.
- Q12.** Write a query to display full name (first + last name combined) as full_name, department, and city from employees.
- Q13.** Write an INSERT statement to add 3 new customers at once into the customers table.
- Q14.** Write a query to calculate: emp_id, full_name, salary, salary * 12 as annual_salary, and salary * 12 * 0.18 as tax_amount.
- Q15.** Write SQL to create an orders table that:
- Has order_id as primary key
 - Has customer_id as a foreign key referencing customers
 - Has amount that cannot be negative (use CHECK constraint)
 - Has order_date that defaults to today's date
 - Has status that defaults to 'Pending'

● Hard / Interview Level

- Q16.** A table has 1 million rows. A junior analyst writes SELECT * FROM sales. What's wrong with this? How would you fix it?
- Q17.** Explain why this query fails:

```
SELECT first_name, salary * 12 AS annual_salary
FROM employees
WHERE annual_salary > 1000000;
```

How do you fix it?

- Q18.** What is the difference between DELETE, DROP, and TRUNCATE?

Operation	What it does	Can be rolled back?	Resets AUTO_INCREMENT?
DELETE			
DROP			
TRUNCATE			

Fill in the table.

- Q19.** A company's database has:
- users table: 10 million rows
 - purchases table: 50 million rows
- Why is SELECT DISTINCT user_id FROM purchases potentially very slow? What optimizations can help?

- Q20.** You need to create a table to track ML model experiments with columns: model name, version, accuracy (0.0 to 1.0), training date, is_deployed flag, and notes. Write the full CREATE TABLE statement with appropriate data types and constraints.

Solutions

- Q1.**

```
SELECT * FROM products;
```

Q2.

```
SELECT name, price, stock
FROM products;
```

Q3.

```
SELECT DISTINCT category FROM products;
```

Q4.

```
SELECT name AS "Customer Name", country AS "Country"
FROM customers;
```

Q5.

```
SELECT DISTINCT status FROM orders;
```

Q6.

```
product_id → SMALLINT or INTproduct_price → DECIMAL(10, 2)description → TEXTis_available → BOOLEANorder_timestamp → DATETIME or TIMESTAMP
```

Q7.

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    price DECIMAL(10,2) NOT NULL,
    stock INT DEFAULT 0
);
```

Q8.

```
INSERT INTO products (product_id, name, category, price, stock)
VALUES (1, 'Laptop', 'Electronics', 75000.00, 50);
```

Q9.

```
SELECT
    first_name,
    last_name,
    salary,
    salary * 1.15 AS "Salary After Raise"
FROM employees;
```

Q10.

```
SELECT DISTINCT department, city
FROM employees;
```

Q11.

```
SELECT COUNT(DISTINCT department) AS unique_departments
FROM employees;
-- Output: 5
```

Q12.

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name,
    department,
    city
FROM employees;
```

Q13.

```
INSERT INTO customers (customer_id, name, email, city, country, age, signup_date)
VALUES
    (8, 'Amit Shah', 'amit@email.com', 'Ahmedabad', 'India', 34, '2024-01-01'),
    (9, 'Neha Roy', 'neha@email.com', 'Kolkata', 'India', 29, '2024-01-15'),
    (10, 'Alex Brown', 'alex@email.com', 'Sydney', 'Australia', 41, '2024-02-01');
```

Q14.

```
SELECT
    emp_id,
    CONCAT(first_name, ' ', last_name) AS full_name,
    salary,
```

```
salary * 12 AS annual_salary,
ROUND(salary * 12 * 0.18, 2) AS tax_amount
FROM employees;
```

Q15.

```
CREATE TABLE orders (
  order_id      INT          PRIMARY KEY,
  customer_id   INT          NOT NULL,
  amount        DECIMAL(12,2) NOT NULL CHECK (amount >= 0),
  order_date    DATE          DEFAULT CURRENT_DATE,
  status        VARCHAR(20)   DEFAULT 'Pending',

  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Q16.

```
Problem: SELECT * fetches ALL columns from 1 million rows.
- Wastes network bandwidth (slow data transfer)
- Loads unnecessary data into memory
- Never use SELECT * in production queries

Fix:
SELECT order_id, customer_id, amount, order_date
FROM sales
WHERE order_date >= '2024-01-01'
LIMIT 100;
```

Q17.

```
Reason: SQL execution order is:
FROM → WHERE → SELECT → ORDER BY
WHERE runs BEFORE SELECT, so the alias 'annual_salary' doesn't exist yet when WHERE
evaluates.

Fix:
SELECT first_name, salary * 12 AS annual_salary
FROM employees
WHERE salary * 12 > 1000000;
```

Q18.

Operation	What it does	Can be rolled back?	Resets AUTO_INCREMENT?
DELETE	Removes specific rows (or all if no WHERE)	YES (if in transaction)	NO
DROP	Permanently deletes entire table + structure	NO	YES (table gone)
TRUNCATE	Removes ALL rows, keeps table structure	NO (usually)	YES

Q19.

```
Reason: DISTINCT on 50M rows requires:
1. Full table scan (read all 50M rows)
2. Sort or hash operation to find unique values
This is I/O intensive and memory-heavy.

Optimizations:
1. Add an index on user_id column
2. Use approximate distinct count functions (HyperLogLog in some DBs)
3. Filter with WHERE before applying DISTINCT
4. Use EXPLAIN/EXPLAIN ANALYZE to check query plan
```

Q20.

```
CREATE TABLE ml_experiments (
  experiment_id INT          AUTO_INCREMENT PRIMARY KEY,
  model_name     VARCHAR(100) NOT NULL,
  version        VARCHAR(20)  NOT NULL,
  accuracy       DECIMAL(5,4) CHECK (accuracy BETWEEN 0 AND 1),
  training_date  DATE          DEFAULT CURRENT_DATE,
  is_deployed    BOOLEAN      DEFAULT FALSE,
  notes          TEXT,

  UNIQUE (model_name, version) -- Same model can't have duplicate versions
);
```

Real-World Use Cases

Topic	Data Science Application
Database Tables	Raw data from APIs, CSVs, and scrapers land in structured tables
Data Types	Choosing correct types ensures no encoding errors before ML preprocessing
CREATE TABLE	Data engineers create tables for feature stores and training datasets

Topic	Data Science Application
INSERT INTO	ETL pipelines insert transformed records into analytics databases
SELECT *	Quick data exploration (but avoid in production queries)
SELECT columns	Extract only required features for model training
Aliases	Rename features to meaningful names in analytical queries
DISTINCT	EDA: check cardinality of categorical variables before one-hot encoding

Phase 2: Filtering with WHERE

 **Dataset Reminder:** We use the same `employees`, `customers`, `orders`, `products` tables defined in Phase 1.

TOPIC 1: WHERE Clause Basics

Concept Explanation

The `WHERE` clause **filters rows** based on a condition. Only rows where the condition is `TRUE` are returned.

Why it's critical for Data Science:

- Extract specific subsets of data (e.g., churn customers, high-value transactions)
- Apply business logic filters before analysis
- Reduce dataset size before loading into Python/pandas

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

How it works:

```
All rows in table
      ↓
WHERE condition evaluated for each row
      ↓
TRUE  → Row is included in result
FALSE → Row is excluded from result
NULL  → Row is excluded from result (NULL is NOT equal to TRUE)
```

Examples

Example 1: Filter by Department

```
SELECT first_name, last_name, department, salary
FROM employees
WHERE department = 'Sales';
```

Output:

first_name	last_name	department	salary
Alice	Kumar	Sales	85000
David	Mehta	Sales	75000
Isla	Patel	Sales	72000

Example 2: Filter by Salary Threshold

```
SELECT first_name, salary
FROM employees
WHERE salary > 90000;
```

Output:

first_name	salary
Bob	95000
Eva	130000
Frank	110000

Example 3: Filter Orders by Status

```
SELECT order_id, customer_id, amount, status
FROM orders
WHERE status = 'Delivered';
```

Output:

order_id	customer_id	amount	status
101	1	150000	Delivered
103	1	75000	Delivered
105	4	75000	Delivered
107	6	30000	Delivered
108	2	15000	Delivered

Example 4: Filter Customers by Country

```
SELECT name, city, country
FROM customers
WHERE country = 'India';
```

Output:

name	city	country
Ravi Desai	Mumbai	India
Priya Menon	Delhi	India
Meera Iyer	Bangalore	India

Example 5: Filter Products by Stock

```
SELECT name, price, stock
FROM products
WHERE stock > 100;
```

Output:

name	price	stock
Smartphone	30000	200
Headphones	5000	500

TOPIC 2: Comparison Operators

Concept Explanation

Comparison operators compare a column value to a given value and return TRUE or FALSE.

Operators Table

Operator	Meaning	Example
=	Equal to	salary = 85000
!= or <>	Not equal to	gender != 'M' or gender <> 'M'
>	Greater than	salary > 80000
<	Less than	age < 30
>=	Greater than or equal to	salary >= 85000
<=	Less than or equal to	age <= 35

Examples

Example 1: Equal To (=)

```
SELECT first_name, department
FROM employees
WHERE department = 'IT';
```

Output:

first_name	department
Bob	IT
Frank	IT
Henry	IT

Example 2: Not Equal To (!=)

```
SELECT first_name, department
FROM employees
WHERE department != 'IT';
```

Output:

first_name	department
Alice	Sales
Carol	HR
David	Sales
Eva	Management
Grace	HR
Isla	Sales

first_name	department
Jake	Finance

Example 3: Greater Than (>)

```
SELECT first_name, age
FROM employees
WHERE age > 35;
```

Output:

first_name	age
David	40
Eva	45
Frank	38

Example 4: Less Than (<)

```
SELECT first_name, age
FROM employees
WHERE age < 30;
```

Output:

first_name	age
Carol	28
Grace	26
Isla	29

Example 5: Greater Than or Equal (>=)

```
SELECT first_name, salary
FROM employees
WHERE salary >= 85000;
```

Output:

first_name	salary
Alice	85000
Bob	95000
Eva	130000
Frank	110000
Henry	88000

Example 6: Less Than or Equal (<=)

```
SELECT first_name, salary
FROM employees
WHERE salary <= 60000;
```

Output:

first_name	salary
Carol	60000
Grace	55000

Example 7: Comparing Dates (works like numbers)

```
SELECT first_name, hire_date
FROM employees
WHERE hire_date > '2020-01-01';
```

Output:

first_name	hire_date
Alice	2020-01-15
Carol	2021-03-20
Grace	2022-08-01
Isla	2021-11-30
Jake	2020-05-14

Example 8: Comparing String Values Alphabetically

```
SELECT first_name
FROM employees
WHERE first_name > 'E';    -- Names starting after 'E' alphabetically
```

Output:

first_name
Frank

Grace
Henry
Isla
Jake

Example 9: Orders with Amount >= 50000

```
SELECT order_id, customer_id, amount
FROM orders
WHERE amount >= 50000;
```

Output:

order_id	customer_id	amount
101	1	150000
103	1	75000
105	4	75000
106	5	150000

Example 10: Using <> (Alternative to !=)

```
SELECT name, status
FROM orders
WHERE status <> 'Cancelled';
```

Output: All orders except Cancelled ones (Delivered and Pending).

TOPIC 3: Logical Operator — AND

Concept Explanation

AND combines two or more conditions. **ALL conditions must be TRUE** for a row to be included.

Syntax

```
SELECT columns
FROM table
WHERE condition1 AND condition2 AND condition3;
```

Truth Table:

Condition 1	Condition 2	Result
TRUE	TRUE	TRUE 
TRUE	FALSE	FALSE 
FALSE	TRUE	FALSE 
FALSE	FALSE	FALSE 

Examples

Example 1: Sales dept AND Salary > 75000

```
SELECT first_name, department, salary
FROM employees
WHERE department = 'Sales' AND salary > 75000;
```

Output:

first_name	department	salary
Alice	Kumar	85000

Example 2: Female AND IT Department

```
SELECT first_name, gender, department
FROM employees
WHERE gender = 'F' AND department = 'IT';
```

Output: *(no rows — no female IT employees in our dataset)*

first_name	gender	department
(no rows)		

Example 3: Age > 30 AND Salary > 80000

```
SELECT first_name, age, salary
FROM employees
```

```
WHERE age > 30 AND salary > 80000;
```

Output:

first_name	age	salary
Bob	35	95000
David	40	75000
Eva	45	130000
Frank	38	110000
Henry	33	88000

Actual Output (salary > 80000 AND age > 30):

first_name	age	salary
Bob	35	95000
Eva	45	130000
Frank	38	110000
Henry	33	88000

Example 4: Orders: Delivered AND Amount > 50000

```
SELECT order_id, amount, status
FROM orders
WHERE status = 'Delivered' AND amount > 50000;
```

Output:

order_id	amount	status
101	150000	Delivered
103	75000	Delivered
105	75000	Delivered

Example 5: Three AND Conditions

```
SELECT first_name, department, city, salary
FROM employees
WHERE department = 'Sales' AND city = 'Mumbai' AND salary > 70000;
```

Output:

first_name	department	city	salary
Alice	Sales	Mumbai	85000
David	Sales	Mumbai	75000

Example 6: Customers from India AND Signed Up After 2022

```
SELECT name, country, signup_date
FROM customers
WHERE country = 'India' AND signup_date > '2022-01-01';
```

Output:

name	country	signup_date
Priya Menon	India	2022-03-15

TOPIC 4: Logical Operator — OR

Concept Explanation

OR combines conditions where **ANY ONE condition being TRUE** is enough to include the row.

Syntax

```
SELECT columns
FROM table
WHERE condition1 OR condition2;
```

Truth Table:

Condition 1	Condition 2	Result
TRUE	TRUE	TRUE 
TRUE	FALSE	TRUE 
FALSE	TRUE	TRUE 
FALSE	FALSE	FALSE 

Examples

Example 1: Sales OR IT Department

```
SELECT first_name, department
FROM employees
WHERE department = 'Sales' OR department = 'IT';
```

Output:

first_name	department
Alice	Sales
Bob	IT
David	Sales
Frank	IT
Henry	IT
Isla	Sales

Example 2: Salary < 60000 OR Salary > 100000

```
SELECT first_name, salary
FROM employees
WHERE salary < 60000 OR salary > 100000;
```

Output:

first_name	salary
Eva	130000
Frank	110000
Grace	55000

Example 3: Mumbai OR Delhi

```
SELECT first_name, city
FROM employees
WHERE city = 'Mumbai' OR city = 'Delhi';
```

Output:

first_name	city
Alice	Mumbai
Bob	Delhi
David	Mumbai
Eva	Delhi
Henry	Delhi
Isla	Mumbai

Example 4: Orders — Pending OR Cancelled

```
SELECT order_id, customer_id, amount, status
FROM orders
WHERE status = 'Pending' OR status = 'Cancelled';
```

Output:

order_id	customer_id	amount	status
102	2	30000	Pending
104	3	9000	Cancelled
106	5	150000	Pending

Example 5: Age < 28 OR Age > 40

```
SELECT first_name, age
FROM employees
WHERE age < 28 OR age > 40;
```

Output:

first_name	age
Eva	45
Grace	26

TOPIC 5: Logical Operator — NOT

Concept Explanation

NOT **negates** (reverses) a condition. Rows where the condition is originally TRUE are excluded.

Syntax

```
SELECT columns
FROM table
WHERE NOT condition;
```

Examples

Example 1: NOT in Sales Department

```
SELECT first_name, department
FROM employees
WHERE NOT department = 'Sales';

-- Equivalent to:
WHERE department != 'Sales'
WHERE department <> 'Sales'
```

Output:

first_name	department
Bob	IT
Carol	HR
Eva	Management
Frank	IT
Grace	HR
Henry	IT
Jake	Finance

Example 2: NOT Delivered Orders

```
SELECT order_id, status
FROM orders
WHERE NOT status = 'Delivered';
```

Output:

order_id	status
102	Pending
104	Cancelled
106	Pending

Example 3: NOT Female Employees

```
SELECT first_name, gender
FROM employees
WHERE NOT gender = 'F';
```

Output: Bob, David, Frank, Henry, Jake (all males)

Example 4: NOT Mumbai

```
SELECT first_name, city
FROM employees
WHERE NOT city = 'Mumbai';
```

Output: All employees NOT in Mumbai.

Example 5: Combining NOT with AND

```
SELECT first_name, department, salary
FROM employees
WHERE NOT department = 'HR' AND salary > 70000;
```

Output: Employees not in HR with salary > 70000.

Example 6: NOT BETWEEN (Covered more in BETWEEN topic)

```
SELECT first_name, salary
FROM employees
WHERE NOT (salary BETWEEN 70000 AND 90000);

-- Same as:
WHERE salary < 70000 OR salary > 90000
```

TOPIC 6: BETWEEN Operator

Concept Explanation

BETWEEN filters rows where a column value falls **within a range** (inclusive on both ends).

NOT BETWEEN excludes the range.

Works with: Numbers, Dates, Text (alphabetical)

Syntax

```
SELECT columns
FROM table
WHERE column BETWEEN lower_value AND upper_value;

-- BETWEEN is INCLUSIVE – both boundary values ARE included
-- Equivalent to: column >= lower_value AND column <= upper_value
```

Examples

Example 1: Salary Between 70000 and 90000

```
SELECT first_name, salary
FROM employees
WHERE salary BETWEEN 70000 AND 90000;
```

Output:

first_name	salary
Alice	85000
David	75000
Henry	88000
Isla	72000
Jake	82000

Note: 70000 and 90000 are INCLUDED (BETWEEN is inclusive)

Example 2: Age Between 28 and 35

```
SELECT first_name, age
FROM employees
WHERE age BETWEEN 28 AND 35;
```

Output:

first_name	age
Alice	30
Bob	35
Carol	28
Henry	33
Isla	29
Jake	31

Example 3: Hire Date Between 2019 and 2021

```
SELECT first_name, hire_date
FROM employees
WHERE hire_date BETWEEN '2019-01-01' AND '2021-12-31';
```

Output:

first_name	hire_date
Alice	2020-01-15
Carol	2021-03-20
Henry	2019-04-22
Isla	2021-11-30
Jake	2020-05-14

Example 4: Orders Between Amount Range

```
SELECT order_id, amount
FROM orders
WHERE amount BETWEEN 20000 AND 100000;
```

Output:

order_id	amount
102	30000
103	75000
105	75000
107	30000

Example 5: NOT BETWEEN


```
SELECT first_name, salary
FROM employees
WHERE salary NOT BETWEEN 70000 AND 90000;
```

Output:

first_name	salary
Bob	95000
Carol	60000
Eva	130000
Frank	110000
Grace	55000

Example 6: BETWEEN with Text (Alphabetical)

```
SELECT first_name
FROM employees
WHERE first_name BETWEEN 'A' AND 'E';
```

Output:

first_name
Alice
Bob
Carol
David

Names starting with A through D (before 'E' alphabetically)

Example 7: Product Price Range

```
SELECT name, price
FROM products
WHERE price BETWEEN 5000 AND 30000;
```

Output:

name	price
Smartphone	30000
Headphones	5000
Monitor	15000

Example 8: BETWEEN Equivalent with AND

```
-- These two are IDENTICAL:
SELECT * FROM employees WHERE salary BETWEEN 70000 AND 90000;
SELECT * FROM employees WHERE salary >= 70000 AND salary <= 90000;
```

TOPIC 7: IN Operator

Concept Explanation

IN checks if a column value **matches any value in a list**. It replaces multiple **OR** conditions.

NOT IN excludes rows matching any value in the list.

When to use IN:

- Filtering by multiple specific values
- Replacing long chains of OR conditions
- When filtering on results from a subquery (covered in Phase 6)

Syntax

```
SELECT columns
FROM table
WHERE column IN (value1, value2, value3, ...);

-- Equivalent to:
WHERE column = value1 OR column = value2 OR column = value3;
```

Examples

Example 1: Employees in Specific Departments

```
SELECT first_name, department
FROM employees
WHERE department IN ('Sales', 'IT', 'HR');
```

Output:

first_name	department
Alice	Sales
Bob	IT
Carol	HR
David	Sales
Frank	IT
Grace	HR
Henry	IT
Isla	Sales

Example 2: Employees in Specific Cities

```
SELECT first_name, city
FROM employees
WHERE city IN ('Mumbai', 'Delhi');
```

Output:

first_name	city
Alice	Mumbai
Bob	Delhi
David	Mumbai
Eva	Delhi
Henry	Delhi
Isla	Mumbai

Example 3: NOT IN — Exclude Departments

```
SELECT first_name, department
FROM employees
WHERE department NOT IN ('HR', 'Finance', 'Management');
```

Output: Employees in Sales and IT only.

Example 4: Specific Order IDs

```
SELECT order_id, customer_id, amount
FROM orders
WHERE order_id IN (101, 103, 105, 107);
```

Output:

order_id	customer_id	amount
101	1	150000
103	1	75000
105	4	75000
107	6	30000

Example 5: Multiple Statuses

```
SELECT order_id, status
FROM orders
WHERE status IN ('Pending', 'Cancelled');
```

Output:

order_id	status
102	Pending
104	Cancelled
106	Pending

Example 6: Specific Customer IDs

```
SELECT name, city
FROM customers
WHERE customer_id IN (1, 3, 5, 7);
```

Output:

name	city
Ravi Desai	Mumbai
John Smith	New York
Chen Wei	Shanghai
Carlos Ruiz	Madrid

Example 7: IN vs OR (Both identical results)


```
-- Using OR (verbose)
SELECT first_name FROM employees
WHERE department = 'Sales' OR department = 'IT' OR department = 'HR';

-- Using IN (clean)
SELECT first_name FROM employees
WHERE department IN ('Sales', 'IT', 'HR');

-- Both return same results. IN is preferred for readability.
```

Example 8: NOT IN Pitfall with NULLs (Important!)

```
-- ⚠ If the list contains NULL, NOT IN returns no results!
-- This is a VERY common bug in SQL

SELECT first_name FROM employees
WHERE department NOT IN ('HR', NULL);
-- Returns ZERO rows! Because NULL comparisons are always UNKNOWN

-- ✅ Safe approach: Filter NULLs separately
SELECT first_name FROM employees
WHERE department NOT IN ('HR')
  AND department IS NOT NULL;
```

TOPIC 8: LIKE Operator — Pattern Matching

Concept Explanation

LIKE is used to search for **patterns** in string columns using **wildcard characters**.

Wildcards:

Wildcard	Meaning	Example
	Any sequence of zero or more characters	<code>'A%'</code> → starts with A
	Exactly one character	<code>'_ob'</code> → Bob, Rob, Job

Use Cases:

- Search by partial name, email domain
- Find records with specific patterns (codes, formats)
- Data cleaning: find messy/inconsistent entries

Syntax

```
SELECT columns
FROM table
WHERE column LIKE 'pattern';

-- Case sensitivity depends on database:
-- MySQL: LIKE is case-insensitive by default
-- PostgreSQL: LIKE is case-sensitive; use ILIKE for case-insensitive
```

Examples

Example 1: Names Starting with 'A'

```
SELECT first_name FROM employees
WHERE first_name LIKE 'A%';
```

Output:

first_name

Alice

Example 2: Names Ending with 'e'

```
SELECT first_name FROM employees
WHERE first_name LIKE '%e';
```

Output:

first_name

Alice

Grace

Example 3: Names Containing 'a' (Anywhere)

```
SELECT first_name FROM employees
WHERE first_name LIKE '%a%';
```

Output:

first_name
Alice
Carol
Eva
Frank
Grace
Jake

Example 4: Exactly 4 Characters Long

```
SELECT first_name FROM employees
WHERE first_name LIKE '____'; -- 4 underscores
```

Output:

first_name
Isla
Jake

Example 5: Name Starts with 'B' and Exactly 3 Characters

```
SELECT first_name FROM employees
WHERE first_name LIKE 'B__'; -- B + exactly 2 more chars
```

Output:

first_name
Bob

Example 6: Email Domains

```
SELECT name, email FROM customers
WHERE email LIKE '%@email.com';
```

Output: All customers with @email.com domain.

Example 7: Gmail Users

```
-- Find customers using Gmail
SELECT name, email FROM customers
WHERE email LIKE '%@gmail.com';
-- None in our dataset, but this is a very common real-world query
```

Example 8: NOT LIKE

```
SELECT first_name FROM employees
WHERE first_name NOT LIKE 'A%'; -- Not starting with A
```

Output: Bob, Carol, David, Eva, Frank, Grace, Henry, Isla, Jake

Example 9: City Names Starting with 'D'

```
SELECT first_name, city FROM employees
WHERE city LIKE 'D%';
```

Output:

first_name	city
Bob	Delhi
Eva	Delhi
Henry	Delhi

Example 10: Find Employees with 'ar' in Last Name

```
SELECT first_name, last_name FROM employees
WHERE last_name LIKE '%ar%';
```

Output:

first_name	last_name
Eva	Sharma

Example 11: Second Character is 'a'

```
SELECT first_name FROM employees
WHERE first_name LIKE '_a%';  -- Any char, then 'a', then anything
```

Output:

first_name
Carol
David
Grace
Jake

Example 12: ILIKE — Case Insensitive (PostgreSQL)

```
-- PostgreSQL: case-insensitive search
SELECT first_name FROM employees
WHERE first_name ILIKE 'alice';  -- Finds 'Alice', 'ALICE', 'alice'

-- MySQL: LIKE is already case-insensitive by default
SELECT first_name FROM employees
WHERE first_name LIKE 'alice';   -- Also finds 'Alice' in MySQL
```

Example 13: Escaping Wildcard Characters

```
-- What if you need to search for a literal '%' or '_'?
-- Use ESCAPE clause
SELECT product_name FROM products
WHERE product_name LIKE '10\\%' ESCAPE '\\';
-- Finds products named literally '10%'
```

Example 14: Combining LIKE with AND

```
SELECT first_name, department, city
FROM employees
WHERE first_name LIKE 'A%'
      AND city LIKE 'M%';
```

Output:

first_name	department	city
Alice	Sales	Mumbai

Example 15: LIKE for Data Cleaning (Finding Bad Data)

```
-- Find email addresses without @ (invalid emails)
SELECT name, email FROM customers
WHERE email NOT LIKE '%@%';

-- Find phone numbers not in standard format
SELECT * FROM contacts
WHERE phone NOT LIKE '____-____-____';
```

Practice Questions

All questions use the standard dataset from Phase 1.

● Easy Level

- Q1. Write a query to get all employees in the `HR` department.
- Q2. Write a query to show all customers from `India`.
- Q3. Write a query to find all products with a price less than `10000`.
- Q4. Write a query to show all orders with status `Delivered`.

- Q5.** Write a query to find all employees whose salary is exactly `85000` .
- Q6.** Write a query to find all employees NOT in the `Sales` department.
- Q7.** Write a query to find all customers older than `30` .
- Q8.** Write a query to find all employees whose names start with `'A'` .

 Medium Level

- Q9.** Write a query to find all employees in the `IT` department with a salary greater than `90000` .
- Q10.** Write a query to find employees in `Mumbai` OR `Bangalore` .
- Q11.** Write a query to find all orders with amount between `30000` and `100000` .
- Q12.** Write a query to find all employees in departments `Sales` , `IT` , and `Finance` .
- Q13.** Write a query to find all employees hired between `2019-01-01` and `2021-12-31` .
- Q14.** Write a query to find all employees whose last name contains the letter `'a'` .
- Q15.** Write a query to find all employees who are NOT in `HR` , `Finance` , or `Management` .
- Q16.** Write a query to find all customers whose email ends with `@email.com` .

 Hard / Interview Level

Q17. Write a query to find all employees who:

- Work in `Sales` or `IT`
- Have a salary greater than `80000`
- Were hired after `2019-01-01`

Q18. Write a query to find all orders that:

- Are NOT `Cancelled`
- Have amount greater than `25000`
- Were placed in January 2024

Q19. Explain the difference between these two queries and what each returns:

```
-- Query A
SELECT * FROM employees WHERE salary BETWEEN 70000 AND 90000;

-- Query B
SELECT * FROM employees WHERE salary > 70000 AND salary < 90000;
```

Q20. A customer support analyst needs to find all customers whose name contains "Kumar" OR "Singh". Write the query using:

- LIKE operator approach
- IN operator approach (if possible)
- Which approach is better and why?

Q21. Write a query to find employees where:

- Salary is NOT between 60000 and 90000
- Department is NOT in ('HR', 'Management')
- City starts with either 'M' or 'D'

Q22. Explain why this query might return unexpected results:

```
SELECT * FROM employees WHERE department NOT IN ('Sales', NULL);
```

Q23. A Data Scientist wants to find all customers from countries that are 5 characters long (like India, China, Spain). Write the query using `LIKE` with `_` wildcard.

Solutions

Q1.

```
SELECT * FROM employees
WHERE department = 'HR';
```

Q2.

```
SELECT * FROM customers
WHERE country = 'India';
```

Q3.

```
SELECT name, price FROM products
WHERE price < 10000;
```

Q4.

```
SELECT * FROM orders
WHERE status = 'Delivered';
```

Q5.

```
SELECT first_name, salary FROM employees
```

```
WHERE salary = 85000;
```

Q6.

```
SELECT first_name, department FROM employees
WHERE department != 'Sales';
-- OR: WHERE NOT department = 'Sales'
```

Q7.

```
SELECT name, age FROM customers
WHERE age > 30;
```

Q8.

```
SELECT first_name FROM employees
WHERE first_name LIKE 'A%';
```

Q9.

```
SELECT first_name, department, salary
FROM employees
WHERE department = 'IT' AND salary > 90000;
```

Output:

first_name	department	salary
Bob	IT	95000
Frank	IT	110000

Q10.

```
SELECT first_name, city FROM employees
WHERE city = 'Mumbai' OR city = 'Bangalore';
-- OR: WHERE city IN ('Mumbai', 'Bangalore')
```

Q11.

```
SELECT order_id, amount FROM orders
WHERE amount BETWEEN 30000 AND 100000;
```

Q12.

```
SELECT first_name, department FROM employees
WHERE department IN ('Sales', 'IT', 'Finance');
```

Q13.

```
SELECT first_name, hire_date FROM employees
WHERE hire_date BETWEEN '2019-01-01' AND '2021-12-31';
```

Q14.

```
SELECT first_name, last_name FROM employees
WHERE last_name LIKE '%a%';
```

Q15.

```
SELECT first_name, department FROM employees
WHERE department NOT IN ('HR', 'Finance', 'Management');
```

Q16.

```
SELECT name, email FROM customers
WHERE email LIKE '%@email.com';
```

Q17.

```
SELECT first_name, department, salary, hire_date
FROM employees
WHERE department IN ('Sales', 'IT')
  AND salary > 80000
  AND hire_date > '2019-01-01';
```

Output:

first_name	department	salary	hire_date
Alice	Sales	85000	2020-01-15
Henry	IT	88000	2019-04-22

Q18.

```
SELECT order_id, amount, order_date, status
FROM orders
```

```
WHERE status != 'Cancelled'
  AND amount > 25000
  AND order_date BETWEEN '2024-01-01' AND '2024-01-31';
```

Q19.

Query A: WHERE salary BETWEEN 70000 AND 90000
→ Includes 70000 and 90000 (BOTH endpoints INCLUDED)
→ Returns: 70000, 72000, 75000, 82000, 85000, 88000, 90000 would all match

Query B: WHERE salary > 70000 AND salary < 90000
→ EXCLUDES both 70000 and 90000 (strict inequality)
→ 70000 would NOT be returned; 90000 would NOT be returned

Key Difference: BETWEEN is inclusive; > and < are exclusive of boundaries.

Q20.

```
-- LIKE approach:
SELECT name FROM customers
WHERE name LIKE '%Kumar%' OR name LIKE '%Singh%';

-- IN approach: NOT possible with IN (IN checks exact match, not patterns)
-- IN requires exact values: WHERE name IN ('Kumar', 'Singh')
-- This would only match if name is EXACTLY 'Kumar' or 'Singh'

-- Better approach: LIKE is correct here because we need partial matching.
-- If searching for exact last names you could split first/last name columns.
```

Q21.

```
SELECT first_name, department, salary, city
FROM employees
WHERE salary NOT BETWEEN 60000 AND 90000
  AND department NOT IN ('HR', 'Management')
  AND (city LIKE 'M%' OR city LIKE 'D%');
```

Output:

first_name	department	salary	city
Bob	IT	95000	Delhi
Frank	IT	110000	Bangalore

```
-- Actual output (city LIKE 'M%' OR LIKE 'D%'):
-- Bob - IT - 95000 - Delhi ✓
-- Frank - IT - 110000 - Bangalore ✗ (not M or D)
```

Actual Output:

first_name	department	salary	city
-----	-----	-----	-----
Bob	IT	95000	Delhi

Q22.

The NOT IN ('Sales', NULL) issue:

When you use NOT IN with a list containing NULL:
SQL internally evaluates: department != 'Sales' AND department != NULL

But any comparison with NULL returns UNKNOWN (not TRUE or FALSE).
UNKNOWN AND anything = UNKNOWN (not TRUE)

So no row ever satisfies the condition – query returns ZERO rows!

Fix:

```
WHERE department != 'Sales' OR department IS NULL
OR
WHERE department NOT IN ('Sales') AND department IS NOT NULL
```

Q23.

```
SELECT name, country FROM customers
WHERE country LIKE '_____'; -- 5 underscores = exactly 5 characters
```

Output:

name	country
Priya Menon	India
Chen Wei	China
Carlos Ruiz	Spain

 Real-World Use Cases

Operator	Data Science Application
WHERE	Extract specific time ranges, segments, or user cohorts for analysis
AND	Combine multiple business rules (active users AND high spenders)
OR	Include multiple valid categories (churn OR inactive)
NOT	Exclude test accounts, internal users, cancelled records
BETWEEN	Date range slicing for time-series analysis, age/income binning
IN	Filter by list of IDs from another system, select specific experiment groups
LIKE	Clean dirty data, find pattern-based records, email/phone validation

Phase 3: Sorting & Limiting

TOPIC 1: ORDER BY — Single Column

Concept Explanation

ORDER BY sorts the results of a query in **ascending** (A → Z, 0 → 9) or **descending** (Z → A, 9 → 0) order.

Why it matters for Data Science:

- Rank employees, products, or customers by metrics
- Find top-N or bottom-N records
- Sort time-series data chronologically for analysis

Key Rules:

- ASC** (ascending) is the **default** — no need to write it explicitly
- DESC** (descending) must be explicitly specified
- ORDER BY** runs **AFTER SELECT** → you CAN use column aliases
- NULL values sort to the END in ASC and to the BEGINNING in DESC (default behavior)

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column_name [ASC | DESC];
```

Examples

Example 1: Sort by Salary (Ascending — Lowest First)

```
SELECT first_name, salary
FROM employees
ORDER BY salary ASC;

-- Or simply (ASC is default):
ORDER BY salary;
```

Output:

first_name	salary
Grace	55000
Carol	60000
Isla	72000
David	75000
Jake	82000
Alice	85000
Henry	88000
Bob	95000
Frank	110000
Eva	130000

Example 2: Sort by Salary (Descending — Highest First)

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC;
```

Output:

first_name	salary
Eva	130000
Frank	110000

first_name	salary
Bob	95000
Henry	88000
Alice	85000
Jake	82000
David	75000
Isla	72000
Carol	60000
Grace	55000

Example 3: Sort by Name Alphabetically

```
SELECT first_name, department
FROM employees
ORDER BY first_name ASC;
```

Output:

first_name	department
Alice	Sales
Bob	IT
Carol	HR
David	Sales
Eva	Management
Frank	IT
Grace	HR
Henry	IT
Isla	Sales
Jake	Finance

Example 4: Sort by Hire Date (Most Recent First)

```
SELECT first_name, hire_date
FROM employees
ORDER BY hire_date DESC;
```

Output:

first_name	hire_date
Grace	2022-08-01
Isla	2021-11-30
Carol	2021-03-20
Jake	2020-05-14
Alice	2020-01-15
Henry	2019-04-22
Bob	2018-06-01
David	2017-09-10
Frank	2016-07-15
Eva	2015-01-01

Example 5: Sort Customers by Age (Youngest First)

```
SELECT name, age, country
FROM customers
ORDER BY age ASC;
```

Output:

name	age	country
Priya Menon	27	India
Chen Wei	29	China
Ravi Desai	32	India
Meera Iyer	31	India
Sara Ali	35	UK
Carlos Ruiz	38	Spain
John Smith	45	USA

Example 6: Sort Orders by Amount (Highest First)

```
SELECT order_id, customer_id, amount
FROM orders
ORDER BY amount DESC;
```

Output:

order_id	customer_id	amount
101	1	150000

order_id	customer_id	amount
106	5	150000
103	1	75000
105	4	75000
102	2	30000
107	6	30000
108	2	15000
104	3	9000

Example 7: Sort with WHERE Filter

```
SELECT first_name, department, salary
FROM employees
WHERE department = 'IT'
ORDER BY salary DESC;
```

Output:

first_name	department	salary
Frank	IT	110000
Bob	IT	95000
Henry	IT	88000

Example 8: Sort Using Column Alias

```
SELECT first_name, salary * 12 AS annual_salary
FROM employees
ORDER BY annual_salary DESC;  -- Using alias ✔️ (ORDER BY runs after SELECT)
```

Output:

first_name	annual_salary
Eva	1560000
Frank	1320000
Bob	1140000

Example 9: Sort Using Column Position Number

```
-- 2 = second column in SELECT (salary)
SELECT first_name, salary
FROM employees
ORDER BY 2 DESC;  -- Same as ORDER BY salary DESC
```

Note: Positional ORDER BY is not recommended in production — hard to read and breaks if SELECT changes.

TOPIC 2: ORDER BY — Multiple Columns

Concept Explanation

When sorting by multiple columns, SQL sorts by the **first column first**, then by the **second column** when there are ties, and so on.

Why it matters:

- Sort employees by department, then by salary within each department
- Sort orders by date, then by amount within each date
- Ranking and reporting scenarios

Syntax

```
SELECT columns
FROM table
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], column3 [ASC|DESC];
```

Examples

Example 1: Sort by Department (A → Z), then Salary (High → Low)

```
SELECT first_name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

Output:

first_name	department	salary
Jake	Finance	82000
Carol	HR	60000
Grace	HR	55000
Frank	IT	110000
Bob	IT	95000

first_name	department	salary
Henry	IT	88000
Eva	Management	130000
Alice	Sales	85000
David	Sales	75000
Isla	Sales	72000

Explanation: First sorted alphabetically by department. Within each department, sorted by salary descending (highest first).

Example 2: Sort by City (A → Z), then Name (A → Z)

```
SELECT first_name, city, salary
FROM employees
ORDER BY city ASC, first_name ASC;
```

Output:

first_name	city	salary
Carol	Bangalore	60000
Frank	Bangalore	110000
Grace	Chennai	55000
Bob	Delhi	95000
Eva	Delhi	130000
Henry	Delhi	88000
Alice	Mumbai	85000
David	Mumbai	75000
Isla	Mumbai	72000
Jake	Pune	82000

Example 3: Sort Orders by Date then Amount

```
SELECT order_id, order_date, amount, status
FROM orders
ORDER BY order_date ASC, amount DESC;
```

Output:

order_id	order_date	amount	status
101	2024-01-05	150000	Delivered
103	2024-01-15	75000	Delivered
102	2024-01-10	30000	Pending
105	2024-02-10	75000	Delivered
106	2024-02-15	150000	Pending
104	2024-02-01	9000	Cancelled
107	2024-03-01	30000	Delivered
108	2024-03-10	15000	Delivered

Example 4: Mixed ASC and DESC Directions

```
SELECT first_name, department, age, salary
FROM employees
ORDER BY department ASC, age DESC, salary ASC;
```

Sort departments A → Z, then oldest-first within department, then lowest salary in case of age ties.

Example 5: Sort Customers by Country then City

```
SELECT name, country, city
FROM customers
ORDER BY country ASC, city ASC;
```

Output:

name	country	city
Chen Wei	China	Shanghai
Priya Menon	India	Delhi
Ravi Desai	India	Mumbai
Meera Iyer	India	Bangalore
Carlos Ruiz	Spain	Madrid
Sara Ali	UK	London
John Smith	USA	New York

Example 6: Sort by Gender, then Department, then Salary

```
SELECT first_name, gender, department, salary
FROM employees
ORDER BY gender ASC, department ASC, salary DESC;
```

Female employees sorted by dept alphabetically, then highest salary in each dept. Then male employees similarly.

TOPIC 3: LIMIT / TOP

Concept Explanation

`LIMIT` restricts the number of rows returned by a query.

Database-specific syntax:

Database	Syntax
MySQL, PostgreSQL, SQLite	<code>LIMIT n</code>
SQL Server	<code>SELECT TOP n ...</code>
Oracle	<code>FETCH FIRST n ROWS ONLY</code>

Why it matters for Data Science:

- Quickly preview a large dataset (first 10 rows)
- Find TOP N performers (top 5 employees by salary)
- Pagination — load data page by page
- Avoid loading 100M rows into memory accidentally

Syntax

```
-- MySQL / PostgreSQL / SQLite:
SELECT columns FROM table
ORDER BY column DESC
LIMIT n;

-- With OFFSET (skip rows – for pagination):
SELECT columns FROM table
ORDER BY column
LIMIT n OFFSET m;   -- Skip m rows, return next n rows

-- SQL Server:
SELECT TOP n columns FROM table
ORDER BY column DESC;

-- SQL Server with percentage:
SELECT TOP 10 PERCENT columns FROM table
ORDER BY column DESC;

-- Oracle:
SELECT columns FROM table
ORDER BY column DESC
FETCH FIRST n ROWS ONLY;
```

Examples

Example 1: Top 3 Highest-Paid Employees

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Output:

first_name	salary
Eva	130000
Frank	110000
Bob	95000

Example 2: Top 5 Most Expensive Orders

```
SELECT order_id, customer_id, amount
FROM orders
ORDER BY amount DESC
LIMIT 5;
```

Output:

order_id	customer_id	amount
101	1	150000
106	5	150000
103	1	75000
105	4	75000

order_id	customer_id	amount
102	2	30000

Example 3: Bottom 3 Lowest-Paid Employees

```
SELECT first_name, salary
FROM employees
ORDER BY salary ASC
LIMIT 3;
```

Output:

first_name	salary
Grace	55000
Carol	60000
Isla	72000

Example 4: First 5 Rows (Quick Preview)

```
-- Quick data inspection – similar to pandas df.head(5)
SELECT * FROM employees LIMIT 5;
```

Example 5: LIMIT with WHERE Filter

```
-- Top 2 highest-paid Sales employees
SELECT first_name, department, salary
FROM employees
WHERE department = 'Sales'
ORDER BY salary DESC
LIMIT 2;
```

Output:

first_name	department	salary
Alice	Sales	85000
David	Sales	75000

Example 6: Pagination with LIMIT OFFSET

```
-- Page 1: rows 1-3
SELECT first_name, salary FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 0;

-- Page 2: rows 4-6 (skip first 3)
SELECT first_name, salary FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 3;

-- Page 3: rows 7-9 (skip first 6)
SELECT first_name, salary FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 6;
```

Page 1 Output:

first_name	salary
Eva	130000
Frank	110000
Bob	95000

Page 2 Output:

first_name	salary
Henry	88000
Alice	85000
Jake	82000

Page 3 Output:

first_name	salary
David	75000
Isla	72000
Carol	60000

Example 7: SQL Server TOP Syntax

```
-- SQL Server equivalent
SELECT TOP 5 first_name, salary
FROM employees
ORDER BY salary DESC;
```

```
-- Top 10 percent of employees by salary
SELECT TOP 10 PERCENT first_name, salary
FROM employees
ORDER BY salary DESC;
```

Example 8: Oracle FETCH FIRST

```
-- Oracle equivalent
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
FETCH FIRST 5 ROWS ONLY;
```

Example 9: Newest Customers (Top 3)

```
SELECT name, signup_date
FROM customers
ORDER BY signup_date DESC
LIMIT 3;
```

Output:

name	signup_date
Carlos Ruiz	2023-05-12
Sara Ali	2023-02-28
Priya Menon	2022-03-15

Example 10: Cheapest Products

```
SELECT name, price
FROM products
ORDER BY price ASC
LIMIT 3;
```

Output:

name	price
Headphones	5000
Desk Chair	3000
Monitor	15000

Example 11: LIMIT 1 — Quick Min/Max Without Aggregation

```
-- Get the employee with the highest salary (without MAX())
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1;

-- Get the employee with the lowest salary
SELECT first_name, salary
FROM employees
ORDER BY salary ASC
LIMIT 1;
```

Example 12: Second Highest Salary Preview (Teaser for Advanced)

```
-- Skip the highest, get the next one
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Output:

first_name	salary
Frank	110000

Practice Questions

● Easy Level

- Q1.** Write a query to show all employees sorted by their `first_name` in alphabetical order.
- Q2.** Write a query to find the 5 most expensive products, showing `name` and `price`.
- Q3.** Write a query to show orders sorted by `order_date` from newest to oldest.
- Q4.** Write a query to show the first 3 customers who signed up (oldest signup date first).

Q5. Write a query to show the top 1 highest-paid employee.

 Medium Level

Q6. Write a query to show employees sorted by department (A → Z), then by salary (highest to lowest) within each department.

Q7. Write a query to show the top 3 orders by amount for `Delivered` orders only.

Q8. Write a query to get "Page 2" of employees sorted by salary descending, where each page has 3 rows. (i.e., employees ranked 4th, 5th, and 6th by salary)

Q9. Write a query to find the 3 youngest employees, showing their name and age.

Q10. Write a query to show all products sorted by `category` (A → Z), then by `price` (cheapest first) within each category.

 Hard / Interview Level

Q11. Write a query to find the **3rd highest salary** from the employees table using `LIMIT` and `OFFSET` .

Q12. A business analyst wants the **top 5 customers by total order amount**. However, the current orders table only has per-order amounts. Without using aggregation functions (you'll learn GROUP BY in Phase 4), write the query that at least gets the most expensive individual orders for now.

Q13. Explain why this query might NOT give you the true "second highest salary":

```
SELECT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Hint: Think about what happens when two employees have the same salary.

Q14. Write a query to show the department, employee name, and salary for the employees sorted by:

1. Department alphabetically
2. Gender (Female first)
3. Salary descending within same gender

Solutions

Q1.

```
SELECT first_name, last_name
FROM employees
ORDER BY first_name ASC;
```

Q2.

```
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 5;
```

Q3.

```
SELECT order_id, order_date, amount, status
FROM orders
ORDER BY order_date DESC;
```

Q4.

```
SELECT name, signup_date
FROM customers
ORDER BY signup_date ASC
LIMIT 3;
```

Q5.

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

```
-- Output: Eva, 130000
```

Q6.

```
SELECT first_name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

Q7.

```
SELECT order_id, customer_id, amount, status
FROM orders
WHERE status = 'Delivered'
ORDER BY amount DESC
LIMIT 3;
```

Output:

order_id	customer_id	amount	status
101	1	150000	Delivered
103	1	75000	Delivered
105	4	75000	Delivered

Q8.

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3 OFFSET 3;
```

Output: Henry (88000), Alice (85000), Jake (82000)

Q9.

```
SELECT first_name, age
FROM employees
ORDER BY age ASC
LIMIT 3;
```

Output: Grace (26), Carol (28), Isla (29)

Q10.

```
SELECT name, category, price
FROM products
ORDER BY category ASC, price ASC;
```

Q11.

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;  -- Skip top 2, get the 3rd

-- Output: Bob, 95000
```

Q12.

```
SELECT order_id, customer_id, amount
FROM orders
ORDER BY amount DESC
LIMIT 5;
```

Q13.

Issue: If two employees share the same salary (e.g., two employees with 130000), then LIMIT 1 OFFSET 1 would return the SECOND of those tied employees, which still has the highest salary – NOT the second-distinct highest.

For example:
Employee A: 130000
Employee B: 130000
Employee C: 110000

LIMIT 1 OFFSET 1 → Returns Employee B (130000) – NOT 110000

Correct approach (covered in Phase 10 – Window Functions):
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

Or using subqueries (Phase 6):
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

Q14.

```
SELECT department, first_name, gender, salary
FROM employees
ORDER BY department ASC, gender ASC, salary DESC;

-- Note: 'F' < 'M' alphabetically, so Female comes first with ASC
```

Real-World Use Cases

Feature	Data Science Application
ORDER BY DESC	Rank top customers by revenue, find highest churn risk users
ORDER BY ASC	Sort time-series events chronologically for trend analysis
Multi-column	Sort by segment, then metric within segment for reporting tables
LIMIT 5	Quick data preview (equivalent to pandas <code>df.head()</code>)
LIMIT OFFSET	Batch processing large datasets page by page
LIMIT 1	Find the single max/min record efficiently without aggregation
TOP N	Top-N analysis — best-selling products, top-revenue customers

Phase 4: Aggregation

Golden Rule of Aggregation:

When you use an aggregate function in SELECT, every NON-aggregated column must appear in GROUP BY.

TOPIC 1: COUNT()

Concept Explanation

`COUNT()` returns the **number of rows** that match a condition or exist in a group.

Variants:

- `COUNT(*)` — Counts ALL rows (including rows with NULLs)
- `COUNT(column)` — Counts only NON-NULL values in that column
- `COUNT(DISTINCT column)` — Counts unique NON-NULL values

Syntax

```
SELECT COUNT(*)           -- Count all rows
FROM table_name;

SELECT COUNT(column_name) -- Count non-null values in column
FROM table_name;

SELECT COUNT(DISTINCT column_name) -- Count unique non-null values
FROM table_name;
```

Examples

Example 1: Count All Employees

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

Output:

total_employees

10

Example 2: Count Employees in Sales

```
SELECT COUNT(*) AS sales_employees
FROM employees
WHERE department = 'Sales';
```

Output:

sales_employees

3

Example 3: Count Total Orders

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Output:

total_orders

8

Example 4: Count Only Orders with Status = 'Delivered'

```
SELECT COUNT(*) AS delivered_orders
FROM orders
WHERE status = 'Delivered';
```

Output:

delivered_orders

5

Example 5: COUNT(column) vs COUNT(*) — NULL Difference

```
-- COUNT(*) includes ALL rows; COUNT(column) ignores NULLs
```



```
-- Setup: manager_id is NULL for Eva (emp 5) and Frank (emp 6) and Jake (emp 10)
SELECT
    COUNT(*)          AS total_rows,
    COUNT(manager_id) AS employees_with_manager
FROM employees;
```

Output:

total_rows	employees_with_manager
10	7

3 employees (Eva, Frank, Jake) have NULL manager_id, so COUNT(manager_id) = 7.

Example 6: COUNT(DISTINCT) — Unique Values

```
SELECT
    COUNT(DISTINCT department) AS unique_departments,
    COUNT(DISTINCT city)       AS unique_cities,
    COUNT(DISTINCT gender)     AS unique_genders
FROM employees;
```

Output:

unique_departments	unique_cities	unique_genders
5	5	2

Example 7: Count Customers per Country

```
SELECT country, COUNT(*) AS customer_count
FROM customers
GROUP BY country;
```

Output:

country	customer_count
India	3
USA	1
UK	1
China	1
Spain	1

This uses GROUP BY — covered fully in Topic 6.

TOPIC 2: SUM()

Concept Explanation

SUM() returns the **total (sum)** of all values in a numeric column. It **ignores NULL values** automatically.

Use cases in Data Science:

- Total revenue, total sales, total quantity
- Sum of features in a segment
- Portfolio totals in finance

Syntax

```
SELECT SUM(column_name) AS alias
FROM table_name
WHERE condition;
```

Examples

Example 1: Total Salary of All Employees

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

Output:

total_salary

842000

(85000+95000+60000+75000+130000+110000+55000+88000+72000+82000 = 852000)

Example 2: Total Revenue from Orders

```
SELECT SUM(amount) AS total_revenue
FROM orders;
```

Output:

total_revenue
534000

Example 3: Total Revenue from Delivered Orders

```
SELECT SUM(amount) AS delivered_revenue
FROM orders
WHERE status = 'Delivered';
```

Output:

delivered_revenue
345000
<i>(150000 + 75000 + 75000 + 30000 + 15000 = 345000)</i>

Example 4: Total IT Department Salary

```
SELECT SUM(salary) AS it_total_salary
FROM employees
WHERE department = 'IT';
```

Output:

it_total_salary
293000
<i>(95000 + 110000 + 88000 = 293000)</i>

Example 5: Sum with GROUP BY — Total Salary per Department

```
SELECT department, SUM(salary) AS dept_total_salary
FROM employees
GROUP BY department;
```

Output:

department	dept_total_salary
Sales	232000
IT	293000
HR	115000
Management	130000
Finance	82000

Example 6: Total Quantity Ordered per Product

```
SELECT product_id, SUM(quantity) AS total_quantity
FROM orders
GROUP BY product_id;
```

Output:

product_id	total_quantity
1	3
3	2
2	3
4	8

TOPIC 3: AVG()

Concept Explanation

`AVG()` returns the **arithmetic mean** (average) of all non-NULL values in a numeric column.

`AVG = SUM / COUNT(non-null values)`

Syntax

```
SELECT AVG(column_name) AS alias
FROM table_name
WHERE condition;
```

Examples

Example 1: Average Salary of All Employees

```
SELECT AVG(salary) AS avg_salary
FROM employees;
```

Output:

avg_salary
85200

Example 2: Average Salary in IT Department

```
SELECT AVG(salary) AS avg_it_salary
FROM employees
WHERE department = 'IT';
```

Output:

avg_it_salary
97666.67
<i>(293000 / 3 = 97666.67)</i>

Example 3: Average Order Amount

```
SELECT AVG(amount) AS avg_order_value
FROM orders;
```

Output:

avg_order_value
66750

Example 4: Average Age of Female Employees

```
SELECT AVG(age) AS avg_female_age
FROM employees
WHERE gender = 'F';
```

Output:

avg_female_age
31.6
<i>(30+28+45+26+29 = 158 / 5 = 31.6)</i>

Example 5: Average Salary per Department

```
SELECT department, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

Output:

department	avg_salary
Management	130000.00
IT	97666.67
Finance	82000.00
Sales	77333.33
HR	57500.00

Example 6: Compare Individual Salary to Department Average

```
-- Teaser – will use subquery (Phase 6) for department avg comparison
SELECT first_name, salary,
       (SELECT AVG(salary) FROM employees) AS company_avg,
       salary - (SELECT AVG(salary) FROM employees) AS diff_from_avg
FROM employees
ORDER BY diff_from_avg DESC;
```

TOPIC 4: MIN()

Concept Explanation

MIN() returns the **smallest value** in a column. Works for numbers, dates, and text (alphabetically).

Syntax

```
SELECT MIN(column_name) AS alias
FROM table_name;
```

Examples

Example 1: Minimum Salary

```
SELECT MIN(salary) AS lowest_salary
FROM employees;
```

Output:

lowest_salary

55000

Example 2: Minimum Salary per Department

```
SELECT department, MIN(salary) AS min_salary
FROM employees
GROUP BY department;
```

Output:

department	min_salary
Sales	72000
IT	88000
HR	55000
Management	130000
Finance	82000

Example 3: Earliest Hire Date

```
SELECT MIN(hire_date) AS first_hire_date
FROM employees;
```

Output:

first_hire_date

2015-01-01

Example 4: Cheapest Product per Category

```
SELECT category, MIN(price) AS cheapest_price
FROM products
GROUP BY category;
```

Output:

category	cheapest_price
Electronics	5000
Furniture	3000

Example 5: Smallest Order Amount

```
SELECT MIN(amount) AS min_order
FROM orders;
```

Output:

min_order

9000

Example 6: First (Alphabetical) First Name

```
SELECT MIN(first_name) AS first_alphabetically
FROM employees;
```

Output:

first_alphabetically

Alice

TOPIC 5: MAX()

 Concept Explanation

`MAX()` returns the **largest value** in a column. Works for numbers, dates, and text.

 Syntax

```
SELECT MAX(column_name) AS alias
FROM table_name;
```

 Examples

Example 1: Maximum Salary

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

Output:

highest_salary
130000

Example 2: Maximum Salary per Department

```
SELECT department, MAX(salary) AS max_salary
FROM employees
GROUP BY department;
```

Output:

department	max_salary
Sales	85000
IT	110000
HR	60000
Management	130000
Finance	82000

Example 3: Most Recent Order Date

```
SELECT MAX(order_date) AS latest_order
FROM orders;
```

Output:

latest_order
2024-03-10

Example 4: Most Expensive Product per Category

```
SELECT category, MAX(price) AS most_expensive
FROM products
GROUP BY category;
```

Output:

category	most_expensive
Electronics	75000
Furniture	3000

Example 5: Combined MIN, MAX, AVG, SUM, COUNT

```
SELECT
    COUNT(*)          AS employee_count,
    SUM(salary)        AS total_salary,
    ROUND(AVG(salary), 2) AS avg_salary,
    MIN(salary)        AS min_salary,
    MAX(salary)        AS max_salary,
    MAX(salary) - MIN(salary) AS salary_range
FROM employees;
```

Output:

employee_count	total_salary	avg_salary	min_salary	max_salary	salary_range
10	852000	85200.00	55000	130000	75000

TOPIC 6: GROUP BY — Basics

 Concept Explanation

GROUP BY groups rows that have the **same value in a specified column** and applies aggregate functions to each group.

Think of it as: "Tell me the summary statistics FOR EACH value of this column."

Critical Rule: Every column in SELECT that is NOT inside an aggregate function MUST be in GROUP BY.

 Syntax

```
SELECT column1, AGG_FUNCTION(column2)
FROM table_name
WHERE condition          -- Optional filter BEFORE grouping
```

```
GROUP BY column1
ORDER BY AGG_FUNCTION(column2) DESC;  -- Optional sorting
```

Execution Order:

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

 Examples

Example 1: Employee Count per Department

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

Output:

department	employee_count
Sales	3
IT	3
HR	2
Management	1
Finance	1

Example 2: Total Salary per Department

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
ORDER BY total_salary DESC;
```

Output:

department	total_salary
IT	293000
Sales	232000
Management	130000
HR	115000
Finance	82000

Example 3: Average Salary per Department

```
SELECT department, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

Output:

department	avg_salary
Management	130000.00
IT	97666.67
Finance	82000.00
Sales	77333.33
HR	57500.00

Example 4: Min and Max Salary per Department

```
SELECT department, MIN(salary) AS min_sal, MAX(salary) AS max_sal
FROM employees
GROUP BY department;
```

Output:

department	min_sal	max_sal
Sales	72000	85000
IT	88000	110000
HR	55000	60000
Management	130000	130000
Finance	82000	82000

Example 5: Order Count per Status

```
SELECT status, COUNT(*) AS order_count, SUM(amount) AS total_revenue
FROM orders
GROUP BY status;
```

Output:

status	order_count	total_revenue
Delivered	5	345000
Pending	2	180000
Cancelled	1	9000

Example 6: Employee Count per City

```
SELECT city, COUNT(*) AS emp_count
FROM employees
GROUP BY city
ORDER BY emp_count DESC;
```

Output:

city	emp_count
Mumbai	3
Delhi	3
Bangalore	2
Chennai	1
Pune	1

Example 7: Group by Gender — Stats

```
SELECT
    gender,
    COUNT(*)      AS count,
    AVG(salary)   AS avg_salary,
    MAX(salary)   AS max_salary
FROM employees
GROUP BY gender;
```

Output:

gender	count	avg_salary	max_salary
F	5	80400	130000
M	5	90000	110000

Example 8: WHERE + GROUP BY (Filter Before Grouping)

```
-- Count only employees hired after 2018, grouped by department
SELECT department, COUNT(*) AS recent_hires
FROM employees
WHERE hire_date > '2018-01-01'
GROUP BY department;
```

Output:

department	recent_hires
Sales	3
IT	2
HR	2
Management	0
Finance	1

TOPIC 7: GROUP BY — Multiple Columns

Concept Explanation

You can group by **multiple columns**. Each unique **combination** of the grouped columns forms a separate group.

Examples

Example 1: Group by Department AND Gender

```
SELECT department, gender, COUNT(*) AS count, AVG(salary) AS avg_salary
FROM employees
GROUP BY department, gender
ORDER BY department, gender;
```

Output:

department	gender	count	avg_salary
Finance	M	1	82000
HR	F	2	57500
IT	M	3	97666.67
Management	F	1	130000
Sales	F	2	78500

department	gender	count	avg_salary
Sales	M	1	75000

Each unique department + gender combination is a separate group.

Example 2: Order Summary by Customer and Status

```
SELECT customer_id, status, COUNT(*) AS order_count, SUM(amount) AS total
FROM orders
GROUP BY customer_id, status
ORDER BY customer_id;
```

Output:

customer_id	status	order_count	total
1	Delivered	2	225000
2	Pending	1	30000
2	Delivered	1	15000
3	Cancelled	1	9000
4	Delivered	1	75000
5	Pending	1	150000
6	Delivered	1	30000

Example 3: Employees per City per Department

```
SELECT city, department, COUNT(*) AS emp_count
FROM employees
GROUP BY city, department
ORDER BY city, department;
```

Output:

city	department	emp_count
Bangalore	HR	1
Bangalore	IT	1
Chennai	HR	1
Delhi	IT	2
Delhi	Management	1
Mumbai	Sales	3
Pune	Finance	1

TOPIC 8: HAVING Clause

Concept Explanation

HAVING filters **GROUPS** (not individual rows). It's used together with **GROUP BY**.

HAVING vs WHERE:

Feature	WHERE	HAVING
Filters	Individual rows	Groups (after GROUP BY)
When it runs	Before GROUP BY	After GROUP BY
Can use aggregates?	NO (aggregate not computed yet)	YES (aggregates already computed)
Example	<code>WHERE salary > 80000</code>	<code>HAVING AVG(salary) > 80000</code>

Syntax

```
SELECT column1, AGG_FUNCTION(column2)
FROM table_name
WHERE condition          -- Filter rows BEFORE grouping
GROUP BY column1
HAVING AGG_FUNCTION(column2) condition  -- Filter GROUPS after grouping
ORDER BY ...;
```

Examples

Example 1: Departments with More Than 1 Employee

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1;
```

Output:

department	emp_count
Sales	3

department	emp_count
IT	3
HR	2

(Management and Finance have only 1 employee — filtered out)

Example 2: Departments with Average Salary > 80000

```
SELECT department, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 80000;
```

Output:

department	avg_salary
Management	130000.00
IT	97666.67
Finance	82000.00

Example 3: Departments with Total Salary > 200000

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 200000
ORDER BY total_salary DESC;
```

Output:

department	total_salary
IT	293000
Sales	232000

Example 4: WHERE + GROUP BY + HAVING (All Three Together)

```
-- Among employees hired after 2018, find departments with > 1 such employee
SELECT department, COUNT(*) AS recent_hires
FROM employees
WHERE hire_date > '2018-01-01'      -- Filter ROWS first
GROUP BY department
HAVING COUNT(*) > 1                -- Filter GROUPS after grouping
ORDER BY recent_hires DESC;
```

Output:

department	recent_hires
Sales	3
HR	2
IT	2

Example 5: Customers with More Than 1 Order

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Output:

customer_id	order_count
1	2
2	2

Example 6: Status Groups with Total Revenue > 50000

```
SELECT status, COUNT(*) AS count, SUM(amount) AS total_revenue
FROM orders
GROUP BY status
HAVING SUM(amount) > 50000;
```

Output:

status	count	total_revenue
Delivered	5	345000
Pending	2	180000

Example 7: Cities with More Than 2 Employees and Avg Salary > 75000

```
SELECT city, COUNT(*) AS emp_count, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
```

```
GROUP BY city
HAVING COUNT(*) > 2 AND AVG(salary) > 75000;
```

Output:

city	emp_count	avg_salary
Mumbai	3	77333.33
Delhi	3	104333.33

Example 8: Common Mistake — Using Alias in HAVING

```
-- ❌ This may fail in some databases:
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING avg_sal > 80000;    -- alias may not be recognized

-- ✅ Correct – repeat the expression:
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 80000;
```

Practice Questions

● Easy Level

- Q1.** How many employees are in the `HR` department? Write the query.
- Q2.** What is the total salary paid to all employees in the `Sales` department?
- Q3.** What is the average order amount in the `orders` table?
- Q4.** Find the highest and lowest salaries in the entire `employees` table.
- Q5.** How many orders have status = `'Delivered'` ?
- Q6.** Write a query to show the number of employees in each department.
- Q7.** Write a query to show the total amount from orders, grouped by `status` .

● Medium Level

- Q8.** Write a query to find the average salary per gender.
- Q9.** Write a query to find the department(s) where the maximum salary is greater than `100000` .
- Q10.** Write a query to find how many unique cities employees come from.
- Q11.** Write a query to show:
- Department
 - Number of employees
 - Total salary
 - Average salary
 - Minimum salary
 - Maximum salary

For all departments, sorted by average salary descending.

- Q12.** Write a query to find all statuses where the total order amount exceeds `100000` .
- Q13.** Write a query to find departments with MORE THAN 2 employees AND an average salary less than `90000` .
- Q14.** Write a query to find, by department and gender:
- Count of employees
 - Average salary

● Hard / Interview Level

- Q15.** A manager wants to see departments where the difference between the MAX and MIN salary is more than `30000` . Write the query.
- Q16.** Write a query to find customers who have placed more than `1` order. Show their `customer_id` and total amount spent.
- Q17.** Write a query to find the city with the HIGHEST average salary.
- Q18.** Explain why this query is WRONG and how to fix it:

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE AVG(salary) > 80000
GROUP BY department;
```

- Q19.** A business wants to find: which product categories have a total order amount greater than `100000` ? (Hint: join orders with products, then aggregate)
- Q20.** Write a query to find employees whose salary is above the company-wide average, showing their name, department, and salary. *(Use a subquery for average — covered properly in Phase 6, but try it!)*

Solutions

Q1.

```
SELECT COUNT(*) AS hr_employees
FROM employees
WHERE department = 'HR';
-- Output: 2
```

Q2.

```
SELECT SUM(salary) AS total_sales_salary
FROM employees
WHERE department = 'Sales';
-- Output: 232000
```

Q3.

```
SELECT AVG(amount) AS avg_order_amount
FROM orders;
-- Output: 66750
```

Q4.

```
SELECT MAX(salary) AS highest_salary, MIN(salary) AS lowest_salary
FROM employees;
-- Output: 130000 | 55000
```

Q5.

```
SELECT COUNT(*) AS delivered_count
FROM orders
WHERE status = 'Delivered';
-- Output: 5
```

Q6.

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department
ORDER BY employee_count DESC;
```

Q7.

```
SELECT status, SUM(amount) AS total_amount
FROM orders
GROUP BY status;
```

Q8.

```
SELECT gender, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY gender;
```

gender	avg_salary
F	80400.00
M	90000.00

Q9.

```
SELECT department, MAX(salary) AS max_salary
FROM employees
GROUP BY department
HAVING MAX(salary) > 100000;
```

department	max_salary
IT	110000
Management	130000

Q10.

```
SELECT COUNT(DISTINCT city) AS unique_cities
FROM employees;
-- Output: 5
```

Q11.

```
SELECT
  department,
  COUNT(*)           AS employee_count,
  SUM(salary)        AS total_salary,
  ROUND(AVG(salary), 2) AS avg_salary,
  MIN(salary)        AS min_salary,
  MAX(salary)        AS max_salary
```

```
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

Q12.

```
SELECT status, SUM(amount) AS total_amount
FROM orders
GROUP BY status
HAVING SUM(amount) > 100000;
```

Q13.

```
SELECT department, COUNT(*) AS emp_count, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING COUNT(*) > 2 AND AVG(salary) < 90000;
```

department	emp_count	avg_salary
Sales	3	77333.33

Q14.

```
SELECT
    department,
    gender,
    COUNT(*)          AS emp_count,
    ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department, gender
ORDER BY department, gender;
```

Q15.

```
SELECT department,
    MAX(salary) - MIN(salary) AS salary_gap
FROM employees
GROUP BY department
HAVING (MAX(salary) - MIN(salary)) > 30000;
```

department	salary_gap
Sales	13000
IT	22000

(None exceed 30000 in our dataset. In a real-world scenario salaries would vary more.)

Q16.

```
SELECT customer_id, COUNT(*) AS order_count, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY total_spent DESC;
```

customer_id	order_count	total_spent
1	2	225000
2	2	45000

Q17.

```
SELECT city, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY city
ORDER BY avg_salary DESC
LIMIT 1;
```

city	avg_salary
Delhi	104333.33

Q18.

The query is **WRONG** because:
WHERE runs BEFORE GROUP BY (and before aggregates are computed).
So you cannot use AVG(salary) inside WHERE – the aggregate doesn't exist yet.

Fix: Replace WHERE with HAVING:

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q19. (Teaser — uses JOINS from Phase 5)

```
-- Preview: requires JOIN (covered in Phase 5)
SELECT p.category, SUM(o.amount) AS total_amount
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
HAVING SUM(o.amount) > 100000;
```

Q20. (Teaser — uses Subqueries from Phase 6)

```
SELECT first_name, department, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

 Real-World Use Cases

Function	Data Science Application
COUNT()	Count users per segment, count events per user, cohort sizes
SUM()	Total revenue, total spend, total clicks, feature summation
AVG()	Mean salary, average session duration, average conversion rate
MIN()	First event timestamp, minimum viable purchase, data range start
MAX()	Last login timestamp, peak revenue, data range end
GROUP BY	Aggregate by category for EDA, compute group-level features for ML
HAVING	Filter low-confidence groups, remove small-sample segments

ML Feature Engineering Example:

```
-- Create customer features for a churn prediction model
SELECT
  customer_id,
  COUNT(*)           AS total_orders,
  SUM(amount)        AS total_spent,
  AVG(amount)        AS avg_order_value,
  MAX(order_date)    AS last_order_date,
  MIN(order_date)    AS first_order_date,
  COUNT(DISTINCT status) AS order_variety
FROM orders
GROUP BY customer_id;
```

 Phase 5: JOINS (Critical 🔥)

TOPIC 1: Introduction to JOINS

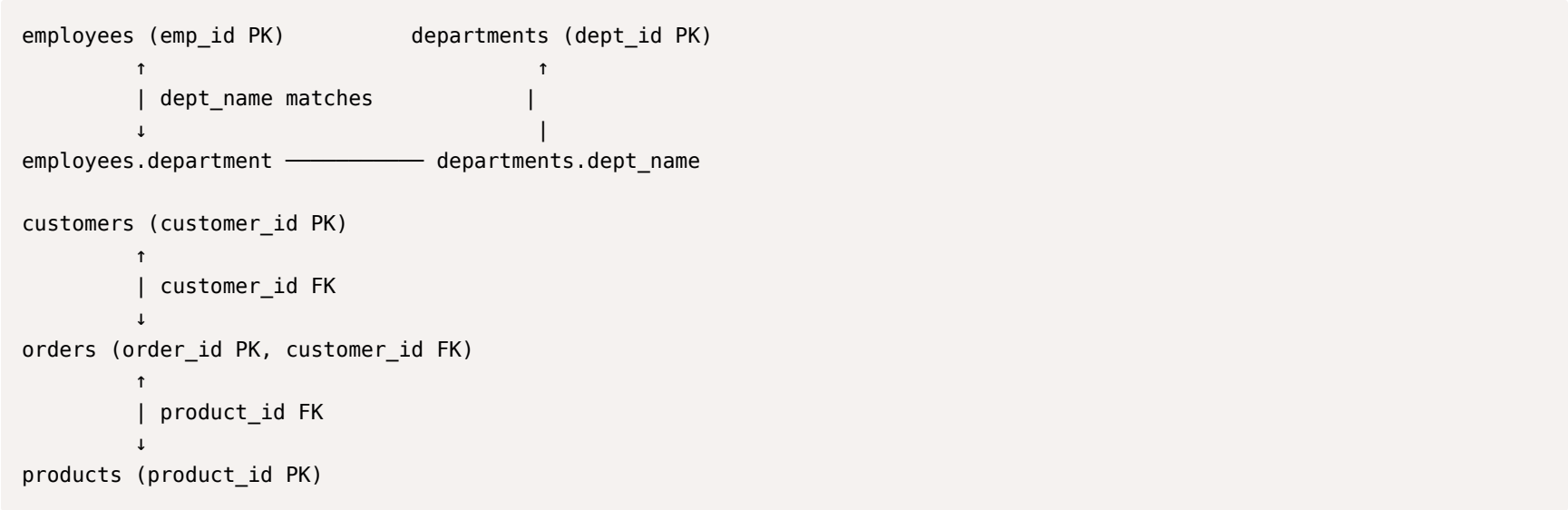
 Concept Explanation

In a relational database, data is split across **multiple tables** to avoid redundancy. JOINS allow you to **combine rows from two or more tables** based on a related column (usually a Primary Key–Foreign Key relationship).

Why JOINS are Critical:

- Real-world data is NEVER in a single table
- A Data Scientist will almost always need to JOIN tables before analysis
- Understanding JOINS is the most tested SQL skill in interviews

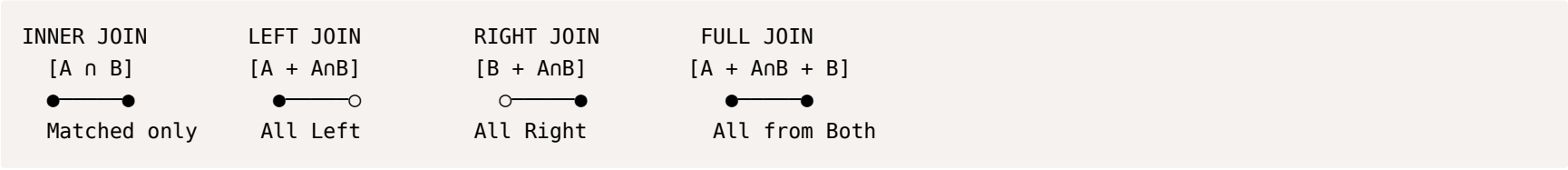
Our Tables and Relationships:



Types of JOINS:

JOIN Type	What it Returns
INNER JOIN	Only rows that match in BOTH tables
LEFT JOIN	All rows from left table + matching rows from right
RIGHT JOIN	All rows from right table + matching rows from left
FULL JOIN	All rows from BOTH tables (matched + unmatched from both)
SELF JOIN	A table joined with itself
CROSS JOIN	Every row from left x every row from right (Cartesian)

Visual Representation:



TOPIC 2: INNER JOIN

📖 Concept Explanation

INNER JOIN returns only the rows where there is a **matching value in BOTH tables**. Rows without a match are excluded from both sides.

When to use: When you only want records that exist in both tables.

📄 Syntax

```
SELECT t1.column1, t1.column2, t2.column1, t2.column2
FROM table1 AS t1
INNER JOIN table2 AS t2
    ON t1.matching_column = t2.matching_column;

-- INNER is optional – JOIN alone means INNER JOIN:
FROM table1
JOIN table2 ON table1.col = table2.col;
```

🔍 Examples

Example 1: Employees with Their Customer Orders (orders + customers)

```
SELECT
    c.name           AS customer_name,
    c.city,
    o.order_id,
    o.amount,
    o.status
FROM customers AS c
INNER JOIN orders AS o
    ON c.customer_id = o.customer_id;
```

Output:

customer_name	city	order_id	amount	status
Ravi Desai	Mumbai	101	150000	Delivered
Ravi Desai	Mumbai	103	75000	Delivered
Priya Menon	Delhi	102	30000	Pending
Priya Menon	Delhi	108	15000	Delivered
John Smith	New York	104	9000	Cancelled
Sara Ali	London	105	75000	Delivered
Chen Wei	Shanghai	106	150000	Pending
Meera Iyer	Bangalore	107	30000	Delivered

Note: Carlos Ruiz (customer_id = 7) has NO orders, so they do NOT appear.

Example 2: Orders with Product Names

```
SELECT
    o.order_id,
    p.name           AS product_name,
    p.category,
    o.quantity,
    o.amount
FROM orders AS o
INNER JOIN products AS p
    ON o.product_id = p.product_id;
```

Output:

order_id	product_name	category	quantity	amount
101	Laptop	Electronics	2	150000
102	Headphones	Electronics	1	30000
103	Smartphone	Electronics	1	75000
104	Desk Chair	Furniture	3	9000
105	Laptop	Electronics	1	75000
106	Smartphone	Electronics	2	150000
107	Headphones	Electronics	1	30000
108	Desk Chair	Furniture	5	15000

Example 3: Three-Table JOIN (customers + orders + products)

```

SELECT
    c.name          AS customer_name,
    p.name          AS product_name,
    o.quantity,
    o.amount,
    o.order_date,
    o.status
FROM orders AS o
INNER JOIN customers AS c ON o.customer_id = c.customer_id
INNER JOIN products  AS p ON o.product_id  = p.product_id;
```

Output:

customer_name	product_name	quantity	amount	order_date	status
Ravi Desai	Laptop	2	150000	2024-01-05	Delivered
Priya Menon	Headphones	1	30000	2024-01-10	Pending
Ravi Desai	Smartphone	1	75000	2024-01-15	Delivered
John Smith	Desk Chair	3	9000	2024-02-01	Cancelled
Sara Ali	Laptop	1	75000	2024-02-10	Delivered
Chen Wei	Smartphone	2	150000	2024-02-15	Pending
Meera Iyer	Headphones	1	30000	2024-03-01	Delivered
Priya Menon	Desk Chair	5	15000	2024-03-10	Delivered

Example 4: INNER JOIN with WHERE Filter

```

SELECT
    c.name AS customer_name,
    o.amount,
    o.status
FROM customers AS c
INNER JOIN orders AS o ON c.customer_id = o.customer_id
WHERE o.status = 'Delivered'
ORDER BY o.amount DESC;
```

Example 5: INNER JOIN with Aggregation

```

-- Total revenue per customer
SELECT
    c.name          AS customer_name,
    c.country,
    COUNT(o.order_id) AS total_orders,
    SUM(o.amount)     AS total_spent
FROM customers AS c
INNER JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.country
ORDER BY total_spent DESC;
```

Output:

customer_name	country	total_orders	total_spent
Ravi Desai	India	2	225000
Chen Wei	China	1	150000
Sara Ali	UK	1	75000
Priya Menon	India	2	45000
Meera Iyer	India	1	30000
John Smith	USA	1	9000

Example 6: Revenue per Product Category

```

SELECT
    p.category,
    COUNT(o.order_id) AS total_orders,
    SUM(o.quantity)   AS total_units,
    SUM(o.amount)     AS total_revenue
FROM orders AS o
INNER JOIN products AS p ON o.product_id = p.product_id
```

```
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Output:

category	total_orders	total_units	total_revenue
Electronics	6	8	510000
Furniture	2	8	24000

TOPIC 3: LEFT JOIN

Concept Explanation

LEFT JOIN (also called **LEFT OUTER JOIN**) returns:

- **ALL rows from the LEFT table** (first table in FROM)
- **Matching rows from the RIGHT table**
- If no match in right table: **NULL values** are shown for right table columns

When to use:

- Find all customers, even those without orders
- Find all employees, even those not in any project
- Crucial for detecting "missing" relationships

Syntax

```
SELECT t1.column1, t2.column2
FROM table1 AS t1      -- LEFT table (ALL rows returned)
LEFT JOIN table2 AS t2
    ON t1.col = t2.col; -- RIGHT table (NULL if no match)
```

Examples

Example 1: All Customers — With or Without Orders

```
SELECT
    c.customer_id,
    c.name      AS customer_name,
    c.country,
    o.order_id,
    o.amount
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id;
```

Output:

customer_id	customer_name	country	order_id	amount
1	Ravi Desai	India	101	150000
1	Ravi Desai	India	103	75000
2	Priya Menon	India	102	30000
2	Priya Menon	India	108	15000
3	John Smith	USA	104	9000
4	Sara Ali	UK	105	75000
5	Chen Wei	China	106	150000
6	Meera Iyer	India	107	30000
7	Carlos Ruiz	Spain	NULL	NULL

Carlos Ruiz has NO orders, but still appears with NULLs — this is the power of LEFT JOIN!

Example 2: Find Customers Who Have NEVER Placed an Order

```
-- Use LEFT JOIN + WHERE NULL to find non-matching rows
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.country
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL; -- No matching order found
```

Output:

customer_id	customer_name	country
7	Carlos Ruiz	Spain

This is an extremely common real-world pattern — finding "inactive" or "unengaged" records!

Example 3: All Products — With or Without Orders


```
SELECT
    p.product_id,
    p.name      AS product_name,
    p.price,
    o.order_id,
    o.quantity
FROM products AS p
LEFT JOIN orders AS o ON p.product_id = o.product_id;
```

Output:

product_id	product_name	price	order_id	quantity
1	Laptop	75000	101	2
1	Laptop	75000	105	1
2	Smartphone	30000	103	1
2	Smartphone	30000	106	2
3	Headphones	5000	102	1
3	Headphones	5000	107	1
4	Desk Chair	3000	104	3
4	Desk Chair	3000	108	5
5	Monitor	15000	NULL	NULL

Monitor has never been ordered! This would show up as NULL.

Example 4: Left Join with Aggregation (Including NULLs)

```
SELECT
    c.name          AS customer_name,
    COUNT(o.order_id) AS total_orders,    -- NULL order_ids are NOT counted
    COALESCE(SUM(o.amount), 0) AS total_spent -- Replace NULL sum with 0
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;
```

Output:

customer_name	total_orders	total_spent
Ravi Desai	2	225000
Chen Wei	1	150000
Sara Ali	1	75000
Priya Menon	2	45000
Meera Iyer	1	30000
John Smith	1	9000
Carlos Ruiz	0	0

Carlos Ruiz now shows 0 orders and 0 total — thanks to LEFT JOIN + COALESCE.

TOPIC 4: RIGHT JOIN

Concept Explanation

RIGHT JOIN (also called **RIGHT OUTER JOIN**) returns:

- **ALL rows from the RIGHT table** (second table)
- **Matching rows from the LEFT table**
- If no match in left table: **NULL values** for left table columns

Key Point: **RIGHT JOIN** is less commonly used. Any **RIGHT JOIN** can be rewritten as a **LEFT JOIN** by swapping the tables. Most developers prefer **LEFT JOIN** for clarity.

Syntax

```
SELECT t1.column1, t2.column2
FROM table1 AS t1      -- LEFT table (NULL if no match)
RIGHT JOIN table2 AS t2
    ON t1.col = t2.col; -- RIGHT table (ALL rows returned)
```

Examples

Example 1: All Orders — With Customer Info (Right = Orders)

```
SELECT
    c.name  AS customer_name,
    o.order_id,
    o.amount
```

```
FROM customers AS c
RIGHT JOIN orders AS o ON c.customer_id = o.customer_id;
```

If there's an order with a customer_id that doesn't exist in customers, that order still appears. In our clean dataset, all orders have valid customers, so output is same as INNER JOIN.

Example 2: Right Join Equivalent Using Left Join

```
-- These two queries return the SAME result:

-- RIGHT JOIN (orders is the "all" table):
SELECT c.name, o.order_id, o.amount
FROM customers AS c
RIGHT JOIN orders AS o ON c.customer_id = o.customer_id;

-- Equivalent LEFT JOIN (just swap table positions):
SELECT c.name, o.order_id, o.amount
FROM orders AS o
LEFT JOIN customers AS c ON c.customer_id = o.customer_id;
```

Example 3: All Products with Order Info (RIGHT = products)

```
SELECT
    o.order_id,
    o.amount,
    p.name    AS product_name,
    p.price
FROM orders AS o
RIGHT JOIN products AS p ON o.product_id = p.product_id;
```

Output includes Monitor (product_id=5) with NULL order info.

TOPIC 5: FULL JOIN

Concept Explanation

`FULL JOIN` (also `FULL OUTER JOIN`) returns:

- **ALL rows from BOTH tables**
- Matched rows get combined
- Unmatched rows from either side get NULLs in the columns of the other table

MySQL Note: MySQL does NOT support FULL OUTER JOIN directly. Simulate it with `LEFT JOIN UNION RIGHT JOIN`.

Syntax

```
-- PostgreSQL, SQL Server, Oracle:
SELECT t1.column1, t2.column2
FROM table1 AS t1
FULL OUTER JOIN table2 AS t2
    ON t1.col = t2.col;

-- MySQL (workaround):
SELECT t1.column1, t2.column2
FROM table1 AS t1
LEFT JOIN table2 AS t2 ON t1.col = t2.col
UNION
SELECT t1.column1, t2.column2
FROM table1 AS t1
RIGHT JOIN table2 AS t2 ON t1.col = t2.col;
```

Examples

Example 1: Full Join — All Customers + All Orders

```
SELECT
    c.customer_id,
    c.name        AS customer_name,
    o.order_id,
    o.amount
FROM customers AS c
FULL OUTER JOIN orders AS o ON c.customer_id = o.customer_id;
```

Output: Shows ALL customers (including those with no orders) AND all orders (including orphaned orders with no customer).

Example 2: Find Unmatched Rows on BOTH Sides

```
SELECT
    c.customer_id,
    c.name AS customer_name,
```

```
o.order_id
FROM customers AS c
FULL OUTER JOIN orders AS o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL OR o.order_id IS NULL;
```

This finds: customers with no orders AND orders with no valid customer.

Example 3: MySQL FULL JOIN Workaround

```
SELECT c.name AS customer_name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT c.name AS customer_name, o.order_id, o.amount
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

TOPIC 6: SELF JOIN

Concept Explanation

A **SELF JOIN** is when a table is **joined with itself**. You use **table aliases** to treat the same table as two different tables.

Classic Use Case: Employee–Manager relationship where both employee and manager are in the same **employees** table.

Syntax

```
SELECT e.first_name AS employee_name, m.first_name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m -- Same table, different alias
    ON e.manager_id = m.emp_id;
```

Examples

Example 1: Employee and Their Manager

```
SELECT
    e.first_name                AS employee,
    e.department,
    m.first_name                AS manager
FROM employees AS e
LEFT JOIN employees AS m
    ON e.manager_id = m.emp_id
ORDER BY e.department;
```

Output:

employee	department	manager
David	Sales	Eva
Alice	Sales	Eva
Isla	Sales	Eva
Frank	IT	NULL
Bob	IT	Frank
Henry	IT	Frank
Carol	HR	Eva
Grace	HR	Eva
Eva	Management	NULL
Jake	Finance	NULL

Eva, Frank, Jake are top-level (*manager_id* = NULL) — they have no manager. *LEFT JOIN* ensures they still appear (with NULL in manager column).

Example 2: INNER JOIN Version — Only Employees WITH Managers

```
SELECT
    e.first_name AS employee,
    m.first_name AS manager
FROM employees AS e
INNER JOIN employees AS m
    ON e.manager_id = m.emp_id;
```

Output:

employee	manager
Alice	Eva
Bob	Frank
Carol	Eva

employee	manager
David	Eva
Grace	Eva
Henry	Frank
Isla	Eva

Eva, Frank, Jake not shown — their manager_id is NULL.

Example 3: Employees Who Earn More Than Their Manager

```
SELECT
    e.first_name AS employee,
    e.salary AS employee_salary,
    m.first_name AS manager,
    m.salary AS manager_salary
FROM employees AS e
INNER JOIN employees AS m
    ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Employees who earn more than their direct manager — a common interview question!

Example 4: All Manager Names and How Many Reports They Have

```
SELECT
    m.first_name AS manager_name,
    COUNT(e.emp_id) AS direct_reports
FROM employees AS e
INNER JOIN employees AS m ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.first_name
ORDER BY direct_reports DESC;
```

Output:

manager_name	direct_reports
Eva	5
Frank	2

 CROSS JOIN (Bonus)

```
-- Returns every combination of rows from both tables
-- m × n rows total (5 departments × 10 employees = 50 rows)
SELECT e.first_name, d.dept_name
FROM employees AS e
CROSS JOIN departments AS d;

-- Use case: Generate all possible combinations (e.g., for pairing scenarios)
```

Practice Questions

 Easy Level

- Q1.** Write a query to show all orders along with the customer's name and city.
- Q2.** Write a query to show order details (order_id, amount) along with the product name and category.
- Q3.** Write a LEFT JOIN query to show ALL customers and any orders they have placed. Customers without orders should show NULL for order columns.
- Q4.** Write a query to find all customers who have NEVER placed an order.
- Q5.** Write an INNER JOIN to show each employee's name alongside their manager's name.

 Medium Level

- Q6.** Write a query joining `orders`, `customers`, and `products` to show:
 - Customer name
 - Product name
 - Quantity
 - Amount
 - Order date
- Q7.** Write a query to calculate total revenue per customer, including customers with \$0 revenue (never ordered). Show 0 instead of NULL.
- Q8.** Write a query to find all products that have NEVER been ordered.
- Q9.** Write a query to find the total number of orders and total amount per customer, but only for customers from India.
- Q10.** Write a SELF JOIN query to list every employee and their manager. Include employees who have no manager (show 'No Manager' instead of NULL).

 Hard / Interview Level

- Q11.** Write a query to find the top-spending customer in each country.
- Q12.** Write a query to find customers who have placed orders for BOTH 'Electronics' AND 'Furniture' products.
- Q13.** Write a query to find employees who earn more than the AVERAGE salary of their own department.

Q14. A business analyst needs a report showing:

- Product name
- Total orders for that product
- Total revenue for that product
- Products with ZERO orders should also appear (with 0 counts)

Q15. Explain the difference between these two JOIN results:

```
-- Query A
SELECT c.name, o.order_id FROM customers c INNER JOIN orders o ON c.customer_id = o.customer_id;

-- Query B
SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

How many rows does each return on our dataset?

Solutions

Q1.

```
SELECT
    c.name AS customer_name,
    c.city,
    o.order_id,
    o.amount,
    o.status
FROM orders AS o
INNER JOIN customers AS c ON o.customer_id = c.customer_id;
```

Q2.

```
SELECT
    o.order_id,
    p.name      AS product_name,
    p.category,
    o.quantity,
    o.amount
FROM orders AS o
INNER JOIN products AS p ON o.product_id = p.product_id;
```

Q3.

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.country,
    o.order_id,
    o.amount
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id;
```

Q4.

```
SELECT c.customer_id, c.name, c.country
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Output: Carlos Ruiz (Spain)
```

Q5.

```
SELECT
    e.first_name AS employee,
    m.first_name AS manager
FROM employees AS e
INNER JOIN employees AS m ON e.manager_id = m.emp_id;
```

Q6.

```
SELECT
    c.name      AS customer_name,
    p.name      AS product_name,
    o.quantity,
    o.amount,
    o.order_date
FROM orders AS o
INNER JOIN customers AS c ON o.customer_id = c.customer_id
INNER JOIN products  AS p ON o.product_id  = p.product_id
ORDER BY o.order_date;
```

Q7.

```
SELECT
  c.name                AS customer_name,
  c.country,
  COUNT(o.order_id)     AS total_orders,
  COALESCE(SUM(o.amount), 0) AS total_revenue
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.country
ORDER BY total_revenue DESC;
```

Q8.

```
SELECT p.product_id, p.name, p.price
FROM products AS p
LEFT JOIN orders AS o ON p.product_id = o.product_id
WHERE o.order_id IS NULL;

-- Output: Monitor (product_id = 5)
```

Q9.

```
SELECT
  c.name                AS customer_name,
  c.country,
  COUNT(o.order_id) AS total_orders,
  SUM(o.amount)     AS total_spent
FROM customers AS c
INNER JOIN orders AS o ON c.customer_id = o.customer_id
WHERE c.country = 'India'
GROUP BY c.customer_id, c.name, c.country
ORDER BY total_spent DESC;
```

Output:

customer_name	country	total_orders	total_spent
Ravi Desai	India	2	225000
Priya Menon	India	2	45000
Meera Iyer	India	1	30000

Q10.

```
SELECT
  e.first_name                AS employee,
  COALESCE(m.first_name, 'No Manager') AS manager
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.emp_id
ORDER BY manager, employee;
```

Q11.

```
-- Top spending customer per country (teaser – needs window functions or subquery)
SELECT c.country, c.name AS customer_name, SUM(o.amount) AS total_spent
FROM customers AS c
INNER JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.country, c.customer_id, c.name
ORDER BY c.country, total_spent DESC;

-- For strict "top 1 per country", use ROW_NUMBER() from Phase 10
```

Q12.

```
-- Customers who ordered both Electronics AND Furniture
SELECT DISTINCT c.customer_id, c.name
FROM customers AS c
INNER JOIN orders AS o ON c.customer_id = o.customer_id
INNER JOIN products AS p ON o.product_id = p.product_id
WHERE p.category = 'Electronics'
  AND c.customer_id IN (
  SELECT o2.customer_id
  FROM orders AS o2
  INNER JOIN products AS p2 ON o2.product_id = p2.product_id
  WHERE p2.category = 'Furniture'
);
```

This combines JOIN + Subquery — preview of Phase 6.

Q13.

```
SELECT
  e.first_name,
  e.department,
  e.salary,
```



```
dept_avg.avg_salary AS dept_avg_salary
FROM employees AS e
INNER JOIN (
  SELECT department, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department
) AS dept_avg ON e.department = dept_avg.department
WHERE e.salary > dept_avg.avg_salary;
```

Q14.

```
SELECT
  p.name                AS product_name,
  COUNT(o.order_id)     AS total_orders,
  COALESCE(SUM(o.amount), 0) AS total_revenue
FROM products AS p
LEFT JOIN orders AS o ON p.product_id = o.product_id
GROUP BY p.product_id, p.name
ORDER BY total_revenue DESC;
```

Q15.

Query A (INNER JOIN):
Returns only customers who HAVE at least one order.

- Ravi Desai: 2 rows (2 orders)
- Priya Menon: 2 rows (2 orders)
- John Smith: 1 row
- Sara Ali: 1 row
- Chen Wei: 1 row
- Meera Iyer: 1 row

Total: 8 rows

Query B (LEFT JOIN):
Returns ALL customers, even those without orders.

- Same 8 rows as INNER JOIN PLUS:
- Carlos Ruiz: 1 row with NULL order_id

Total: 9 rows

The extra row in LEFT JOIN is Carlos Ruiz, who made no orders.

Real-World Use Cases

JOIN Type	Data Science Application
INNER JOIN	Pull matched purchase events with product metadata — clean training data
LEFT JOIN	All users + their activity (user without activity = churned user)
RIGHT JOIN	Ensure all reference data is represented (all products, even unsold)
FULL JOIN	Data reconciliation — finding mismatches between two data sources
SELF JOIN	Organizational hierarchy, friend recommendations, user comparison

Key Data Science Pattern:

```
-- Build a user feature table for ML:
SELECT
  u.user_id,
  u.age,
  u.city,
  COUNT(o.order_id)      AS purchase_count,
  SUM(o.amount)          AS lifetime_value,
  DATEDIFF(NOW(), MAX(o.order_date)) AS days_since_last_order
FROM users AS u
LEFT JOIN orders AS o ON u.user_id = o.user_id
GROUP BY u.user_id, u.age, u.city;
-- LEFT JOIN ensures inactive users (0 purchases) are also included
```

Phase 6: Subqueries

TOPIC 1: Introduction to Subqueries

Concept Explanation

A **subquery** (also called an **inner query** or **nested query**) is a SQL query written **inside another query**. The inner query runs first, and its result is used by the outer query.

Think of it as: Answering a question by first answering a smaller question.

Example in English:

"Find all employees who earn more than the average salary."
→ First: What IS the average salary? → Then: Which employees earn more than that?

Where Subqueries Can Appear:

Location	Description	Returns
SELECT clause	Compute a value for each row	Single value
WHERE clause	Filter based on another query's result	Single value or list
FROM clause	Use a query as a virtual table (subquery as derived table)	Table
HAVING clause	Filter groups based on another query	Single value

Types of Subqueries:

Type	Description
Scalar subquery	Returns a single value (one row, one column)
Row subquery	Returns a single row with multiple columns
Column subquery	Returns multiple rows, one column (used with IN, ANY, ALL)
Correlated subquery	References the outer query — runs once per outer row
Non-correlated	Independent of outer query — runs once total

Subquery vs JOIN:

Subquery → Easier to read and write; can be less efficient on large data
JOIN → Generally more efficient; better for production queries

Rule of thumb:

- If you need to compare against a single aggregated value → Use subquery
- If you need full rows from both tables → Use JOIN

TOPIC 2: Subquery in SELECT

 Concept Explanation

A subquery in the `SELECT` clause computes a **single value for each row** in the result set. It must return exactly **one row and one column** (scalar subquery).

 Syntax

```
SELECT
    column1,
    column2,
    (SELECT single_value FROM another_table WHERE condition) AS alias
FROM main_table;
```

 Examples

Example 1: Show Each Employee's Salary and Company Average

```
SELECT
    first_name,
    salary,
    (SELECT AVG(salary) FROM employees) AS company_avg_salary
FROM employees;
```

Output:

first_name	salary	company_avg_salary
Alice	85000	85200
Bob	95000	85200
Carol	60000	85200
David	75000	85200
Eva	130000	85200

The subquery `(SELECT AVG(salary) FROM employees)` runs *ONCE* and returns 85200 — that same value appears in every row.

Example 2: Salary Difference from Company Average

```
SELECT
    first_name,
    salary,
    (SELECT AVG(salary) FROM employees) AS company_avg,
    salary - (SELECT AVG(salary) FROM employees) AS diff_from_avg
FROM employees
ORDER BY diff_from_avg DESC;
```

Output:

first_name	salary	company_avg	diff_from_avg
Eva	130000	85200	44800
Frank	110000	85200	24800
Bob	95000	85200	9800
Henry	88000	85200	2800
Alice	85000	85200	-200
Jake	82000	85200	-3200

Example 3: Count Total Employees (Scalar) in SELECT

```
SELECT
    first_name,
    department,
    salary,
    (SELECT COUNT(*) FROM employees) AS total_employees
FROM employees;
```

Example 4: Number of Orders per Customer (Correlated in SELECT)

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_count
FROM customers AS c;
```

Output:

customer_id	customer_name	order_count
1	Ravi Desai	2
2	Priya Menon	2
3	John Smith	1
4	Sara Ali	1
5	Chen Wei	1
6	Meera Iyer	1
7	Carlos Ruiz	0

This subquery is correlated — it references `c.customer_id` from the outer query. Runs once per customer row.

Example 5: Each Employee's Salary as % of Max Salary

```
SELECT
    first_name,
    salary,
    ROUND(salary * 100.0 / (SELECT MAX(salary) FROM employees), 2) AS pct_of_max
FROM employees
ORDER BY pct_of_max DESC;
```

Output:

first_name	salary	pct_of_max
Eva	130000	100.00
Frank	110000	84.62
Bob	95000	73.08
Henry	88000	67.69
Alice	85000	65.38

TOPIC 3: Subquery in WHERE

Concept Explanation

A subquery in the `WHERE` clause is the **most common** use of subqueries. The outer query uses the result of the inner query as a **filter condition**.

Forms:

Form	When to use
<code>WHERE col = (subquery)</code>	Subquery returns exactly ONE value
<code>WHERE col IN (subquery)</code>	Subquery returns a list of values
<code>WHERE col > (subquery)</code>	Comparison with a single returned value
<code>WHERE EXISTS (subquery)</code>	Check if subquery returns ANY row
<code>WHERE NOT EXISTS (subquery)</code>	Check if subquery returns NO rows

Syntax

```
SELECT columns
FROM table
WHERE column operator (SELECT ... FROM ... WHERE ...);
```

Examples

Example 1: Employees Earning More Than Company Average

```
SELECT first_name, department, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

Subquery result: AVG(salary) = 85200

Output:

first_name	department	salary
Eva	Management	130000
Frank	IT	110000
Bob	IT	95000
Henry	IT	88000

Example 2: Employee with the Highest Salary

```
SELECT first_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Output:

first_name	salary
Eva	130000

Example 3: Employees in the Same Department as Alice

```
SELECT first_name, department
FROM employees
WHERE department = (SELECT department FROM employees WHERE first_name = 'Alice');
```

Subquery returns: 'Sales'

Output:

first_name	department
Alice	Sales
David	Sales
Isla	Sales

Example 4: Subquery with IN — Customers Who Have Placed Orders

```
SELECT name, city, country
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```

Subquery returns: (1, 2, 3, 4, 5, 6) — customers who placed orders

Output: All customers EXCEPT Carlos Ruiz (customer_id = 7).

Example 5: Customers Who Have NOT Placed Orders (NOT IN)

```
SELECT name, city
FROM customers
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders);
```

Output:

name	city
Carlos Ruiz	Madrid

Example 6: Products That Have Been Ordered

```
SELECT name, price
FROM products
WHERE product_id IN (SELECT DISTINCT product_id FROM orders);
```

Output: Laptop, Headphones, Smartphone, Desk Chair (excludes Monitor).

Example 7: Products That Have NEVER Been Ordered

```
SELECT name, price
FROM products
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM orders);
```

Output:

name	price
Monitor	15000

Example 8: Employees Whose Salary is Above Their Department Average

```
-- Step 1: For each employee, what is their dept's average?
-- Step 2: Where salary > that dept's average

SELECT e.first_name, e.department, e.salary
FROM employees AS e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees AS e2
    WHERE e2.department = e.department -- Match current employee's dept
)
ORDER BY e.department, e.salary DESC;
```

Output:

first_name	department	salary
Jake	Finance	82000
Frank	IT	110000
Bob	IT	95000
Eva	Management	130000
Alice	Sales	85000
Carol	HR	60000

Note: This is actually a Correlated Subquery — covered in Topic 4.

Example 9: Second Highest Salary (Classic Interview Question!)

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Outer query: MAX(salary) WHERE salary is NOT the absolute max
Subquery returns: 130000

Output:

second_highest_salary
110000

(Frank earns 110000 — the second highest)

Example 10: EXISTS — Customers Who Have at Least One Order

```
SELECT c.name, c.country
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
);
```

EXISTS returns TRUE if the subquery finds ANY row (even a single row with value 1). More efficient than IN for large datasets.

Output: All customers except Carlos Ruiz.

Example 11: NOT EXISTS — Customers With No Orders

```
SELECT c.name, c.country
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
);
```

Output:

name	country
Carlos Ruiz	Spain

Example 12: Subquery in HAVING

```
-- Departments whose average salary is above the company average
SELECT department, AVG(salary) AS dept_avg
FROM employees
GROUP BY department
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Output:

department	dept_avg
IT	97666.67
Management	130000.00

TOPIC 4: Correlated Subquery

📖 Concept Explanation

A **correlated subquery** is a subquery that **references columns from the outer query**. Unlike a regular subquery, it:

- Runs **once for every row** in the outer query
- Cannot run on its own (it depends on the outer query's current row)

Performance Note: Correlated subqueries can be SLOW on large tables because they run per row. Often replaceable with JOINS or Window Functions.

📄 Syntax

```
SELECT outer_col1, outer_col2
FROM outer_table AS ot
WHERE outer_col3 operator (
    SELECT AGG(inner_col)
    FROM inner_table AS it
    WHERE it.col = ot.col    -- ← This references the outer query!
);
```

🔍 Examples

Example 1: Employees Earning Above Their Own Department Average (Correlated)

```
SELECT e.first_name, e.department, e.salary
FROM employees AS e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = e.department    -- ← References outer 'e'
)
ORDER BY e.department;
```

How it works:

```
For Alice (Sales, 85000):
→ Subquery runs: AVG(salary) WHERE department = 'Sales' = 77333
→ 85000 > 77333? YES → Include Alice

For Bob (IT, 95000):
→ Subquery runs: AVG(salary) WHERE department = 'IT' = 97666
→ 95000 > 97666? NO → Exclude Bob
```

Output:

first_name	department	salary
Alice	Sales	85000
Frank	IT	110000
Carol	HR	60000

(Carol is the only HR employee — equals her own dept avg. In strict >, she might not show.)

Example 2: Orders Above Average Order Amount for That Customer

```
SELECT o.order_id, o.customer_id, o.amount
FROM orders AS o
WHERE o.amount > (
    SELECT AVG(amount)
    FROM orders
    WHERE customer_id = o.customer_id    -- ← References outer o
);
```

Output:

order_id	customer_id	amount
101	1	150000
102	2	30000

(For customer 1: avg = 112500; 150000 > 112500 ✅; 75000 < 112500 ❌)(For customer 2: avg = 22500; 30000 > 22500 ✅; 15000 < 22500 ❌)

Example 3: Customers Whose Latest Order is After 2024-01-10

```
SELECT c.name, c.country
FROM customers AS c
WHERE (
    SELECT MAX(order_date)
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
) > '2024-01-10';
```

```
WHERE o.customer_id = c.customer_id
) > '2024-01-10';
```

Output: Customers whose most recent order was placed after Jan 10, 2024.

Example 4: Correlated Subquery in SELECT (from Topic 2)

```
-- Order count per customer – using correlated subquery in SELECT
SELECT
  c.name,
  (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_count,
  (SELECT SUM(amount) FROM orders o WHERE o.customer_id = c.customer_id) AS total_spent
FROM customers AS c;
```

Example 5: Correlated vs Non-Correlated — Key Difference

```
-- NON-CORRELATED (runs once):
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
-- This subquery has NO reference to outer query
-- Runs ONCE → Returns 85200 → Used for all rows

-- CORRELATED (runs per row):
SELECT first_name, salary
FROM employees AS e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department = e.department);
-- This subquery REFERENCES e.department from outer query
-- Runs ONCE PER ROW → Different result for each employee
```

Example 6: Rewriting Correlated Subquery as JOIN (Performance Optimization)

```
-- Correlated Subquery (slower on large data):
SELECT e.first_name, e.department, e.salary
FROM employees AS e
WHERE e.salary > (
  SELECT AVG(salary) FROM employees WHERE department = e.department
);

-- Equivalent JOIN (faster):
SELECT e.first_name, e.department, e.salary
FROM employees AS e
INNER JOIN (
  SELECT department, AVG(salary) AS dept_avg
  FROM employees
  GROUP BY department
) AS dept ON e.department = dept.department
WHERE e.salary > dept.dept_avg;
```

Both return identical results; the JOIN version is generally faster because it doesn't re-run per row.

Practice Questions

● Easy Level

- Q1.** Write a subquery to find employees who earn more than Jake (emp_id = 10).
- Q2.** Write a subquery to find the name of the employee who earns the least.
- Q3.** Write a subquery to find all customers who have placed at least one order.
- Q4.** Write a subquery to find all products that have been ordered at least once.
- Q5.** Write a subquery to show each employee's salary and the overall company maximum salary.

● Medium Level

- Q6.** Write a subquery in WHERE to find all employees in the same city as Eva .
- Q7.** Write a query using NOT IN with a subquery to find customers who have never ordered a Laptop .
- Q8.** Write a query to find departments where the average salary is higher than the average salary of the Sales department.
- Q9.** Write a subquery to find the second highest salary in the employees table.
- Q10.** Write a query using EXISTS to find all customers who have placed at least 2 orders.
- Q11.** Write a correlated subquery to find employees who earn more than the average salary of their own department.

● Hard / Interview Level

- Q12.** Write a query to find the 3rd highest salary without using LIMIT/OFFSET.
- Q13.** Write a query to find employees whose salary is:
- Above the average of their own department
 - AND above the company-wide average
- Q14.** Write a query to find customers who have ordered EVERY product in the Electronics category.

Q15. Explain the difference between `IN` and `EXISTS` . When should you use each? What happens with NULLs in the subquery result?

Solutions

Q1.

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT salary FROM employees WHERE emp_id = 10);

-- Subquery returns: 82000 (Jake's salary)
-- Result: All employees earning more than 82000
```

Output:

first_name	salary
Alice	85000
Bob	95000
Eva	130000
Frank	110000
Henry	88000

Q2.

```
SELECT first_name, salary
FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);

-- Output: Grace, 55000
```

Q3.

```
SELECT name, country
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```

Q4.

```
SELECT name, price
FROM products
WHERE product_id IN (SELECT DISTINCT product_id FROM orders);
```

Output: Laptop, Smartphone, Headphones, Desk Chair (not Monitor)

Q5.

```
SELECT
    first_name,
    salary,
    (SELECT MAX(salary) FROM employees) AS company_max
FROM employees;
```

Q6.

```
SELECT first_name, city
FROM employees
WHERE city = (SELECT city FROM employees WHERE first_name = 'Eva');

-- Eva's city = 'Delhi'
-- Returns: Bob, Eva, Henry (all from Delhi)
```

Q7.

```
SELECT c.name
FROM customers AS c
WHERE c.customer_id NOT IN (
    SELECT o.customer_id
    FROM orders AS o
    INNER JOIN products AS p ON o.product_id = p.product_id
    WHERE p.name = 'Laptop'
);
```

Q8.

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = 'Sales'
);
```



```
-- Sales avg = 77333.33
-- Returns: IT (97666), Management (130000), Finance (82000)
```

Q9.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Output: 110000 (Frank)
```

Q10.

```
SELECT c.name, c.customer_id
FROM customers AS c
WHERE EXISTS (
    SELECT customer_id
    FROM orders
    WHERE customer_id = c.customer_id
    GROUP BY customer_id
    HAVING COUNT(*) >= 2
);

-- Output: Ravi Desai (2 orders), Priya Menon (2 orders)
```

Q11.

```
SELECT e.first_name, e.department, e.salary
FROM employees AS e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = e.department
)
ORDER BY e.department;
```

Q12.

```
-- 3rd highest salary (without LIMIT):
SELECT MIN(salary) AS third_highest
FROM (
    SELECT DISTINCT salary
    FROM employees
    ORDER BY salary DESC
    LIMIT 3
) AS top3;

-- Or using subquery nesting:
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (SELECT MAX(salary) FROM employees)
);

-- Output: 95000 (Bob)
```

Q13.

```
SELECT e.first_name, e.department, e.salary
FROM employees AS e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = e.department
)
AND e.salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Q14.

```
-- Customers who ordered EVERY Electronics product
-- (Advanced – uses COUNT matching)
SELECT c.customer_id, c.name
FROM customers AS c
WHERE (
    SELECT COUNT(DISTINCT p.product_id)
    FROM orders AS o
    INNER JOIN products AS p ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
```

```
        AND p.category = 'Electronics'
    ) = (
        SELECT COUNT(product_id)
        FROM products
        WHERE category = 'Electronics'
    );
```

Q15.

IN vs EXISTS:

IN:

- Returns TRUE if value matches ANY value in the subquery result list
- Best when subquery returns a small list of values
- Subquery result is loaded into memory all at once
- ⚠ If subquery returns ANY NULL, NOT IN returns EMPTY RESULT

EXISTS:

- Returns TRUE as soon as subquery finds the FIRST matching row (stops early)
- Best for large subquery results – stops at first match (faster)
- Use with correlated subqueries
- NOT EXISTS with NULLs is safe – no NULL issue

NULL Gotcha:

```
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE status = 'Cancelled')
--- If ANY customer_id in orders is NULL, NOT IN returns ZERO rows!
```

Safe alternative:

```
WHERE NOT EXISTS (
    SELECT 1 FROM orders
    WHERE orders.customer_id = customers.customer_id
    AND status = 'Cancelled'
)
```

🔗 Real-World Use Cases

Subquery Type	Data Science Application
Scalar in SELECT	Normalize values (% of max/min), compute deviation per row
Subquery in WHERE	Filter by computed threshold (above avg, top percentile)
IN subquery	Dynamic lists — filter based on another system's data
NOT IN / NOT EXISTS	Find missing records, data gaps, inactive users
Correlated subquery	Per-group comparisons, running totals (before window functions)
HAVING subquery	Remove low-confidence groups below population average

📘 Phase 7: Conditional & Transformations

TOPIC 1: CASE Statement

📘 Concept Explanation

CASE is SQL's **IF-THEN-ELSE** logic. It evaluates conditions and returns a value based on the first condition that is TRUE.

Why it matters for Data Science:

- Create categorical labels from numeric ranges (binning)
- Feature engineering — derive new columns from existing data
- Custom sorting and ranking
- Replace long chains of DECODE() or nested IFs

Two Forms:

1. Searched CASE (most flexible):

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ...
  ELSE default_result
END
```

2. Simple CASE (compare one column to fixed values):

```
CASE column
  WHEN value1 THEN result1
  WHEN value2 THEN result2
```



```
...
ELSE default_result
END
```



Syntax

```
SELECT column1,
       CASE
           WHEN condition1 THEN 'Result1'
           WHEN condition2 THEN 'Result2'
           ELSE 'Default'
       END AS alias_name
FROM table_name;
```

Key Rules:

- **THEN** must return the SAME data type for all branches
- **ELSE** is optional; without it, unmatched rows return NULL
- CASE can be used in SELECT, WHERE, ORDER BY, GROUP BY, and HAVING



Examples

Example 1: Salary Category (Binning Salaries)

```
SELECT
  first_name,
  salary,
  CASE
    WHEN salary < 65000 THEN 'Low'
    WHEN salary < 90000 THEN 'Medium'
    WHEN salary < 120000 THEN 'High'
    ELSE 'Very High'
  END AS salary_category
FROM employees;
```

Output:

first_name	salary	salary_category
Alice	85000	Medium
Bob	95000	High
Carol	60000	Low
David	75000	Medium
Eva	130000	Very High
Frank	110000	High
Grace	55000	Low
Henry	88000	Medium
Isla	72000	Medium
Jake	82000	Medium

Example 2: Simple CASE — Department Abbreviation

```
SELECT
  first_name,
  department,
  CASE department
    WHEN 'Sales' THEN 'SLS'
    WHEN 'IT' THEN 'IT'
    WHEN 'HR' THEN 'HR'
    WHEN 'Finance' THEN 'FIN'
    WHEN 'Management' THEN 'MGT'
    ELSE 'OTHER'
  END AS dept_code
FROM employees;
```

Output:

first_name	department	dept_code
Alice	Sales	SLS
Bob	IT	IT
Carol	HR	HR
David	Sales	SLS
Eva	Management	MGT

Example 3: Gender Label

```
SELECT
  first_name,
  gender,
  CASE gender
```

```
        WHEN 'M' THEN 'Male'
        WHEN 'F' THEN 'Female'
        ELSE 'Other'
    END AS gender_label
FROM employees;
```

Example 4: Order Status with Priority

```
SELECT
    order_id,
    status,
    CASE status
        WHEN 'Pending' THEN 1
        WHEN 'Delivered' THEN 2
        WHEN 'Cancelled' THEN 3
        ELSE 4
    END AS status_priority
FROM orders
ORDER BY status_priority;
```

Output: Orders sorted by custom logic (Pending first, then Delivered, then Cancelled).

Example 5: CASE in WHERE (Conditional Filtering)

```
-- Get senior employees (age > 35) in IT, or junior employees (age <= 35) in Sales
SELECT first_name, department, age
FROM employees
WHERE
    CASE
        WHEN department = 'IT' THEN age > 35
        WHEN department = 'Sales' THEN age <= 35
        ELSE FALSE
    END;
```

Example 6: CASE in ORDER BY (Custom Sort)

```
-- Sort: Management first, IT second, Sales third, others last
SELECT first_name, department, salary
FROM employees
ORDER BY
    CASE department
        WHEN 'Management' THEN 1
        WHEN 'IT' THEN 2
        WHEN 'Sales' THEN 3
        ELSE 4
    END ASC,
    salary DESC;
```

Output: Management employees first sorted by salary desc, then IT, then Sales, then others.

Example 7: CASE in GROUP BY — Count by Category

```
SELECT
    CASE
        WHEN salary < 65000 THEN 'Low'
        WHEN salary < 90000 THEN 'Medium'
        WHEN salary < 120000 THEN 'High'
        ELSE 'Very High'
    END AS salary_category,
    COUNT(*) AS employee_count,
    AVG(salary) AS avg_in_category
FROM employees
GROUP BY
    CASE
        WHEN salary < 65000 THEN 'Low'
        WHEN salary < 90000 THEN 'Medium'
        WHEN salary < 120000 THEN 'High'
        ELSE 'Very High'
    END;
```

Output:

salary_category	employee_count	avg_in_category
Low	2	57500
Medium	5	79800
High	2	102500
Very High	1	130000

Example 8: Pivot Table with CASE + SUM (Manual Pivot)

```
-- Count employees per department per gender (Manual Pivot)
SELECT
```

```
department,
SUM(CASE WHEN gender = 'F' THEN 1 ELSE 0 END) AS female_count,
SUM(CASE WHEN gender = 'M' THEN 1 ELSE 0 END) AS male_count,
COUNT(*) AS total
FROM employees
GROUP BY department;
```

Output:

department	female_count	male_count	total
Finance	0	1	1
HR	2	0	2
IT	0	3	3
Management	1	0	1
Sales	2	1	3

This is a KEY technique in Data Science — creating pivot tables entirely in SQL!

Example 9: Calculate Bonus Based on Performance Tier

```
SELECT
  first_name,
  salary,
  CASE
    WHEN salary >= 100000 THEN salary * 0.20
    WHEN salary >= 80000  THEN salary * 0.15
    WHEN salary >= 60000  THEN salary * 0.10
    ELSE                    salary * 0.05
  END AS bonus,
  CASE
    WHEN salary >= 100000 THEN 'Gold'
    WHEN salary >= 80000  THEN 'Silver'
    WHEN salary >= 60000  THEN 'Bronze'
    ELSE 'Standard'
  END AS tier
FROM employees
ORDER BY salary DESC;
```

Output:

first_name	salary	bonus	tier
Eva	130000	26000	Gold
Frank	110000	22000	Gold
Bob	95000	14250	Silver
Henry	88000	13200	Silver
Alice	85000	12750	Silver
Jake	82000	12300	Silver
David	75000	7500	Bronze
Isla	72000	7200	Bronze
Carol	60000	6000	Bronze
Grace	55000	2750	Standard

Example 10: Age Group Segmentation (ML Preprocessing)

```
-- Binning age for ML model feature engineering
SELECT
  first_name,
  age,
  CASE
    WHEN age BETWEEN 18 AND 25 THEN '18-25'
    WHEN age BETWEEN 26 AND 35 THEN '26-35'
    WHEN age BETWEEN 36 AND 45 THEN '36-45'
    WHEN age > 45                THEN '45+'
  END AS age_group
FROM employees
ORDER BY age;
```

TOPIC 2: NULL Handling — IS NULL & IS NOT NULL

Concept Explanation

`NULL` in SQL means **unknown** or **missing value**. It is NOT zero, NOT an empty string, NOT False — it is the absence of any value.

Critical NULL Rules:

- `NULL = NULL` is **FALSE** (NULL compared to anything is UNKNOWN)
- `NULL != NULL` is also **FALSE**
- Only `IS NULL` and `IS NOT NULL` correctly check for NULL

4. Arithmetic with NULL always returns NULL: `5 + NULL = NULL`
5. Aggregate functions (SUM, AVG, MAX) **ignore NULL values automatically**

Syntax

```
-- Find rows WHERE column is NULL:
SELECT * FROM table WHERE column IS NULL;

-- Find rows WHERE column is NOT NULL:
SELECT * FROM table WHERE column IS NOT NULL;

-- ❌ WRONG – always returns no rows:
SELECT * FROM table WHERE column = NULL;
SELECT * FROM table WHERE column != NULL;
```

Examples

Example 1: Find Employees Without a Manager

```
SELECT first_name, department, manager_id
FROM employees
WHERE manager_id IS NULL;
```

Output:

first_name	department	manager_id
Eva	Management	NULL
Frank	IT	NULL
Jake	Finance	NULL

Example 2: Find Employees WHO Have a Manager

```
SELECT first_name, manager_id
FROM employees
WHERE manager_id IS NOT NULL;
```

Output: Alice, Bob, Carol, David, Grace, Henry, Isla (7 employees with managers).

Example 3: NULL in Arithmetic (Common Trap)

```
-- NULL ruins arithmetic!
SELECT
    first_name,
    salary,
    manager_id,
    salary + manager_id AS weird_sum -- manager_id is NULL for some
FROM employees;
```

Output:

first_name	salary	manager_id	weird_sum
Alice	85000	5	85005
Bob	95000	6	95006
Eva	130000	NULL	NULL
Frank	110000	NULL	NULL

Eva and Frank's weird_sum is NULL because manager_id is NULL.

Example 4: COUNT(*) vs COUNT(column) with NULLs

```
SELECT
    COUNT(*)           AS all_rows,
    COUNT(manager_id) AS non_null_manager_ids
FROM employees;
```

Output:

all_rows	non_null_manager_ids
10	7

Example 5: NULL in WHERE — Common Mistake

```
-- ❌ WRONG – returns zero rows (NULL != NULL is UNKNOWN):
SELECT * FROM employees WHERE manager_id = NULL;

-- ✅ CORRECT:
SELECT * FROM employees WHERE manager_id IS NULL;
```

Example 6: Filter NULL vs Non-NULL in the Same Query

```
SELECT
    first_name,
    manager_id,
    CASE
        WHEN manager_id IS NULL THEN 'Top Level'
        ELSE 'Has Manager'
    END AS manager_status
FROM employees;
```

Output:

first_name	manager_id	manager_status
Alice	5	Has Manager
Bob	6	Has Manager
Carol	5	Has Manager
David	5	Has Manager
Eva	NULL	Top Level
Frank	NULL	Top Level
Grace	5	Has Manager
Henry	6	Has Manager
Isla	5	Has Manager
Jake	NULL	Top Level

Example 7: NULL in NOT IN — Critical Bug!

```
-- If NULLs appear in subquery, NOT IN returns ZERO rows!
CREATE TABLE temp_dept (dept_name VARCHAR(50));
INSERT INTO temp_dept VALUES ('Sales'), (NULL), ('IT');

-- ❌ This returns ZERO rows (because NULL is in the list):
SELECT * FROM employees
WHERE department NOT IN (SELECT dept_name FROM temp_dept);

-- ✅ Safe approach – always filter NULLs from subquery:
SELECT * FROM employees
WHERE department NOT IN (SELECT dept_name FROM temp_dept WHERE dept_name IS NOT NULL);
```

TOPIC 3: COALESCE Function

Concept Explanation

`COALESCE(value1, value2, ..., valueN)` returns the **first non-NULL value** from its list of arguments.

Why it's critical:

- Replace NULLs with meaningful defaults
- Handle missing data before displaying or computing
- Combine multiple possible sources into one value

Related functions:

Function	Description	Database
<code>COALESCE(a, b)</code>	First non-NULL of a, b	All
<code>IFNULL(a, b)</code>	Returns b if a is NULL	MySQL only
<code>ISNULL(a, b)</code>	Returns b if a is NULL	SQL Server
<code>NVL(a, b)</code>	Returns b if a is NULL	Oracle only
<code>NULLIF(a, b)</code>	Returns NULL if a = b, else returns a	All

Syntax

```
COALESCE(expression1, expression2, ..., expressionN)

-- Returns expression1 if it's NOT NULL
-- Returns expression2 if expression1 IS NULL and expression2 is NOT NULL
-- ...and so on
-- If ALL expressions are NULL, returns NULL
```

Examples

Example 1: Replace NULL Manager ID with a Default

```
SELECT
    first_name,
    COALESCE(manager_id, 0) AS manager_id
FROM employees;
```

Output:

first_name	manager_id
Alice	5
Bob	6
Eva	0
Frank	0

Example 2: Show 'No Manager' Instead of NULL

```
SELECT
    e.first_name,
    COALESCE(m.first_name, 'No Manager') AS manager_name
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.emp_id;
```

Output:

first_name	manager_name
Alice	Eva
Bob	Frank
Eva	No Manager
Frank	No Manager
Jake	No Manager

Example 3: Total Revenue Including Non-Ordering Customers

```
SELECT
    c.name,
    COALESCE(SUM(o.amount), 0) AS total_spent
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Output:

name	total_spent
Ravi Desai	225000
Priya Menon	45000
John Smith	9000
Sara Ali	75000
Chen Wei	150000
Meera Iyer	30000
Carlos Ruiz	0

Example 4: Multiple Fallbacks

```
-- Try column1, then column2, then column3, then use 'Unknown'
SELECT
    emp_id,
    COALESCE(city, department, 'Unknown Location') AS location
FROM employees;

-- city      → if not NULL, use city
-- department → if city is NULL but dept is not, use dept
-- 'Unknown Location' → if both are NULL
```

Example 5: COALESCE in Calculations

```
-- Avoid NULL results in bonus calculation
SELECT
    first_name,
    salary,
    COALESCE(manager_id, 0) AS mgr_id,
    salary * COALESCE(manager_id, 1) / 100 AS adjusted_value
FROM employees;
```

Example 6: NULLIF — Return NULL When Values Are Equal

```
-- NULLIF returns NULL if both arguments are equal, else returns first argument
-- Use case: Prevent division by zero!

SELECT
    first_name,
    salary,
    10 / NULLIF(0, 0) AS safe_division -- Returns NULL instead of ERROR
FROM employees;

-- More typical use: division by a column that might be zero
SELECT
    dept_name,
```



```
budget / NULLIF(employee_count, 0) AS budget_per_employee
FROM departments_summary;
```

Example 7: COALESCE for Data Cleaning

```
-- Replace missing city data with 'Unknown'
UPDATE employees
SET city = COALESCE(city, 'Unknown')
WHERE city IS NULL;

-- Clean display without updating:
SELECT
    first_name,
    COALESCE(city, 'Unknown') AS city
FROM employees;
```

Example 8: Combined CASE + COALESCE + NULL Handling

```
SELECT
    e.first_name,
    e.department,
    e.salary,
    COALESCE(m.first_name, 'Self-Managed') AS manager,
    CASE
        WHEN e.manager_id IS NULL THEN 'Executive'
        WHEN e.salary > 90000 THEN 'Senior'
        ELSE 'Regular'
    END AS employee_level
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.emp_id;
```

Output:

first_name	department	salary	manager	employee_level
Alice	Sales	85000	Eva	Regular
Bob	IT	95000	Frank	Senior
Carol	HR	60000	Eva	Regular
David	Sales	75000	Eva	Regular
Eva	Management	130000	Self-Managed	Executive
Frank	IT	110000	Self-Managed	Executive
Grace	HR	55000	Eva	Regular
Henry	IT	88000	Frank	Regular
Isla	Sales	72000	Eva	Regular
Jake	Finance	82000	Self-Managed	Executive

Practice Questions

Easy Level

- Q1.** Write a query to label employees as 'Senior' if age > 35, else 'Junior'.
- Q2.** Write a query to show employees with NULL manager_id and label them as 'Top Level Executive'.
- Q3.** Write a query to replace NULL manager_ids with 0 using COALESCE.
- Q4.** Write a query to count how many employees have NULL manager_id.
- Q5.** Write a CASE statement to categorize products by price:
- Under 5000 → 'Budget'
 - 5001 to 30000 → 'Mid-Range'
 - Above 30000 → 'Premium'

Medium Level

- Q6.** Write a query that shows each employee, their salary, and a raise_amount column computed as:
- 20% raise if salary < 65000
 - 15% raise if salary between 65000 and 90000
 - 10% raise otherwise
- Q7.** Write a CASE in ORDER BY to sort employees: IT department first, Sales second, all others last. Within each group, sort by salary descending.
- Q8.** Using CASE and SUM, create a pivot table showing:
- department
 - number of employees from Mumbai
 - number of employees from Delhi
 - total employees
- Q9.** Write a query to show each order's status as:
- 'Completed' instead of 'Delivered'

- 'In Progress' instead of 'Pending'
- 'Rejected' instead of 'Cancelled'

Q10. Write a query showing each customer's name, total orders, total spent (use 0 if no orders). Use LEFT JOIN and COALESCE.

● Hard / Interview Level

Q11. Write a query to compute the following for each employee:

- `salary_vs_avg` : 'Above Average', 'At Average', or 'Below Average' relative to company average
- `dept_rank` : 'Top Earner' if highest in dept, 'Lowest Earner' if lowest, else 'Middle'

Q12. Using only CASE + SUM (no pivot tables or window functions), write a query to show:

- How many orders were placed each month (Jan=1, Feb=2, Mar=3)

Q13. Explain the difference between:

```
COALESCE(a, b)
NULLIF(a, b)
IFNULL(a, b)
```

Give an example of when each would be used.

Q14. Write a query to find the "safe average salary" per department — if a department has zero salaried employees (salary is NULL), show 0 instead of NULL.

Solutions

Q1.

```
SELECT
  first_name,
  age,
  CASE
    WHEN age > 35 THEN 'Senior'
    ELSE 'Junior'
  END AS experience_level
FROM employees;
```

Q2.

```
SELECT first_name, department, manager_id
FROM employees
WHERE manager_id IS NULL;

-- Or with label:
SELECT first_name, 'Top Level Executive' AS role
FROM employees
WHERE manager_id IS NULL;
```

Q3.

```
SELECT first_name, COALESCE(manager_id, 0) AS manager_id
FROM employees;
```

Q4.

```
SELECT COUNT(*) AS null_manager_count
FROM employees
WHERE manager_id IS NULL;
-- Output: 3
```

Q5.

```
SELECT
  name,
  price,
  CASE
    WHEN price <= 5000 THEN 'Budget'
    WHEN price <= 30000 THEN 'Mid-Range'
    ELSE 'Premium'
  END AS price_category
FROM products;
```

Q6.

```
SELECT
  first_name,
  salary,
  CASE
    WHEN salary < 65000 THEN salary * 0.20
    WHEN salary BETWEEN 65000 AND 90000 THEN salary * 0.15
    ELSE salary * 0.10
  END AS raise_amount
FROM employees;
```


Q7.

```
SELECT first_name, department, salary
FROM employees
ORDER BY
    CASE department
        WHEN 'IT'      THEN 1
        WHEN 'Sales' THEN 2
        ELSE 3
    END ASC,
    salary DESC;
```

Q8.

```
SELECT
    department,
    SUM(CASE WHEN city = 'Mumbai' THEN 1 ELSE 0 END) AS mumbai_count,
    SUM(CASE WHEN city = 'Delhi'  THEN 1 ELSE 0 END) AS delhi_count,
    COUNT(*) AS total_count
FROM employees
GROUP BY department
ORDER BY department;
```

Output:

department	mumbai_count	delhi_count	total_count
Finance	0	0	1
HR	0	0	2
IT	0	2	3
Management	0	1	1
Sales	3	0	3

Q9.

```
SELECT
    order_id,
    amount,
    CASE status
        WHEN 'Delivered' THEN 'Completed'
        WHEN 'Pending'   THEN 'In Progress'
        WHEN 'Cancelled' THEN 'Rejected'
        ELSE status
    END AS order_status
FROM orders;
```

Q10.

```
SELECT
    c.name,
    COUNT(o.order_id)      AS total_orders,
    COALESCE(SUM(o.amount), 0) AS total_spent
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;
```

Q11.

```
SELECT
    e.first_name,
    e.department,
    e.salary,
    CASE
        WHEN e.salary > (SELECT AVG(salary) FROM employees) THEN 'Above Average'
        WHEN e.salary = (SELECT AVG(salary) FROM employees) THEN 'At Average'
        ELSE 'Below Average'
    END AS salary_vs_avg,
    CASE
        WHEN e.salary = (SELECT MAX(salary) FROM employees e2 WHERE e2.department = e.department) THEN 'Top Earner'
        WHEN e.salary = (SELECT MIN(salary) FROM employees e3 WHERE e3.department = e.department) THEN 'Lowest Earner'
        ELSE 'Middle'
    END AS dept_rank
FROM employees AS e;
```

Q12.

```
SELECT
    SUM(CASE WHEN MONTH(order_date) = 1 THEN 1 ELSE 0 END) AS january_orders,
    SUM(CASE WHEN MONTH(order_date) = 2 THEN 1 ELSE 0 END) AS february_orders,
    SUM(CASE WHEN MONTH(order_date) = 3 THEN 1 ELSE 0 END) AS march_orders
FROM orders;
```

Output:

january_orders	february_orders	march_orders
3	3	2

Q13.

```
COALESCE(a, b):
→ Returns first non-NULL value from its arguments
→ Can take MULTIPLE arguments: COALESCE(a, b, c, d)
→ Use: Replace NULLs with default values
    COALESCE(city, 'Unknown') -- if city is NULL, return 'Unknown'

NULLIF(a, b):
→ Returns NULL if a = b; otherwise returns a
→ Use: Prevent division by zero
    10 / NULLIF(count, 0) -- if count = 0, return NULL instead of division error

IFNULL(a, b):
→ MySQL-only shorthand for COALESCE(a, b) with exactly 2 arguments
→ Returns b if a is NULL, else returns a
→ Use: Same as COALESCE with 2 args, but not portable (MySQL only)
    IFNULL(manager_id, 0) -- same as COALESCE(manager_id, 0)
```

Q14.

```
SELECT
    department,
    COALESCE(AVG(NULLIF(salary, 0)), 0) AS safe_avg_salary
FROM employees
GROUP BY department;

-- Or simpler:
SELECT
    department,
    COALESCE(AVG(salary), 0) AS avg_salary
FROM employees
GROUP BY department;
-- AVG already ignores NULLs; COALESCE handles case where all salaries in group are NULL
```

 Real-World Use Cases

Feature	Data Science Application
CASE	Bin continuous variables (age, salary) for ML categorical features
CASE SUM	Manual pivot — cohort analysis, time-series aggregation by category
IS NULL	Data quality checks, find incomplete records before model training
IS NOT NULL	Filter only complete cases for analysis
COALESCE	Imputation — fill missing values with defaults for ML preprocessing
NULLIF	Safe division for ratio features (avoid divide-by-zero errors)

ML Feature Engineering Example:

```
-- Prepare customer features for churn prediction with NULL handling:
SELECT
    c.customer_id,
    c.age,
    COALESCE(c.city, 'Unknown')           AS city,
    CASE
        WHEN c.age < 30 THEN 'Young'
        WHEN c.age < 50 THEN 'Mid-Age'
        ELSE 'Senior'
    END AS age_group,
    COALESCE(COUNT(o.order_id), 0)         AS order_count,
    COALESCE(SUM(o.amount), 0)             AS lifetime_value,
    CASE
        WHEN COUNT(o.order_id) = 0 THEN 1
        ELSE 0
    END AS is_churned
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.age, c.city;
```

 Phase 8: String Functions

TOPIC 1: CONCAT

Concept Explanation

`CONCAT()` joins (concatenates) two or more strings together into a single string.

Why it matters:

- Build full names from first/last name columns
- Create composite keys or identifiers
- Format output strings for reports
- Combine email addresses, URLs, addresses

Syntax

```
-- MySQL, PostgreSQL:
CONCAT(string1, string2, ..., stringN)

-- PostgreSQL also supports || operator:
string1 || ' ' || string2

-- SQL Server:
CONCAT(string1, string2)  -- or use +
string1 + ' ' + string2
```

Important: If ANY argument is NULL, MySQL's CONCAT returns NULL. Use `CONCAT_WS` to ignore NULLs.

Examples

Example 1: Combine First and Last Name

```
SELECT
    emp_id,
    CONCAT(first_name, ' ', last_name) AS full_name,
    department
FROM employees;
```

Output:

emp_id	full_name	department
1	Alice Kumar	Sales
2	Bob Singh	IT
3	Carol Rao	HR
4	David Mehta	Sales
5	Eva Sharma	Management

Example 2: Create Email from Name

```
SELECT
    first_name,
    CONCAT(LOWER(first_name), '@company.com') AS company_email
FROM employees;
```

Output:

first_name	company_email
Alice	alice@company.com
Bob	bob@company.com
Carol	carol@company.com

Example 3: Build Full Address

```
SELECT
    name AS customer_name,
    CONCAT(city, ', ', country) AS location
FROM customers;
```

Output:

customer_name	location
Ravi Desai	Mumbai, India
Priya Menon	Delhi, India
John Smith	New York, USA
Sara Ali	London, UK
Chen Wei	Shanghai, China

Example 4: CONCAT with Numbers

```
SELECT
    CONCAT('Employee ID: ', emp_id, ' | Name: ', first_name) AS label
FROM employees;
```

Output:

label
Employee ID: 1 Name: Alice
Employee ID: 2 Name: Bob

Example 5: CONCAT_WS — Separator with NULL Safety

```
-- CONCAT_WS(separator, string1, string2, ...) skips NULLs automatically
SELECT
    emp_id,
    CONCAT_WS(' | ', first_name, last_name, department, city) AS profile
FROM employees;
```

Output:

emp_id	profile
1	Alice Kumar Sales Mumbai
2	Bob Singh IT Delhi

Example 6: PostgreSQL || Operator

```
-- PostgreSQL style:
SELECT first_name || ' ' || last_name AS full_name
FROM employees;
```

Example 7: Concatenate for Dynamic Messages

```
SELECT
    first_name,
    salary,
    CONCAT(first_name, ' earns ₹', FORMAT(salary, 0), ' per month.') AS summary
FROM employees;
```

TOPIC 2: LENGTH

Concept Explanation

`LENGTH()` returns the number of characters in a string.

Variants:

- `LENGTH(str)` → MySQL: number of bytes (same as chars for ASCII); PostgreSQL: number of chars
- `CHAR_LENGTH(str)` → Number of characters (handles multi-byte Unicode correctly)
- `LEN(str)` → SQL Server equivalent

Syntax

```
SELECT LENGTH(column_name) AS char_count
FROM table_name;

-- For Unicode safety:
SELECT CHAR_LENGTH(column_name) AS char_count
FROM table_name;
```

Examples

Example 1: Length of Each First Name

```
SELECT
    first_name,
    LENGTH(first_name) AS name_length
FROM employees
ORDER BY name_length;
```

Output:

first_name	name_length
Bob	3
Eva	3
Isla	4
Jake	4
Alice	5
Carol	5
David	5
Frank	5
Grace	5

first_name	name_length
Henry	5

Example 2: Find Names Longer Than 4 Characters

```
SELECT first_name
FROM employees
WHERE LENGTH(first_name) > 4;
```

Output: Alice, Carol, David, Frank, Grace, Henry

Example 3: Email Length Validation

```
SELECT name, email, LENGTH(email) AS email_length
FROM customers
WHERE LENGTH(email) > 20;
```

Example 4: Combined LENGTH and LIKE

```
-- Find employees with exactly 5-character first names
SELECT first_name
FROM employees
WHERE LENGTH(first_name) = 5;
```

Example 5: Longest Name

```
SELECT first_name, LENGTH(first_name) AS len
FROM employees
ORDER BY len DESC
LIMIT 1;
```

Example 6: LENGTH in Data Quality Check

```
-- Find invalid phone numbers (not 10 digits)
SELECT customer_id, phone
FROM customers
WHERE LENGTH(REPLACE(phone, '-', '')) != 10;

-- Find short product names (might be data entry errors)
SELECT name FROM products WHERE LENGTH(name) < 3;
```

TOPIC 3: SUBSTRING

Concept Explanation

`SUBSTRING()` (or `SUBSTR()`) extracts a portion of a string starting from a given position.

Variants:

- `SUBSTRING(str, start, length)` — standard SQL
- `SUBSTR(str, start, length)` — Oracle, SQLite shorthand
- `LEFT(str, n)` — first n characters
- `RIGHT(str, n)` — last n characters

Positions are 1-indexed in SQL (first character = position 1)

Syntax

```
SUBSTRING(string, start_position, length)

-- start_position: character to start from (1 = first character)
-- length: how many characters to extract
-- If length is omitted: extracts from start to end of string
```

Examples

Example 1: First 3 Characters of Name

```
SELECT
    first_name,
    SUBSTRING(first_name, 1, 3) AS first_3_chars
FROM employees;
```

Output:

first_name	first_3_chars
Alice	Ali
Bob	Bob

first_name	first_3_chars
Carol	Car
David	Dav
Eva	Eva

Example 2: Extract Year from Date String

```
-- If dates are stored as strings (bad practice, but happens)
SELECT
    order_id,
    SUBSTRING(CAST(order_date AS CHAR), 1, 4) AS order_year
FROM orders;
```

Example 3: Extract Email Domain

```
SELECT
    name,
    email,
    SUBSTRING(email, LOCATE('@', email) + 1) AS email_domain
FROM customers;
```

Output:

name	email	email_domain
Ravi Desai	ravi@email.com	email.com
Priya Menon	priya@email.com	email.com

Example 4: LEFT() — First N Characters

```
SELECT
    first_name,
    LEFT(first_name, 1) AS initial,
    CONCAT(LEFT(first_name, 1), '. ', last_name) AS formal_name
FROM employees;
```

Output:

first_name	initial	formal_name
Alice	A	A. Kumar
Bob	B	B. Singh
Carol	C	C. Rao

Example 5: RIGHT() — Last N Characters

```
SELECT
    email,
    RIGHT(email, 3) AS tld -- top-level domain (.com, .org, etc.)
FROM customers;
```

Output:

email	tld
ravi@email.com	com
priya@email.com	com

Example 6: Masking Sensitive Data

```
-- Show only last 4 digits of a credit card:
SELECT
    customer_id,
    CONCAT('****-****-****-', RIGHT(card_number, 4)) AS masked_card
FROM payment_info;

-- For names – show only first 2 characters:
SELECT CONCAT(LEFT(first_name, 2), '***') AS masked_name
FROM employees;
```

Example 7: Extract Month from Date

```
SELECT
    order_id,
    order_date,
    SUBSTRING(CAST(order_date AS CHAR), 6, 2) AS month
FROM orders;

-- Better way (covered in Phase 9):
SELECT order_id, MONTH(order_date) AS month FROM orders;
```

Example 8: MID() — MySQL Alias for SUBSTRING


```
-- MySQL also supports MID() which works exactly like SUBSTRING()
SELECT MID('Hello World', 7, 5);    -- Returns 'World'
SELECT SUBSTRING('Hello World', 7, 5); -- Same result: 'World'
```

TOPIC 4: UPPER & LOWER

Concept Explanation

- `UPPER(str)` — Converts all characters to UPPERCASE
- `LOWER(str)` — Converts all characters to lowercase

Why it matters:

- Case-insensitive search and comparison
- Standardizing data before storage
- Creating usernames, email addresses from names

Syntax

```
SELECT UPPER(column_name) FROM table;
SELECT LOWER(column_name) FROM table;

-- SQL Server alias:
UCASE() = UPPER()
LCASE() = LOWER()
```

Examples

Example 1: Convert Names to Uppercase

```
SELECT
    UPPER(first_name) AS first_name_upper,
    UPPER(last_name)  AS last_name_upper
FROM employees;
```

Output:

first_name_upper	last_name_upper
ALICE	KUMAR
BOB	SINGH
CAROL	RAO

Example 2: Convert to Lowercase

```
SELECT
    LOWER(first_name) AS first_name_lower,
    LOWER(CONCAT(first_name, '.', last_name, '@company.in')) AS email
FROM employees;
```

Output:

first_name_lower	email
alice	alice.kumar@company.in
bob	bob.singh@company.in

Example 3: Case-Insensitive Search

```
-- Search regardless of how data is stored
SELECT first_name FROM employees
WHERE LOWER(first_name) = LOWER('ALICE');

-- Also finds: 'Alice', 'ALICE', 'alice', 'aLiCe'
```

Example 4: Standardize Department Names

```
-- If department column has inconsistent casing:
-- 'Sales', 'SALES', 'sales' → all become 'SALES'
SELECT UPPER(department) AS dept_standard
FROM employees;

-- Or standardize on update:
UPDATE employees SET department = UPPER(department);
```

Example 5: Proper Case (Title Case) — Custom Function

```
-- SQL doesn't have a built-in INITCAP in MySQL (PostgreSQL has it)
-- PostgreSQL:
```

```
SELECT INITCAP(LOWER('ALICE KUMAR')) AS proper_case;
-- Output: 'Alice Kumar'

-- MySQL workaround with CONCAT:
SELECT CONCAT(
    UPPER(LEFT(first_name, 1)),
    LOWER(SUBSTRING(first_name, 2))
) AS proper_case_name
FROM employees;
```

Example 6: UPPER in WHERE for Case-Insensitive Filter

```
SELECT *
FROM employees
WHERE UPPER(city) = 'MUMBAI';

-- Finds: 'Mumbai', 'MUMBAI', 'mumbai'
```

Example 7: TRIM — Remove Extra Spaces

```
-- Often used alongside UPPER/LOWER for data cleaning:
TRIM(' Alice ')      -- → 'Alice'   (removes both leading and trailing spaces)
LTRIM(' Alice ')     -- → 'Alice '  (removes leading spaces only)
RTRIM(' Alice ')     -- → ' Alice'  (removes trailing spaces only)

-- Combined cleaning:
SELECT TRIM(UPPER(city)) AS clean_city FROM employees;
```

TOPIC 5: Other Essential String Functions

📖 Quick Reference of All Important String Functions:

Function	Description	Example
CONCAT(a, b)	Joins strings	CONCAT('Hello', ' World') → 'Hello World'
CONCAT_WS(sep,...)	Join with separator, skips NULLs	CONCAT_WS('-', 'a', 'b') → 'a-b'
LENGTH(s)	Length in bytes	LENGTH('Hello') → 5
CHAR_LENGTH(s)	Length in characters (Unicode safe)	CHAR_LENGTH('Hello') → 5
UPPER(s)	Convert to uppercase	UPPER('hello') → 'HELLO'
LOWER(s)	Convert to lowercase	LOWER('HELLO') → 'hello'
SUBSTRING(s,p,l)	Extract from position p, length l	SUBSTRING('Hello',2,3) → 'ell'
LEFT(s, n)	First n characters	LEFT('Hello', 3) → 'Hel'
RIGHT(s, n)	Last n characters	RIGHT('Hello', 3) → 'llo'
TRIM(s)	Remove spaces from both ends	TRIM(' Hi ') → 'Hi'
LTRIM(s)	Remove leading spaces	LTRIM(' Hi') → 'Hi'
RTRIM(s)	Remove trailing spaces	RTRIM('Hi ') → 'Hi'
REPLACE(s,f,r)	Replace all occurrences of f with r	REPLACE('Hello', 'l', '!') → 'He!!o'
LOCATE(f, s)	Position of first occurrence of f in s	LOCATE('@', 'a@b.com') → 2
INSTR(s, f)	MySQL: same as LOCATE	INSTR('a@b.com', '@') → 2
REPEAT(s, n)	Repeat s n times	REPEAT('ha', 3) → 'hahaha'
REVERSE(s)	Reverse the string	REVERSE('abc') → 'cba'
LPAD(s, n, p)	Pad left with p until length n	LPAD('5', 3, '0') → '005'
RPAD(s, n, p)	Pad right with p until length n	RPAD('Hi', 5, '!') → 'Hi...'
FORMAT(n, d)	Format number with d decimal places	FORMAT(85000, 0) → '85,000'

Examples:

```
-- REPLACE: Fix typos or clean data
SELECT REPLACE('Slaes', 'Slaes', 'Sales'); -- → 'Sales'
SELECT REPLACE(phone, '-', '') AS clean_phone FROM customers;

-- LPAD: Zero-pad ID numbers
SELECT LPAD(emp_id, 5, '0') AS padded_id FROM employees;
-- Output: 00001, 00002, 00003...

-- REPEAT: Create separators
SELECT REPEAT('=', 30) AS separator;
-- Output: =====

-- REVERSE: Palindrome check (conceptual)
SELECT first_name FROM employees
WHERE first_name = REVERSE(first_name);
```



```
-- LOCATE: Find position of a character
SELECT email, LOCATE('@', email) AS at_position FROM customers;
-- Output: ravi@email.com → 5 (@ is at position 5)
```

Practice Questions

● Easy Level

- Q1.** Write a query to display each employee's full name (first + last) separated by a space.
- Q2.** Write a query to display employee names in ALL CAPS.
- Q3.** Write a query to find the length of each customer's name, sorted longest first.
- Q4.** Write a query to extract only the first 3 characters of each employee's first name.
- Q5.** Write a query to create a company email for each employee in format: `firstname.lastname@company.com` (all lowercase).

● Medium Level

- Q6.** Write a query to show:
- Customer name
 - Email domain (everything after the `@`)
 - Email username (everything before the `@`)
- Q7.** Write a query to find all employees whose first name starts with a vowel (A, E, I, O, U). Use SUBSTRING or LEFT + UPPER.
- Q8.** Write a query to mask employee names: show only the first 2 characters of first name followed by `***`. Example: `Al***`.
- Q9.** Write a query to create a unique employee code in this format: `DEPT_ABBR-000EMP_ID`. Example: `SAL-00001` for Alice (Sales, emp_id=1).
- Use LEFT(department, 3) for dept abbreviation
 - Use LPAD(emp_id, 5, '0') for the ID
- Q10.** Write a query to count how many employees' full names (first + last) have more than 10 characters.

● Hard / Interview Level

- Q11.** Write a query to extract the year, month, and day from order_date as separate string columns WITHOUT using date functions (use SUBSTRING or CAST).
- Q12.** A data quality check is needed. Write a query to find all customers whose email is "invalid" — meaning it:
- Does NOT contain `@`
 - OR has a domain shorter than 4 characters
 - OR the total email length is less than 6
- Q13.** Write a query to reverse each employee's name and find any employees whose name reads the same forwards and backwards (palindromes).
- Q14.** Write a query to generate an employee badge string like:
`[ID:001] Alice Kumar | Sales | Mumbai | ₹85,000`
- Use: LPAD for ID, CONCAT, FORMAT for salary.

Solutions

Q1.

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

Q2.

```
SELECT
    UPPER(CONCAT(first_name, ' ', last_name)) AS full_name_upper
FROM employees;
```

Q3.

```
SELECT
    name,
    LENGTH(name) AS name_length
FROM customers
ORDER BY name_length DESC;
```

Q4.

```
SELECT
    first_name,
    SUBSTRING(first_name, 1, 3) AS name_short
FROM employees;
```

Q5.

```
SELECT
    first_name,
    last_name,
```

```
LOWER(CONCAT(first_name, '.', last_name, '@company.com')) AS company_email
FROM employees;
```

Output:

first_name	last_name	company_email
Alice	Kumar	alice.kumar@company.com
Bob	Singh	bob.singh@company.com

Q6.

```
SELECT
  name,
  email,
  LEFT(email, LOCATE('@', email) - 1)      AS email_username,
  SUBSTRING(email, LOCATE('@', email) + 1)  AS email_domain
FROM customers;
```

Output:

name	email	email_username	email_domain
Ravi Desai	ravi@email.com	ravi	email.com
Priya Menon	priya@email.com	priya	email.com

Q7.

```
SELECT first_name
FROM employees
WHERE UPPER(LEFT(first_name, 1)) IN ('A', 'E', 'I', 'O', 'U');
```

Output: Alice, Eva, Isla

Q8.

```
SELECT
  first_name,
  CONCAT(LEFT(first_name, 2), '***') AS masked_name
FROM employees;
```

Output:

first_name	masked_name
Alice	A***
Bob	Bo***
Carol	Ca***

Q9.

```
SELECT
  emp_id,
  first_name,
  department,
  CONCAT(
    UPPER(LEFT(department, 3)),
    '-',
    LPAD(emp_id, 5, '0')
  ) AS employee_code
FROM employees;
```

Output:

emp_id	first_name	department	employee_code
1	Alice	Sales	SAL-00001
2	Bob	IT	IT-00002
3	Carol	HR	HR-00003
5	Eva	Management	MAN-00005

Q10.

```
SELECT COUNT(*) AS count_above_10
FROM employees
WHERE LENGTH(CONCAT(first_name, last_name)) > 10;
```

Q11.

```
SELECT
  order_id,
  order_date,
  SUBSTRING(CAST(order_date AS CHAR), 1, 4) AS order_year,
  SUBSTRING(CAST(order_date AS CHAR), 6, 2) AS order_month,
  SUBSTRING(CAST(order_date AS CHAR), 9, 2) AS order_day
FROM orders;
```

Q12.

```
SELECT customer_id, name, email
FROM customers
WHERE
    email NOT LIKE '%@%'
    OR LENGTH(SUBSTRING(email, LOCATE('@', email) + 1)) < 4
    OR LENGTH(email) < 6;
```

Q13.

```
SELECT first_name
FROM employees
WHERE first_name = REVERSE(first_name);

-- Output: None in current dataset. 'Eva' → REVERSE = 'avE' (case-sensitive mismatch)
-- Case-insensitive check:
WHERE LOWER(first_name) = LOWER(REVERSE(first_name));
-- 'Eva' → lower → 'eva', reverse('eva') → 'ave' → NOT a palindrome
-- 'Bob' → lower → 'bob', reverse('bob') → 'bob'  ✅ PALINDROME!
```

Output: Bob

Q14.

```
SELECT
    CONCAT(
        '[ID:', LPAD(emp_id, 3, '0'), '] ',
        first_name, ' ', last_name,
        ' | ', department,
        ' | ', city,
        ' | ₹', FORMAT(salary, 0)
    ) AS employee_badge
FROM employees;
```

Output:

employee_badge

[ID:001] Alice Kumar | Sales | Mumbai | ₹85,000

[ID:002] Bob Singh | IT | Delhi | ₹95,000

Real-World Use Cases

Function	Data Science Application
CONCAT	Build composite keys, feature strings, profile identifiers
LENGTH	Data quality — detect missing/short values before model training
SUBSTRING	Extract standardized codes, date parts from strings
UPPER/LOWER	Standardize categories for consistent grouping and encoding
TRIM	Data cleaning — remove whitespace from imported CSV data
REPLACE	Fix typos, normalize categories ('Slaes' → 'Sales')
LPAD	Create fixed-width codes for matching with other systems

Phase 9: Date Functions

 **Database Note:** Date function names differ slightly across databases.

This file covers **MySQL (primary)** with **PostgreSQL equivalents** noted.

TOPIC 1: CURRENT DATE / NOW()

Concept Explanation

Functions to get the **current date and/or time** from the database server.

Function	Returns	Example Output	Database
<code>CURRENT_DATE</code>	Current date only	2024-03-30	All
<code>CURDATE()</code>	Current date only	2024-03-30	MySQL only
<code>NOW()</code>	Current date + time	2024-03-30 18:45:00	MySQL, PostgreSQL
<code>CURRENT_TIMESTAMP</code>	Current date + time	2024-03-30 18:45:00	All
<code>GETDATE()</code>	Current date + time	2024-03-30 18:45:00	SQL Server
<code>SYSDATE()</code>	Current date + time	2024-03-30 18:45:00	Oracle

Syntax

```
SELECT CURRENT_DATE;           -- Standard SQL
SELECT CURDATE();              -- MySQL shorthand
SELECT NOW();                  -- Date + Time
SELECT CURRENT_TIMESTAMP;      -- Standard SQL, same as NOW()
```

Examples

Example 1: Get Today's Date

```
SELECT
  CURRENT_DATE    AS today,
  CURDATE()       AS also_today,
  NOW()           AS now_with_time,
  CURRENT_TIMESTAMP AS full_timestamp;
```

Output (as of 2024-03-30):

today	also_today	now_with_time	full_timestamp
2024-03-30	2024-03-30	2024-03-30 18:45:00	2024-03-30 18:45:00

Example 2: How Many Days Since Each Employee Was Hired

```
SELECT
  first_name,
  hire_date,
  DATEDIFF(CURRENT_DATE, hire_date) AS days_at_company
FROM employees
ORDER BY days_at_company DESC;
```

Output (approximate, assuming today = 2024-03-30):

first_name	hire_date	days_at_company
Eva	2015-01-01	3376
Frank	2016-07-15	2815
David	2017-09-10	2393
Bob	2018-06-01	2128
Henry	2019-04-22	1804

Example 3: Orders Placed in the Last 30 Days

```
SELECT order_id, order_date, amount
FROM orders
WHERE order_date >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY);

-- PostgreSQL equivalent:
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
```

Example 4: Insert with Default Current Date

```
-- When creating tables:
CREATE TABLE activity_log (
  log_id      INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT,
  action      VARCHAR(100),
  logged_at   DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Manual insert with today's date:
INSERT INTO orders (order_id, customer_id, amount, order_date)
VALUES (109, 1, 50000, CURRENT_DATE);
```

Example 5: How Long Ago Each Order Was Placed

```
SELECT
  order_id,
  order_date,
  DATEDIFF(CURRENT_DATE, order_date) AS days_ago
FROM orders
ORDER BY order_date DESC;
```

TOPIC 2: EXTRACT — YEAR, MONTH, DAY

Concept Explanation

Extract specific parts (year, month, day, hour, etc.) from a date or datetime value.

Why it matters for Data Science:

- Group sales by month or year (time-series analysis)
- Find seasonal patterns in data
- Calculate age from birth date
- Time-based feature engineering for ML models



Syntax — Database Comparison

Part	MySQL	PostgreSQL	SQL Server
Year	<code>YEAR(date)</code> or <code>EXTRACT(YEAR FROM date)</code>	<code>EXTRACT(YEAR FROM date)</code>	<code>YEAR(date)</code>
Month	<code>MONTH(date)</code> or <code>EXTRACT(MONTH FROM date)</code>	<code>EXTRACT(MONTH FROM date)</code>	<code>MONTH(date)</code>
Day	<code>DAY(date)</code> or <code>EXTRACT(DAY FROM date)</code>	<code>EXTRACT(DAY FROM date)</code>	<code>DAY(date)</code>
Hour	<code>hour(datetime)</code>	<code>EXTRACT(HOUR FROM datetime)</code>	<code>DATEPART(hour, dt)</code>
Quarter	<code>QUARTER(date)</code>	<code>EXTRACT(QUARTER FROM date)</code>	<code>DATEPART(quarter, dt)</code>
Week	<code>WEEK(date)</code>	<code>EXTRACT(WEEK FROM date)</code>	<code>DATEPART(week, dt)</code>
Weekday	<code>DAYOFWEEK(date)</code>	<code>EXTRACT(DOW FROM date)</code>	<code>DATEPART(weekday, dt)</code>



Examples

Example 1: Extract Year, Month, Day from hire_date

```
SELECT
    first_name,
    hire_date,
    YEAR(hire_date)    AS hire_year,
    MONTH(hire_date)   AS hire_month,
    DAY(hire_date)     AS hire_day
FROM employees;
```

Output:

first_name	hire_date	hire_year	hire_month	hire_day
Alice	2020-01-15	2020	1	15
Bob	2018-06-01	2018	6	1
Carol	2021-03-20	2021	3	20
David	2017-09-10	2017	9	10
Eva	2015-01-01	2015	1	1

Example 2: Number of Employees Hired Per Year

```
SELECT
    YEAR(hire_date) AS hire_year,
    COUNT(*)        AS employees_hired
FROM employees
GROUP BY YEAR(hire_date)
ORDER BY hire_year;
```

Output:

hire_year	employees_hired
2015	1
2016	1
2017	1
2018	1
2019	1
2020	2
2021	2
2022	1

Example 3: Monthly Order Revenue

```
SELECT
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    COUNT(*)          AS order_count,
    SUM(amount)       AS monthly_revenue
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY order_year, order_month;
```

Output:

order_year	order_month	order_count	monthly_revenue
2024	1	3	255000
2024	2	3	234000
2024	3	2	45000

Example 4: EXTRACT Function (Standard SQL — Works in All Databases)

```
-- PostgreSQL-style EXTRACT (also works in MySQL 8+):
SELECT
    first_name,
    EXTRACT(YEAR FROM hire_date) AS hire_year,
    EXTRACT(MONTH FROM hire_date) AS hire_month,
    EXTRACT(DAY FROM hire_date) AS hire_day
FROM employees;
```

Example 5: Filter by Month or Year

```
-- All employees hired in 2020:
SELECT first_name, hire_date
FROM employees
WHERE YEAR(hire_date) = 2020;

-- All orders placed in January:
SELECT order_id, order_date, amount
FROM orders
WHERE MONTH(order_date) = 1;

-- Orders in Q1 (January, February, March):
SELECT order_id, order_date
FROM orders
WHERE MONTH(order_date) BETWEEN 1 AND 3;
-- OR: WHERE QUARTER(order_date) = 1;
```

Example 6: Day of Week Analysis

```
-- What day of the week do most orders occur?
SELECT
    DAYOFWEEK(order_date) AS day_number,    -- 1=Sunday, 2=Monday, ..., 7=Saturday
    DAYNAME(order_date)   AS day_name,
    COUNT(*)              AS order_count
FROM orders
GROUP BY DAYOFWEEK(order_date), DAYNAME(order_date)
ORDER BY day_number;
```

Example 7: Calculate Age from Birthdate

```
-- If we had a birthdate column:
SELECT
    name,
    birthdate,
    FLOOR(DATEDIFF(CURRENT_DATE, birthdate) / 365.25) AS age_years
FROM people;

-- MySQL's TIMESTAMPDIFF (more accurate):
SELECT TIMESTAMPDIFF(YEAR, birthdate, CURRENT_DATE) AS age FROM people;
```

Example 8: Quarter-based Analysis

```
SELECT
    YEAR(order_date)    AS year,
    QUARTER(order_date) AS quarter,
    SUM(amount)         AS quarterly_revenue
FROM orders
GROUP BY YEAR(order_date), QUARTER(order_date)
ORDER BY year, quarter;
```

Output:

year	quarter	quarterly_revenue
2024	1	534000

TOPIC 3: Date Difference



Concept Explanation

Calculate the **difference between two dates** in days, months, years, or other units.

Function	Description	Database
DATEDIFF(d1, d2)	d1 - d2 in DAYS	MySQL
DATEDIFF(unit, d1, d2)	Difference in specified unit	SQL Server
d1 - d2	Difference in DAYS (as integer)	PostgreSQL
TIMESTAMPDIFF(unit, d1, d2)	Difference in any unit	MySQL
DATE_ADD(d, INTERVAL n unit)	Add to date	MySQL
DATE_SUB(d, INTERVAL n unit)	Subtract from date	MySQL
DATEADD(unit, n, d)	Add to date	SQL Server

Function	Description	Database
AGE(d1, d2)	Returns interval object	PostgreSQL



Syntax

```
-- MySQL: Days difference
DATEDIFF(end_date, start_date)  -- positive if end > start

-- MySQL: Difference in any unit (more flexible)
TIMESTAMPDIFF(YEAR,  start_date, end_date)
TIMESTAMPDIFF(MONTH, start_date, end_date)
TIMESTAMPDIFF(DAY,   start_date, end_date)
TIMESTAMPDIFF(HOUR,  start_datetime, end_datetime)

-- MySQL: Add/subtract dates
DATE_ADD(date, INTERVAL n YEAR)
DATE_ADD(date, INTERVAL n MONTH)
DATE_ADD(date, INTERVAL n DAY)
DATE_SUB(date, INTERVAL n DAY)

-- Shorthand (MySQL):
date + INTERVAL n DAY
date - INTERVAL n DAY
```



Examples

Example 1: Days Since Hiring

```
SELECT
  first_name,
  hire_date,
  DATEDIFF(CURRENT_DATE, hire_date) AS days_at_company,
  ROUND(DATEDIFF(CURRENT_DATE, hire_date) / 365.25, 1) AS years_at_company
FROM employees
ORDER BY days_at_company DESC;
```

Output (approx as of 2024-03-30):

first_name	hire_date	days_at_company	years_at_company
Eva	2015-01-01	3376	9.2
Frank	2016-07-15	2815	7.7
David	2017-09-10	2393	6.5
Bob	2018-06-01	2128	5.8

Example 2: TIMESTAMPDIFF — Years of Service

```
SELECT
  first_name,
  hire_date,
  TIMESTAMPDIFF(YEAR,  hire_date, CURRENT_DATE) AS years,
  TIMESTAMPDIFF(MONTH, hire_date, CURRENT_DATE) AS months_total
FROM employees
ORDER BY years DESC;
```

Example 3: Order Processing Time (How long to deliver)

```
-- If we had a delivery_date column:
SELECT
  order_id,
  order_date,
  delivery_date,
  DATEDIFF(delivery_date, order_date) AS delivery_days
FROM orders_extended;
```

Example 4: DATE_ADD — Calculate Future Date

```
-- What date is 90 days from each order?
SELECT
  order_id,
  order_date,
  DATE_ADD(order_date, INTERVAL 90 DAY) AS expiry_date,
  DATE_ADD(order_date, INTERVAL 3 MONTH) AS warranty_end
FROM orders;
```

Output:

order_id	order_date	expiry_date	warranty_end
101	2024-01-05	2024-04-04	2024-04-05
102	2024-01-10	2024-04-09	2024-04-10

Example 5: DATE_SUB — Filter Recent Orders

```
-- Orders placed in the last 60 days
SELECT order_id, order_date, amount
FROM orders
WHERE order_date >= DATE_SUB(CURRENT_DATE, INTERVAL 60 DAY);

-- Last 3 months
WHERE order_date >= DATE_SUB(CURRENT_DATE, INTERVAL 3 MONTH);

-- Last year
WHERE order_date >= DATE_SUB(CURRENT_DATE, INTERVAL 1 YEAR);
```

Example 6: Date Formatting with DATE_FORMAT (MySQL)

```
SELECT
  order_date,
  DATE_FORMAT(order_date, '%d-%b-%Y') AS formatted_date, -- 05-Jan-2024
  DATE_FORMAT(order_date, '%M %d, %Y') AS long_format,    -- January 05, 2024
  DATE_FORMAT(order_date, '%Y/%m/%d') AS iso_format,      -- 2024/01/05
  DATE_FORMAT(order_date, '%W') AS weekday_name          -- Friday
FROM orders;
```

Output:

order_date	formatted_date	long_format	iso_format	weekday_name
2024-01-05	05-Jan-2024	January 05, 2024	2024/01/05	Friday
2024-01-10	10-Jan-2024	January 10, 2024	2024/01/10	Wednesday

Example 7: TO_CHAR — PostgreSQL Date Formatting

```
-- PostgreSQL:
SELECT
  TO_CHAR(hire_date, 'DD-Mon-YYYY') AS formatted,
  TO_CHAR(hire_date, 'Month YYYY') AS month_year
FROM employees;
```

Example 8: Convert String to Date

```
-- MySQL: STR_TO_DATE
SELECT STR_TO_DATE('30-March-2024', '%d-%M-%Y') AS converted_date;
-- Output: 2024-03-30

-- Cast string to date:
SELECT CAST('2024-03-30' AS DATE) AS date_value;

-- PostgreSQL:
SELECT TO_DATE('30-03-2024', 'DD-MM-YYYY') AS date_value;
```

Example 9: First and Last Day of Month

```
-- First day of current month:
SELECT DATE_FORMAT(CURRENT_DATE, '%Y-%m-01') AS first_of_month;

-- Last day of current month:
SELECT LAST_DAY(CURRENT_DATE) AS last_of_month;
```

TOPIC 4: Other Essential Date Functions

📌 Quick Reference

Function	Description	Example
CURRENT_DATE	Today's date	2024-03-30
NOW()	Current datetime	2024-03-30 18:45:00
YEAR(d) / MONTH(d) / DAY(d)	Extract parts	YEAR('2024-03-30') → 2024
QUARTER(d)	Quarter number (1-4)	QUARTER('2024-03-30') → 1
WEEK(d)	Week number of year (1-53)	WEEK('2024-03-30') → 13
DAYOFWEEK(d)	1=Sun, 2=Mon, ..., 7=Sat	DAYOFWEEK('2024-03-30') → 7
DAYNAME(d)	Name of weekday	DAYNAME('2024-03-30') → 'Saturday'
MONTHNAME(d)	Name of month	MONTHNAME('2024-03-30') → 'March'
DATEDIFF(d1, d2)	Difference in days (d1 - d2)	DATEDIFF('2024-03-30', '2024-01-01') → 89
TIMESTAMPDIFF(unit, d1, d2)	Difference in unit	TIMESTAMPDIFF(MONTH, '2023-01-01', '2024-01-01') → 12
DATE_ADD(d, INTERVAL n unit)	Add to date	DATE_ADD('2024-01-01', INTERVAL 30 DAY) → '2024-01-31'

Function	Description	Example
<code>DATE_SUB(d, INTERVAL n unit)</code>	Subtract from date	<code>DATE_SUB('2024-01-31', INTERVAL 30 DAY)</code> → '2024-01-01'
<code>DATE_FORMAT(d, format)</code>	Format date as string (MySQL)	<code>DATE_FORMAT(hire_date, '%Y-%m')</code> → '2020-01'
<code>LAST_DAY(d)</code>	Last day of month	<code>LAST_DAY('2024-02-01')</code> → '2024-02-29'
<code>STR_TO_DATE(s, format)</code>	Convert string to date (MySQL)	<code>STR_TO_DATE('30/01/2024', '%d/%m/%Y')</code>

Practice Questions

● Easy Level

- Q1.** Write a query to display all orders along with the year, month, and day of the order date as separate columns.
- Q2.** Write a query to show all employees hired in the year `2020`.
- Q3.** Write a query to show all orders placed in `January` (month = 1).
- Q4.** Write a query to calculate how many days each employee has been with the company (from hire_date to today).
- Q5.** Write a query to show the current date, current time (NOW()), and a label 'Today' as a constant.

● Medium Level

- Q6.** Write a query to show the number of employees hired per year, sorted from oldest to newest.
- Q7.** Write a query to show total revenue by month. Display month name (January, February, etc.) and total revenue.
- Q8.** Write a query to find how many years each employee has worked at the company (use TIMESTAMPDIFF or DATEDIFF).
- Q9.** Write a query to find all employees who have been with the company for more than 5 years.
- Q10.** Write a query to show for each order:
- Order date in format: `DD-Mon-YYYY` (like 05-Jan-2024)
 - What date is 30 days after the order (estimated delivery)
 - What date is 1 year after (warranty end)

● Hard / Interview Level

- Q11.** Write a query to show monthly revenue trends: year, month name, revenue, and cumulative revenue up to that month.
- Q12.** Write a query to find employees who were hired in Q1 (January to March) of any year.
- Q13.** A company wants to identify "long-tenured" vs "new" employees:
- ≥ 5 years → 'Veteran'
 - 2 to < 5 years → 'Experienced'
 - < 2 years → 'New'
- Write the query using CASE + TIMESTAMPDIFF.
- Q14.** Write a query to find which day of the week has the most orders. Show day name and count.
- Q15.** Write a query to find customers whose signup date was within the last 2 years.

Solutions

Q1.

```
SELECT
  order_id,
  order_date,
  YEAR(order_date) AS year,
  MONTH(order_date) AS month,
  DAY(order_date) AS day
FROM orders;
```

Q2.

```
SELECT first_name, hire_date
FROM employees
WHERE YEAR(hire_date) = 2020;

-- Output: Alice (2020-01-15), Jake (2020-05-14)
```

Q3.

```
SELECT order_id, order_date, amount
FROM orders
WHERE MONTH(order_date) = 1;

-- Output: orders 101, 102, 103 (all January 2024)
```

Q4.

```
SELECT
    first_name,
    hire_date,
    DATEDIFF(CURRENT_DATE, hire_date) AS days_at_company
FROM employees
ORDER BY days_at_company DESC;
```

Q5.

```
SELECT
    CURRENT_DATE    AS today_date,
    NOW()           AS current_datetime,
    'Today'         AS label;
```

Q6.

```
SELECT
    YEAR(hire_date) AS hire_year,
    COUNT(*)        AS employees_hired
FROM employees
GROUP BY YEAR(hire_date)
ORDER BY hire_year ASC;
```

Q7.

```
SELECT
    MONTHNAME(order_date) AS month_name,
    MONTH(order_date)     AS month_num,
    SUM(amount)           AS total_revenue
FROM orders
GROUP BY MONTH(order_date), MONTHNAME(order_date)
ORDER BY month_num;
```

Output:

month_name	month_num	total_revenue
January	1	255000
February	2	234000
March	3	45000

Q8.

```
SELECT
    first_name,
    hire_date,
    TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) AS years_of_service
FROM employees
ORDER BY years_of_service DESC;
```

Q9.

```
SELECT first_name, hire_date
FROM employees
WHERE TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) > 5;

-- Or: WHERE DATEDIFF(CURRENT_DATE, hire_date) > 5 * 365
```

Output (as of 2024): Eva (9 yrs), Frank (7 yrs), David (6 yrs), Bob (5.8 yrs)

Q10.

```
SELECT
    order_id,
    DATE_FORMAT(order_date, '%d-%b-%Y')           AS formatted_date,
    DATE_ADD(order_date, INTERVAL 30 DAY)          AS estimated_delivery,
    DATE_ADD(order_date, INTERVAL 1 YEAR)          AS warranty_end
FROM orders;
```

Output:

order_id	formatted_date	estimated_delivery	warranty_end
101	05-Jan-2024	2024-02-04	2025-01-05
102	10-Jan-2024	2024-02-09	2025-01-10

Q11.

```
-- Monthly cumulative revenue (requires window functions from Phase 10)
-- Preview:
SELECT
    YEAR(order_date) AS yr,
    MONTHNAME(order_date) AS month,
    SUM(amount) AS monthly_revenue,
    SUM(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS cumulative_revenue
FROM orders
```

```
GROUP BY YEAR(order_date), MONTH(order_date), MONTHNAME(order_date)
ORDER BY yr, MONTH(order_date);
```

Q12.

```
SELECT first_name, hire_date
FROM employees
WHERE MONTH(hire_date) BETWEEN 1 AND 3;
-- OR: WHERE QUARTER(hire_date) = 1

-- Output: Alice (Jan), Eva (Jan), Carol (Mar)
```

Q13.

```
SELECT
  first_name,
  hire_date,
  TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) AS years,
  CASE
    WHEN TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) >= 5 THEN 'Veteran'
    WHEN TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) >= 2 THEN 'Experienced'
    ELSE 'New'
  END AS employee_tenure
FROM employees
ORDER BY years DESC;
```

Q14.

```
SELECT
  DAYNAME(order_date) AS day_name,
  DAYOFWEEK(order_date) AS day_number,
  COUNT(*) AS order_count
FROM orders
GROUP BY DAYOFWEEK(order_date), DAYNAME(order_date)
ORDER BY order_count DESC;
```

Q15.

```
SELECT name, signup_date
FROM customers
WHERE signup_date >= DATE_SUB(CURRENT_DATE, INTERVAL 2 YEAR);

-- PostgreSQL:
WHERE signup_date >= CURRENT_DATE - INTERVAL '2 years'
```

 Real-World Use Cases

Function	Data Science Application
YEAR() / MONTH()	Time-series decomposition — extract seasonality features for ML
DATEDIFF	Customer age, product age, recency features for RFM analysis
TIMESTAMPDIFF	Accurate age computation for user segmentation
DATE_ADD / DATE_SUB	Cohort analysis windows — "users who signed up 30 days ago"
DATE_FORMAT	Data presentation, creating date dimension keys for warehouses
CURRENT_DATE	Real-time dashboards, "days since last purchase" feature
QUARTER()	Quarterly business reporting and trend analysis

RFM Feature Engineering (Recency, Frequency, Monetary) for ML:

```
-- Build RFM features for customer segmentation model:
SELECT
  customer_id,
  DATEDIFF(CURRENT_DATE, MAX(order_date)) AS recency_days,  -- R
  COUNT(order_id) AS frequency,  -- F
  SUM(amount) AS monetary_value  -- M
FROM orders
GROUP BY customer_id;
```

 Phase 10: Advanced SQL - Window Functions

| Window Functions are one of the most tested and powerful SQL features. Master these and you'll stand out in any Data Science interview.

TOPIC 1: Introduction to Window Functions

 Concept Explanation

A **window function** performs a calculation **across a set of rows related to the current row** — without collapsing the rows into groups like GROUP BY does.

The Key Difference:

Feature	GROUP BY + Aggregate	Window Function
Output rows	One row per group	Same number of rows as input
Individual rows	Collapsed into group	Each row kept, with aggregate added
Use case	Summary statistics	Per-row calculations with context

Visual Comparison:

Input Table (employees):

emp_id	dept	salary
1	Sales	85000
2	IT	95000
3	HR	60000
4	Sales	75000
5	IT	110000

After GROUP BY AVG(salary):

dept	avg_salary
Sales	77333
IT	97666
HR	57500

After Window Function (AVG OVER PARTITION BY dept):

emp_id	dept	salary	avg_dept_salary
1	Sales	85000	77333
2	IT	95000	97666
3	HR	60000	57500
4	Sales	75000	77333
5	IT	110000	97666

← still individual rows!

← dept avg assigned to each row

 Syntax

```
function_name() OVER (  
    [PARTITION BY column1, column2] -- Group the window (optional)  
    [ORDER BY column [ASC|DESC]]    -- Order within window (often required)  
    [ROWS/RANGE BETWEEN ...]       -- Frame specification (optional)  
)
```

Window Functions Available:

Category	Functions
Ranking	ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE()
Navigation	LEAD(), LAG(), FIRST_VALUE(), LAST_VALUE()
Aggregation	SUM() OVER, AVG() OVER, COUNT() OVER, MIN/MAX OVER
Distribution	PERCENT_RANK(), CUME_DIST()

TOPIC 2: ROW_NUMBER()

 Concept Explanation

`ROW_NUMBER()` assigns a **unique sequential integer** to each row within a partition, starting at 1. No ties — every row gets a distinct number.

Use Cases:

- Select top-N records per group
- Create unique row IDs
- Pagination
- Remove duplicate rows (keep row 1 per group)

 Syntax

```
ROW_NUMBER() OVER (  
    [PARTITION BY column] -- Optional: restart numbering per group  
    ORDER BY column       -- Required: defines the ordering  
)
```

 Examples

Example 1: Number All Employees by Salary (Globally)

```
SELECT  
    first_name,  
    department,  
    salary,  
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

Output:

first_name	department	salary	salary_rank
Eva	Management	130000	1
Frank	IT	110000	2
Bob	IT	95000	3
Henry	IT	88000	4
Alice	Sales	85000	5

first_name	department	salary	salary_rank
Jake	Finance	82000	6
David	Sales	75000	7
Isla	Sales	72000	8
Carol	HR	60000	9
Grace	HR	55000	10

Example 2: Number Per Department (Partitioned)

```
SELECT
    first_name,
    department,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dept_salary_rank
FROM employees;
```

Output:

first_name	department	salary	dept_salary_rank
Jake	Finance	82000	1
Carol	HR	60000	1
Grace	HR	55000	2
Frank	IT	110000	1
Bob	IT	95000	2
Henry	IT	88000	3
Eva	Management	130000	1
Alice	Sales	85000	1
David	Sales	75000	2
Isla	Sales	72000	3

Each department restarts numbering from 1.

Example 3: Top 1 Employee per Department (IMPORTANT Pattern!)

```
-- Get the highest-paid employee in each department:
SELECT first_name, department, salary
FROM (
    SELECT
        first_name,
        department,
        salary,
        ROW_NUMBER() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS rn
    FROM employees
) AS ranked
WHERE rn = 1;
```

Output:

first_name	department	salary
Jake	Finance	82000
Carol	HR	60000
Frank	IT	110000
Eva	Management	130000
Alice	Sales	85000

Example 4: Remove Duplicate Rows Using ROW_NUMBER

```
-- Delete duplicate rows, keep only the first occurrence:
DELETE FROM employees
WHERE emp_id NOT IN (
    SELECT emp_id FROM (
        SELECT emp_id,
            ROW_NUMBER() OVER (
                PARTITION BY first_name, last_name, department
                ORDER BY emp_id
            ) AS rn
        FROM employees
    ) AS deduped
    WHERE rn = 1
);
```

TOPIC 3: RANK()

Concept Explanation

`RANK()` assigns a rank to each row. **Ties get the SAME rank**, and the **next rank is SKIPPED** (gaps created).

Example: Two employees tie for rank 2 → both get rank 2, next employee gets rank 4 (3 is skipped).

Examples

Example 1: Rank Employees by Salary (With Ties)

```
SELECT
  first_name,
  department,
  salary,
  RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

In our dataset, no two employees have the same salary, so `RANK()` and `ROW_NUMBER()` give same results.

Let's add a tie scenario:

```
-- Scenario: Bob and Henry both earn 95000
SELECT
  first_name,
  salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
  RANK() OVER (ORDER BY salary DESC) AS rank_val,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_val
FROM employees;
```

Hypothetical Output with Ties (Bob and Henry both = 95000):

first_name	salary	row_num	rank_val	dense_rank_val
Eva	130000	1	1	1
Frank	110000	2	2	2
Bob	95000	3	3	3
Henry	95000	4	3	3
Alice	88000	5	5	4

Example 2: Rank by Department Salary

```
SELECT
  first_name,
  department,
  salary,
  RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS dept_rank
FROM employees;
```

TOPIC 4: DENSE_RANK()

Concept Explanation

`DENSE_RANK()` is like `RANK()` but **NO GAPS** in the ranking sequence. Tied rows share the same rank, but the next rank is the very next integer.

Comparison Table (ROW_NUMBER vs RANK vs DENSE_RANK):

```
Salary:    95000, 95000, 88000, 82000

ROW_NUMBER:  1,      2,      3,      4  (always unique, no ties)
RANK:         1,      1,      3,      4  (tie → same rank; gap after tie)
DENSE_RANK:   1,      1,      2,      3  (tie → same rank; NO gap)
```

Example 1: DENSE_RANK per Department

```
SELECT
  first_name,
  department,
  salary,
  DENSE_RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS dense_rank_in_dept
FROM employees;
```

Example 2: Find Second Highest Salary Using DENSE_RANK (Better than LIMIT/OFFSET!)

```
SELECT first_name, department, salary
FROM (
```



```
SELECT
    first_name,
    department,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dr
FROM employees
) AS ranked
WHERE dr = 2;
```

Output:

first_name	department	salary
Frank	IT	110000

This correctly handles ties! If two people share the highest salary, the "second highest" is the next distinct value.

Example 3: Nth Highest Salary per Department (General Pattern)

```
-- 2nd highest in EACH department:
SELECT first_name, department, salary
FROM (
    SELECT
        first_name,
        department,
        salary,
        DENSE_RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS dr
    FROM employees
) AS ranked
WHERE dr = 2;
```

Output:

first_name	department	salary
Grace	HR	55000
Bob	IT	95000
David	Sales	75000

(Single-employee departments like Finance, Management have no "2nd highest")

TOPIC 5: LEAD()

Concept Explanation

LEAD() accesses the value from a **SUBSEQUENT (next) row** within the same window partition, without using a self-join.

Use Case: Compare current row to the next row (e.g., month-over-month revenue change).

Syntax

```
LEAD(column, offset, default_value) OVER (
    [PARTITION BY col]
    ORDER BY col
)
-- offset: how many rows ahead (default = 1)
-- default_value: returned if there's no next row (default = NULL)
```

Examples

Example 1: Next Employee's Salary (Ordered by Salary)

```
SELECT
    first_name,
    salary,
    LEAD(salary, 1, 0) OVER (ORDER BY salary DESC) AS next_lower_salary
FROM employees;
```

Output:

first_name	salary	next_lower_salary
Eva	130000	110000
Frank	110000	95000
Bob	95000	88000
Henry	88000	85000
Alice	85000	82000
Jake	82000	75000
David	75000	72000
Isla	72000	60000
Carol	60000	55000

first_name	salary	next_lower_salary
Grace	55000	0

Example 2: Month-over-Month Revenue (Most Common Use Case!)

```
SELECT
    YEAR(order_date)      AS yr,
    MONTH(order_date)     AS mth,
    SUM(amount)           AS monthly_revenue,
    LEAD(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS next_month_revenue
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY yr, mth;
```

Output:

yr	mth	monthly_revenue	next_month_revenue
2024	1	255000	234000
2024	2	234000	45000
2024	3	45000	NULL

Example 3: Salary Gap to Next Employee

```
SELECT
    first_name,
    salary,
    LEAD(salary) OVER (ORDER BY salary DESC) AS next_salary,
    salary - LEAD(salary) OVER (ORDER BY salary DESC) AS gap_to_next
FROM employees;
```

Output:

first_name	salary	next_salary	gap_to_next
Eva	130000	110000	20000
Frank	110000	95000	15000
Bob	95000	88000	7000
Henry	88000	85000	3000

TOPIC 6: LAG()

📘 Concept Explanation

LAG() accesses the value from a **PREVIOUS (prior) row** within the window partition.

Use Case: Compare current row to the previous row (e.g., month-over-month growth rate).

📋 Syntax

```
LAG(column, offset, default_value) OVER (
    [PARTITION BY col]
    ORDER BY col
)
```

🔍 Examples

Example 1: Previous Month's Revenue + Growth

```
SELECT
    YEAR(order_date)      AS yr,
    MONTH(order_date)     AS mth,
    SUM(amount)           AS monthly_revenue,
    LAG(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS prev_month_revenue,
    SUM(amount) - LAG(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS revenue_change
FROM orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY yr, mth;
```

Output:

yr	mth	monthly_revenue	prev_month_revenue	revenue_change
2024	1	255000	NULL	NULL
2024	2	234000	255000	-21000
2024	3	45000	234000	-189000

Example 2: LAG with PARTITION BY — Each Employee Compared to Previous Hire

```
SELECT
    first_name,
    hire_date,
    salary,
```



```
LAG(salary) OVER (ORDER BY hire_date) AS prev_hired_salary,
salary - LAG(salary) OVER (ORDER BY hire_date) AS salary_diff
FROM employees
ORDER BY hire_date;
```

Example 3: Day-over-Day Order Counts

```
SELECT
    order_date,
    COUNT(*) AS orders_today,
    LAG(COUNT(*)) OVER (ORDER BY order_date) AS orders_yesterday
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

Example 4: Month-over-Month Growth Percentage

```
SELECT
    yr,
    mth,
    monthly_revenue,
    prev_month_revenue,
    ROUND((monthly_revenue - prev_month_revenue) * 100.0 / prev_month_revenue, 2) AS growth_pct
FROM (
    SELECT
        YEAR(order_date) AS yr,
        MONTH(order_date) AS mth,
        SUM(amount) AS monthly_revenue,
        LAG(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS prev_month_revenue
    FROM orders
    GROUP BY YEAR(order_date), MONTH(order_date)
) AS monthly
ORDER BY yr, mth;
```

TOPIC 7: PARTITION BY



Concept Explanation

PARTITION BY in a window function is like **GROUP BY**, but it **doesn't collapse rows**. It divides the result set into partitions (subsets), and the window function runs independently within each partition.

WITHOUT PARTITION BY: Window function applies to ALL rows.

WITH PARTITION BY: Window function restarts/resets for each partition.



Examples

Example 1: Running Total of Salary vs Partition by Department

```
-- Global Running Total (no partition):
SELECT
    first_name,
    department,
    salary,
    SUM(salary) OVER (ORDER BY salary DESC) AS global_running_total
FROM employees;

-- Per-Department Running Total (partitioned):
SELECT
    first_name,
    department,
    salary,
    SUM(salary) OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dept_running_total
FROM employees;
```

Per-Department Running Total Output:

first_name	department	salary	dept_running_total
Jake	Finance	82000	82000
Carol	HR	60000	60000
Grace	HR	55000	115000
Frank	IT	110000	110000
Bob	IT	95000	205000
Henry	IT	88000	293000
Eva	Management	130000	130000
Alice	Sales	85000	85000
David	Sales	75000	160000

first_name	department	salary	dept_running_total
Isla	Sales	72000	232000

Example 2: Percentage of Department Total Salary

```
SELECT
    first_name,
    department,
    salary,
    SUM(salary) OVER (PARTITION BY department) AS dept_total,
    ROUND(salary * 100.0 / SUM(salary) OVER (PARTITION BY department), 2) AS pct_of_dept
FROM employees
ORDER BY department, salary DESC;
```

Output:

first_name	department	salary	dept_total	pct_of_dept
Jake	Finance	82000	82000	100.00
Carol	HR	60000	115000	52.17
Grace	HR	55000	115000	47.83
Frank	IT	110000	293000	37.54
Bob	IT	95000	293000	32.42
Henry	IT	88000	293000	30.03
Eva	Management	130000	130000	100.00
Alice	Sales	85000	232000	36.64
David	Sales	75000	232000	32.33
Isla	Sales	72000	232000	31.03

Example 3: Partition by Multiple Columns

```
SELECT
    first_name,
    department,
    city,
    salary,
    AVG(salary) OVER (PARTITION BY department, city) AS avg_dept_city_salary
FROM employees;
```

TOPIC 8: Other Window Functions

Aggregate Functions with OVER()

Any aggregate function (SUM, AVG, MIN, MAX, COUNT) can be used as a window function by adding `OVER()`.

Examples

Example 1: SUM OVER — Running Total (Cumulative Sum)

```
SELECT
    order_id,
    order_date,
    amount,
    SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders
ORDER BY order_date;
```

Output:

order_id	order_date	amount	running_total
101	2024-01-05	150000	150000
102	2024-01-10	30000	180000
103	2024-01-15	75000	255000
104	2024-02-01	9000	264000
105	2024-02-10	75000	339000
106	2024-02-15	150000	489000
107	2024-03-01	30000	519000
108	2024-03-10	15000	534000

Example 2: AVG OVER — Moving Average

```
SELECT
    order_id,
    order_date,
    amount,
    AVG(amount) OVER (
        ORDER BY order_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_3
```

```
FROM orders
ORDER BY order_date;
```

3-row moving average: uses current row + 2 preceding rows.

Example 3: COUNT OVER — Count Per Partition

```
SELECT
    first_name,
    department,
    COUNT(*) OVER (PARTITION BY department) AS dept_headcount
FROM employees;
```

Output:

first_name	department	dept_headcount
Alice	Sales	3
David	Sales	3
Isla	Sales	3
Bob	IT	3
Frank	IT	3
Henry	IT	3

Example 4: MIN/MAX OVER — Show Individual + Group Min/Max

```
SELECT
    first_name,
    department,
    salary,
    MIN(salary) OVER (PARTITION BY department) AS dept_min_salary,
    MAX(salary) OVER (PARTITION BY department) AS dept_max_salary
FROM employees;
```

Example 5: NTILE() — Divide into Equal Buckets

```
-- Divide employees into 4 quartiles by salary (bottom 25%, next 25%, etc.)
SELECT
    first_name,
    salary,
    NTILE(4) OVER (ORDER BY salary ASC) AS salary_quartile
FROM employees;
```

Output:

first_name	salary	salary_quartile
Grace	55000	1
Carol	60000	1
Isla	72000	2
David	75000	2
Jake	82000	2
Alice	85000	3
Henry	88000	3
Bob	95000	4
Frank	110000	4
Eva	130000	4

Example 6: FIRST_VALUE() and LAST_VALUE()

```
SELECT
    first_name,
    department,
    salary,
    FIRST_VALUE(first_name) OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS highest_earner_in_dept
FROM employees;
```

Practice Questions

● Easy Level

- Q1.** Write a query to rank all employees by salary (highest first) using ROW_NUMBER().
- Q2.** Write a query to show each employee's salary along with the running total of salary (ordered by emp_id).
- Q3.** Write a query to add a column showing how many employees are in each department (using COUNT OVER PARTITION BY).
- Q4.** Write a query using RANK() to rank employees by salary, highest first.
- Q5.** Write a query to show each order's amount, the previous order's amount (LAG), and the next order's amount (LEAD), ordered by order_date.

● **Medium Level**

- Q6.** Write a query to find the top-paid employee in EACH department using ROW_NUMBER().
- Q7.** Write a query to calculate the month-over-month revenue change using LAG().
- Q8.** Write a query to show each employee's salary as a percentage of their department's total salary.
- Q9.** Write a query to find all employees whose salary rank within their department is 2nd (second highest in dept).
- Q10.** Write a query to assign employees to salary quartiles using NTILE(4).

● **Hard / Interview Level**

- Q11.** Write a query to find the 2nd highest salary in the ENTIRE company using DENSE_RANK() (handle ties correctly).
- Q12.** Write a query to compute a 3-month rolling average of order revenue.
- Q13.** Write a query to show for each employee:
- Their salary
 - Department average salary
 - Company average salary
 - Difference from department average
 - Difference from company average
- Q14.** Write a query to generate a ranking of employees within each city by salary. Show only employees ranked 1 or 2 in their city.
- Q15.** Write a query to find all months where revenue DECREASED compared to the previous month.

Solutions

Q1.

```
SELECT
    first_name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

Q2.

```
SELECT
    emp_id,
    first_name,
    salary,
    SUM(salary) OVER (ORDER BY emp_id) AS running_total
FROM employees;
```

Q3.

```
SELECT
    first_name,
    department,
    COUNT(*) OVER (PARTITION BY department) AS dept_employee_count
FROM employees;
```

Q4.

```
SELECT
    first_name,
    salary,
    RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Q5.

```
SELECT
    order_id,
    order_date,
    amount,
    LAG(amount) OVER (ORDER BY order_date) AS prev_amount,
    LEAD(amount) OVER (ORDER BY order_date) AS next_amount
FROM orders
ORDER BY order_date;
```

Q6.

```
SELECT first_name, department, salary
FROM (
    SELECT
        first_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
    FROM employees
) ranked
WHERE rn = 1;
```

Q7.

```
SELECT
  yr,
  mth,
  monthly_revenue,
  LAG(monthly_revenue) OVER (ORDER BY yr, mth) AS prev_month,
  monthly_revenue - LAG(monthly_revenue) OVER (ORDER BY yr, mth) AS revenue_change
FROM (
  SELECT
    YEAR(order_date) AS yr,
    MONTH(order_date) AS mth,
    SUM(amount) AS monthly_revenue
  FROM orders
  GROUP BY YEAR(order_date), MONTH(order_date)
) monthly_data
ORDER BY yr, mth;
```

Q8.

```
SELECT
  first_name,
  department,
  salary,
  SUM(salary) OVER (PARTITION BY department) AS dept_total,
  ROUND(salary * 100.0 / SUM(salary) OVER (PARTITION BY department), 2) AS pct_of_dept
FROM employees
ORDER BY department, salary DESC;
```

Q9.

```
SELECT first_name, department, salary
FROM (
  SELECT
    first_name,
    department,
    salary,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
  FROM employees
) ranked
WHERE dr = 2;
```

Q10.

```
SELECT
  first_name,
  salary,
  NTILE(4) OVER (ORDER BY salary ASC) AS salary_quartile,
  CASE NTILE(4) OVER (ORDER BY salary ASC)
    WHEN 1 THEN 'Bottom 25%'
    WHEN 2 THEN 'Lower Mid 25%'
    WHEN 3 THEN 'Upper Mid 25%'
    WHEN 4 THEN 'Top 25%'
  END AS quartile_label
FROM employees
ORDER BY salary;
```

Q11.

```
SELECT first_name, department, salary
FROM (
  SELECT
    first_name,
    department,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dr
  FROM employees
) ranked
WHERE dr = 2;

-- Output: Frank, IT, 110000
```

Q12.

```
SELECT
  order_date,
  amount,
  AVG(amount) OVER (
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS rolling_3_avg
```

```
FROM orders
ORDER BY order_date;
```

Q13.

```
SELECT
    first_name,
    department,
    salary,
    AVG(salary) OVER (PARTITION BY department) AS dept_avg,
    AVG(salary) OVER () AS company_avg,
    salary - AVG(salary) OVER (PARTITION BY department) AS diff_from_dept,
    salary - AVG(salary) OVER () AS diff_from_company
FROM employees
ORDER BY department, salary DESC;
```

Q14.

```
SELECT first_name, city, salary
FROM (
    SELECT
        first_name,
        city,
        salary,
        ROW_NUMBER() OVER (PARTITION BY city ORDER BY salary DESC) AS city_rank
    FROM employees
) ranked
WHERE city_rank <= 2
ORDER BY city, city_rank;
```

Q15.

```
SELECT yr, mth, monthly_revenue, prev_month, revenue_change
FROM (
    SELECT
        YEAR(order_date) AS yr,
        MONTH(order_date) AS mth,
        SUM(amount) AS monthly_revenue,
        LAG(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS prev_month,
        SUM(amount) - LAG(SUM(amount)) OVER (ORDER BY YEAR(order_date), MONTH(order_date)) AS revenue_change
    FROM orders
    GROUP BY YEAR(order_date), MONTH(order_date)
) revenue_data
WHERE revenue_change < 0;

-- Output: February and March (both decreased vs prior month)
```

 Real-World Use Cases

Function	Data Science Application
ROW_NUMBER	Top-N per segment, deduplication, pagination of results
RANK	Competitive ranking, leaderboards, customer tier assignment
DENSE_RANK	Nth-highest problems, percentile tier without gaps
LEAD	Next event prediction, churn prediction (time to next purchase)
LAG	Month-over-month/year-over-year comparisons, trend detection
PARTITION BY	Segment-specific metrics without collapsing rows
Running SUM	Cumulative revenue, cumulative user signups over time
Moving AVG	Smoothing time-series data for trend detection in ML features
NTILE	Create quartile/decile features for segmentation models

 Phase 11: CTEs — Common Table Expressions

TOPIC 1: Common Table Expressions (CTE)

 Concept Explanation

A **CTE (Common Table Expression)** is a **named temporary result set** that you define at the beginning of a query and can reference throughout the query. It exists only for the duration of the query.

Think of it as giving a name to a subquery so you can:

1. **Reuse it** multiple times in the same query
2. **Break complex queries** into readable steps
3. **Improve readability** dramatically vs nested subqueries

CTE vs Subquery vs Temp Table:

Feature	CTE	Subquery	Temp Table
Readability	✔ Very clean	✗ Can get messy	✔ Clean
Reusability	Within same query only	Cannot reuse	Multiple queries
Performance	Same as subquery usually	Same as CTE usually	Better for heavy reuse
Recursion	✔ Supported	✗ Not supported	✗ Not (directly)
Scope	Single query	Single query	Session or transaction



Syntax

```
-- Single CTE:
WITH cte_name AS (
    SELECT ...
    FROM ...
    WHERE ...
)
SELECT *
FROM cte_name;

-- Multiple CTEs:
WITH
    cte1 AS (
        SELECT ...
    ),
    cte2 AS (
        SELECT ... FROM cte1  -- Can reference earlier CTEs!
    )
SELECT *
FROM cte1
JOIN cte2 ON ...;
```



Examples

Example 1: Simple CTE — High-Salary Employees

```
WITH high_salary_employees AS (
    SELECT first_name, department, salary
    FROM employees
    WHERE salary > 85000
)
SELECT *
FROM high_salary_employees
ORDER BY salary DESC;
```

Output:

first_name	department	salary
Eva	Management	130000
Frank	IT	110000
Bob	IT	95000
Henry	IT	88000

The CTE `high_salary_employees` acts like a temporary table — used in the final `SELECT`.

Example 2: CTE to Find Department Averages

```
WITH dept_avg AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT
    e.first_name,
    e.department,
    e.salary,
    ROUND(d.avg_salary, 2) AS dept_avg_salary,
    e.salary - d.avg_salary AS diff_from_avg
FROM employees AS e
JOIN dept_avg AS d ON e.department = d.department
ORDER BY e.department, diff_from_avg DESC;
```

Output:

first_name	department	salary	dept_avg_salary	diff_from_avg
Jake	Finance	82000	82000.00	0
Carol	HR	60000	57500.00	2500
Grace	HR	55000	57500.00	-2500
Frank	IT	110000	97666.67	12333.33
Bob	IT	95000	97666.67	-2666.67

first_name	department	salary	dept_avg_salary	diff_from_avg
Henry	IT	88000	97666.67	-9666.67
Eva	Management	130000	130000.00	0

This replaces a correlated subquery with a much cleaner CTE approach!

Example 3: CTE to Get Top 1 Employee per Department

```
WITH ranked_employees AS (
    SELECT
        first_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
    FROM employees
)
SELECT first_name, department, salary
FROM ranked_employees
WHERE rn = 1
ORDER BY salary DESC;
```

Output:

first_name	department	salary
Eva	Management	130000
Frank	IT	110000
Alice	Sales	85000
Jake	Finance	82000
Carol	HR	60000

CTE makes it easy to filter on the window function result (which you cannot do directly in WHERE).

Example 4: Multiple CTEs

```
WITH
-- CTE 1: Employees above average salary
above_avg AS (
    SELECT first_name, department, salary
    FROM employees
    WHERE salary > (SELECT AVG(salary) FROM employees)
),
-- CTE 2: Department count for above-avg employees
dept_count AS (
    SELECT department, COUNT(*) AS above_avg_count
    FROM above_avg
    GROUP BY department
)
SELECT *
FROM dept_count
ORDER BY above_avg_count DESC;
```

Output:

department	above_avg_count
IT	2
Finance	0

(Bob 95000 and Frank 110000 are above avg. Eva 130000 too though — Management)

Corrected output:

department	above_avg_count
IT	2
Management	1
Sales	0

Example 5: CTE + Aggregation — Monthly Revenue with Running Total

```
WITH monthly_revenue AS (
    SELECT
        YEAR(order_date) AS yr,
        MONTH(order_date) AS mth,
        MONTHNAME(order_date) AS month_name,
        SUM(amount) AS revenue
    FROM orders
    GROUP BY YEAR(order_date), MONTH(order_date), MONTHNAME(order_date)
)
SELECT
    yr,
    mth,
    month_name,
    revenue,
    SUM(revenue) OVER (ORDER BY yr, mth) AS cumulative_revenue
```



```
FROM monthly_revenue
ORDER BY yr, mth;
```

Output:

yr	mth	month_name	revenue	cumulative_revenue
2024	1	January	255000	255000
2024	2	February	234000	489000
2024	3	March	45000	534000

Example 6: CTE for Customer Analysis

```
WITH customer_orders AS (
  SELECT
    c.customer_id,
    c.name           AS customer_name,
    c.country,
    COUNT(o.order_id) AS total_orders,
    SUM(o.amount)     AS total_spent,
    AVG(o.amount)     AS avg_order_value
  FROM customers AS c
  LEFT JOIN orders AS o ON c.customer_id = o.customer_id
  GROUP BY c.customer_id, c.name, c.country
),
customer_ranked AS (
  SELECT *,
    RANK() OVER (ORDER BY total_spent DESC) AS spending_rank
  FROM customer_orders
)
SELECT customer_name, country, total_orders, total_spent, spending_rank
FROM customer_ranked
WHERE spending_rank <= 3;
```

Output (Top 3 spenders):

customer_name	country	total_orders	total_spent	spending_rank
Ravi Desai	India	2	225000	1
Chen Wei	China	1	150000	2
Sara Ali	UK	1	75000	3

Example 7: CTE Replacing Nested Subqueries (Readability)

```
-- Compare: Nested Subquery (hard to read):
SELECT first_name, salary FROM employees
WHERE salray > (
  SELECT AVG(salary) FROM employees
  WHERE department IN (
    SELECT department FROM employees
    WHERE city = 'Delhi'
  )
);

-- Same logic as CTEs (easy to read!):
WITH delhi_departments AS (
  SELECT DISTINCT department FROM employees WHERE city = 'Delhi'
),
delhi_dept_avg AS (
  SELECT AVG(salary) AS avg_sal
  FROM employees
  WHERE department IN (SELECT department FROM delhi_departments)
)
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT avg_sal FROM delhi_dept_avg);
```

TOPIC 2: Recursive CTE

Concept Explanation

A **Recursive CTE** references itself to process hierarchical or sequential data.

When to use:

- Organizational hierarchy (employee → manager → their manager → CEO)
- Category trees (parent → children → grandchildren)
- Date series generation (Jan 1, Jan 2, Jan 3, ...)
- Graph traversal (social networks, routes)

Structure of a Recursive CTE:

```
WITH RECURSIVE cte_name AS (
  -- 1. ANCHOR MEMBER: Base case (starting point)
```

```
SELECT ...
FROM ...
WHERE ...

UNION ALL  -- Connects anchor to recursive part

-- 2. RECURSIVE MEMBER: Loops until condition fails
SELECT ...
FROM table
JOIN cte_name ON ...  -- References itself!
WHERE termination_condition
)
SELECT * FROM cte_name;
```

🔍 Examples

Example 1: Employee Hierarchy (Top-Down from CEO)

```
WITH RECURSIVE employee_hierarchy AS (
  -- Anchor: Top-level employees (no manager)
  SELECT
    emp_id,
    first_name,
    manager_id,
    0 AS level,
    first_name AS path
  FROM employees
  WHERE manager_id IS NULL  -- Start from CEO/top-level

  UNION ALL

  -- Recursive: Each employee's subordinate
  SELECT
    e.emp_id,
    e.first_name,
    e.manager_id,
    h.level + 1,
    CONCAT(h.path, ' → ', e.first_name)
  FROM employees AS e
  JOIN employee_hierarchy AS h ON e.manager_id = h.emp_id
)
SELECT
  first_name,
  level,
  path AS reporting_chain
FROM employee_hierarchy
ORDER BY level, first_name;
```

Output:

first_name	level	reporting_chain
Eva	0	Eva
Frank	0	Frank
Jake	0	Jake
Alice	1	Eva → Alice
Carol	1	Eva → Carol
David	1	Eva → David
Grace	1	Eva → Grace
Isla	1	Eva → Isla
Bob	1	Frank → Bob
Henry	1	Frank → Henry

Example 2: Generate a Date Series (Very Useful!)

```
WITH RECURSIVE date_series AS (
  -- Anchor: Start date
  SELECT CAST('2024-01-01' AS DATE) AS dt

  UNION ALL

  -- Recursive: Add 1 day each iteration
  SELECT DATE_ADD(dt, INTERVAL 1 DAY)
  FROM date_series
  WHERE dt < '2024-01-31'  -- Stop condition
)
SELECT dt AS date_in_january
FROM date_series;
```

Output:

date_in_january

2024-01-01
2024-01-02
2024-01-03
...
2024-01-31

This is powerful for filling in missing dates in time-series data!

Example 3: Generate Number Series

```
WITH RECURSIVE numbers AS (  
  SELECT 1 AS n  
  UNION ALL  
  SELECT n + 1 FROM numbers WHERE n < 10  
)  
SELECT n FROM numbers;
```

Output: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Example 4: Cumulative Factorial (Mathematical Recursion)

```
WITH RECURSIVE factorial AS (  
  SELECT 1 AS n, 1 AS fact_value  
  UNION ALL  
  SELECT n + 1, fact_value * (n + 1)  
  FROM factorial  
  WHERE n < 10  
)  
SELECT n, fact_value AS factorial FROM factorial;
```

Output:

n	factorial
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800

Example 5: Data Science Use — Fill Missing Dates

```
-- Generate all dates in Jan 2024  
WITH RECURSIVE all_dates AS (  
  SELECT CAST('2024-01-01' AS DATE) AS dt  
  UNION ALL  
  SELECT DATE_ADD(dt, INTERVAL 1 DAY)  
  FROM all_dates  
  WHERE dt < '2024-01-31'  
)  
-- Get daily order revenue (missing days will not appear)  
daily_revenue AS (  
  SELECT order_date, SUM(amount) AS revenue  
  FROM orders  
  WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31'  
  GROUP BY order_date  
)  
-- Left join dates with revenue → missing days show 0  
SELECT  
  d.dt AS date,  
  COALESCE(r.revenue, 0) AS daily_revenue  
FROM all_dates AS d  
LEFT JOIN daily_revenue AS r ON d.dt = r.order_date  
ORDER BY d.dt;
```

This is how you create a "complete" time-series with no gaps — critical for ARIMA, LSTM, or any time-series model!

Practice Questions

● Easy Level

- Q1.** Write a CTE called `it_employees` to select all IT department employees. Then SELECT all columns from it.
- Q2.** Write a CTE that computes the average salary per department. Then select departments where this average is above 80000.
- Q3.** Write a CTE to find all customers who have placed at least 1 order. Then show their names and order counts.

● Medium Level

Q4. Write a CTE to rank employees by salary within each department using ROW_NUMBER(). Then show only the top 2 in each department.

Q5. Write two CTEs:

- `total_revenue_cte`: Total revenue per customer (from orders)
- `customer_info_cte`: Customer name and country (from customers)

Then JOIN them and show the top 5 customers by revenue.

Q6. Write a CTE to calculate monthly revenue. Then use a window function in the main query to show:

- Month
- Monthly revenue
- Running cumulative revenue

Q7. Write a CTE to find employees who earn MORE than the average salary of their own department. Then show their names, dept, salary, and dept average.

● Hard / Interview Level

Q8. Write a Recursive CTE to generate all dates in February 2024 (1st to 29th — 2024 is a leap year).

Q9. Write a Recursive CTE to traverse the employee hierarchy starting from `Eva` (`emp_id = 5`) and show all employees who ultimately report to her (directly or indirectly).

Q10. Write a CTE-based query to:

1. Calculate monthly revenue
2. Calculate month-over-month growth %
3. Classify each month: 'Growth' if positive, 'Decline' if negative, 'No Change' if zero

Solutions

Q1.

```
WITH it_employees AS (  
    SELECT *  
    FROM employees  
    WHERE department = 'IT'  
)  
SELECT *  
FROM it_employees  
ORDER BY salary DESC;
```

Q2.

```
WITH dept_avg AS (  
    SELECT department, AVG(salary) AS avg_salary  
    FROM employees  
    GROUP BY department  
)  
SELECT department, ROUND(avg_salary, 2) AS avg_salary  
FROM dept_avg  
WHERE avg_salary > 80000  
ORDER BY avg_salary DESC;
```

Output:

department	avg_salary
Management	130000.00
IT	97666.67
Finance	82000.00

Q3.

```
WITH ordered_customers AS (  
    SELECT customer_id, COUNT(order_id) AS order_count  
    FROM orders  
    GROUP BY customer_id  
    HAVING COUNT(order_id) >= 1  
)  
SELECT c.name, c.country, oc.order_count  
FROM customers AS c  
JOIN ordered_customers AS oc ON c.customer_id = oc.customer_id  
ORDER BY oc.order_count DESC;
```

Q4.

```
WITH ranked_ems AS (  
    SELECT  
        first_name,  
        department,  
        salary,  
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn  
    FROM employees  
)  
SELECT first_name, department, salary
```

```
FROM ranked_emps
WHERE rn <= 2
ORDER BY department, salary DESC;
```

Q5.

```
WITH
total_revenue_cte AS (
  SELECT customer_id, SUM(amount) AS total_revenue
  FROM orders
  GROUP BY customer_id
),
customer_info_cte AS (
  SELECT customer_id, name, country
  FROM customers
)
SELECT
  ci.name,
  ci.country,
  tr.total_revenue
FROM customer_info_cte AS ci
JOIN total_revenue_cte AS tr ON ci.customer_id = tr.customer_id
ORDER BY tr.total_revenue DESC
LIMIT 5;
```

Q6.

```
WITH monthly_rev AS (
  SELECT
    YEAR(order_date) AS yr,
    MONTH(order_date) AS mth,
    MONTHNAME(order_date) AS month_name,
    SUM(amount) AS revenue
  FROM orders
  GROUP BY YEAR(order_date), MONTH(order_date), MONTHNAME(order_date)
)
SELECT
  yr, mth, month_name, revenue,
  SUM(revenue) OVER (ORDER BY yr, mth) AS cumulative_revenue
FROM monthly_rev
ORDER BY yr, mth;
```

Q7.

```
WITH dept_avg AS (
  SELECT department, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department
)
SELECT
  e.first_name,
  e.department,
  e.salary,
  ROUND(d.avg_salary, 2) AS dept_avg
FROM employees AS e
JOIN dept_avg AS d ON e.department = d.department
WHERE e.salary > d.avg_salary
ORDER BY e.department, e.salary DESC;
```

Q8.

```
WITH RECURSIVE feb_2024 AS (
  SELECT CAST('2024-02-01' AS DATE) AS dt
  UNION ALL
  SELECT DATE_ADD(dt, INTERVAL 1 DAY)
  FROM feb_2024
  WHERE dt < '2024-02-29'
)
SELECT dt FROM feb_2024;
```

2024 is a leap year — Feb has 29 days.

Q9.

```
WITH RECURSIVE eva_team AS (
  -- Anchor: Start with Eva
  SELECT emp_id, first_name, manager_id, 0 AS level
  FROM employees
  WHERE emp_id = 5

  UNION ALL

  -- Recursive: Find everyone who reports to current level
  SELECT e.emp_id, e.first_name, e.manager_id, eh.level + 1
```

```
FROM employees AS e
JOIN eva_team AS eh ON e.manager_id = eh.emp_id
)
SELECT first_name, level
FROM eva_team
WHERE emp_id != 5    -- Exclude Eva herself
ORDER BY level, first_name;
```

Output:

first_name	level
Alice	1
Carol	1
David	1
Grace	1
Isla	1

All 5 report directly to Eva (level 1). Bob and Henry report to Frank, not Eva.

Q10.

```
WITH monthly_rev AS (
  SELECT
    YEAR(order_date) AS yr,
    MONTH(order_date) AS mth,
    SUM(amount) AS revenue
  FROM orders
  GROUP BY YEAR(order_date), MONTH(order_date)
),
with_growth AS (
  SELECT
    yr, mth, revenue,
    LAG(revenue) OVER (ORDER BY yr, mth) AS prev_revenue
  FROM monthly_rev
)
SELECT
  yr,
  mth,
  revenue,
  prev_revenue,
  ROUND((revenue - prev_revenue) * 100.0 / NULLIF(prev_revenue, 0), 2) AS growth_pct,
  CASE
    WHEN prev_revenue IS NULL      THEN 'First Month'
    WHEN revenue > prev_revenue    THEN 'Growth'
    WHEN revenue < prev_revenue    THEN 'Decline'
    ELSE 'No Change'
  END AS trend
FROM with_growth
ORDER BY yr, mth;
```

Output:

yr	mth	revenue	prev_revenue	growth_pct	trend
2024	1	255000	NULL	NULL	First Month
2024	2	234000	255000	-8.24	Decline
2024	3	45000	234000	-80.77	Decline

Real-World Use Cases

CTE Type	Data Science Application
Simple CTE	Break complex ETL logic into readable, maintainable steps
Multiple CTEs	Build multi-step feature engineering pipelines in SQL
CTE + Window	Top-N per segment, ranking with additional filtering
CTE + JOIN	Denormalize data cleanly for analytical queries
Recursive CTE	Traverse org charts, category trees, generate date spine
Date Spine CTE	Fill missing dates in time-series data before ML preprocessing

Phase 12: Set Operations

Key Rule for ALL Set Operations:

- Both queries must return the **same number of columns**
- Corresponding columns must have **compatible data types**
- Column names in the result come from the **FIRST query**

TOPIC 1: UNION

Concept Explanation

UNION combines the results of two or more SELECT queries into a **single result set**, automatically removing **duplicate rows**.

When to use:

- Merge data from multiple tables with same structure
- Combine results matching different conditions (when OR is complex)
- Data reconciliation across systems

Syntax

```
SELECT column1, column2 FROM table1
UNION
SELECT column1, column2 FROM table2;

-- With ORDER BY (applies to final combined result):
SELECT column1 FROM table1
UNION
SELECT column1 FROM table2
ORDER BY column1;
```

Examples

Example 1: Combine Employee Cities and Customer Cities

```
-- All unique cities where we have employees OR customers:
SELECT city, 'Employee' AS source FROM employees
UNION
SELECT city, 'Customer' AS source FROM customers;
```

Output:

city	source
Mumbai	Employee
Delhi	Employee
Bangalore	Employee
Chennai	Employee
Pune	Employee
New York	Customer
London	Customer
Shanghai	Customer
Madrid	Customer

Example 2: All People Names (Employees + Customers)

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name, 'Employee' AS type
FROM employees
UNION
SELECT name, 'Customer' AS type
FROM customers;
```

Example 3: Employees from IT OR Management (UNION as alternative to OR)

```
-- Using UNION instead of WHERE ... OR:
SELECT first_name, department, salary FROM employees WHERE department = 'IT'
UNION
SELECT first_name, department, salary FROM employees WHERE department = 'Management';
```

Note: For this case, WHERE ... IN (...) or OR is simpler. UNION shines when you need different columns or different tables.

Example 4: UNION Removes Duplicates — Demonstration

```
-- Both queries return Alice (same row):
SELECT first_name FROM employees WHERE emp_id = 1
UNION
SELECT first_name FROM employees WHERE first_name = 'Alice';

-- UNION automatically deduplicates → only one 'Alice' in result
```

Output:

first_name

Alice

Example 5: Combine Two Different Tables with Same Column Count

```
-- Employee contact vs Customer contact list (unified):
SELECT
    CONCAT(first_name, ' ', last_name) AS name,
    city,
```

```
'internal' AS contact_type
FROM employees

UNION

SELECT
    name,
    city,
    'external' AS contact_type
FROM customers
ORDER BY contact_type, name;
```

TOPIC 2: UNION ALL

Concept Explanation

UNION ALL combines result sets **WITHOUT removing duplicates**. It's **faster** than UNION because it skips the deduplication step.

When to use UNION ALL instead of UNION:

- You KNOW there are no duplicates (different tables, different sources)
- You WANT duplicates preserved (e.g., total transaction log)
- Performance is critical (UNION ALL is always faster)

Syntax

```
SELECT column1, column2 FROM table1
UNION ALL
SELECT column1, column2 FROM table2;
```

Examples

Example 1: UNION ALL vs UNION

```
-- UNION ALL keeps duplicates:
SELECT first_name FROM employees WHERE first_name LIKE 'A%'
UNION ALL
SELECT first_name FROM employees WHERE salary > 80000;
```

Output: (Alice appears TWICE — once for each condition she matches)

first_name

Alice

Bob

Alice

Henry

Eva

Frank

Jake

```
-- UNION removes duplicates:
SELECT first_name FROM employees WHERE first_name LIKE 'A%'
UNION
SELECT first_name FROM employees WHERE salary > 80000;
```

Output: Alice appears only ONCE.

Example 2: Append Two Tables (Full Combination with ALL Rows)

```
-- Combine all employees across two separate regional tables:
SELECT emp_id, name, city FROM employees_north
UNION ALL
SELECT emp_id, name, city FROM employees_south;
-- Keeps all rows including any with same emp_id (different regions)
```

Example 3: Consolidating Multiple Period Data

```
-- Consolidate Q1 and Q2 sales (keeps all rows, even same orders):
SELECT order_id, customer_id, amount, 'Q1' AS quarter FROM q1_orders
UNION ALL
SELECT order_id, customer_id, amount, 'Q2' AS quarter FROM q2_orders;
```

Example 4: UNION ALL for Total COUNT (Simulating Full Outer Join)

```
-- FULL OUTER JOIN workaround in MySQL:
SELECT c.name, o.order_id, o.amount
```



```
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id

UNION ALL

SELECT c.name, o.order_id, o.amount
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

Example 5: Performance Note

```
-- If feeding into COUNT or SUM, UNION ALL is almost always better:
SELECT COUNT(*) FROM (
    SELECT name FROM employees
    UNION ALL      -- Don't pay the dedup cost unnecessarily
    SELECT name FROM customers
) combined;

-- vs UNION (slower – deduplicates before counting):
SELECT COUNT(*) FROM (
    SELECT name FROM employees
    UNION
    SELECT name FROM customers
) combined;
```

TOPIC 3: INTERSECT



Concept Explanation

INTERSECT returns only rows that appear in **BOTH** result sets (the common/overlapping rows).

Database Support:

- PostgreSQL: Native **INTERSECT**
- Oracle: Native **INTERSECT**
- SQL Server: Native **INTERSECT**
- MySQL: **Does NOT support INTERSECT** — use INNER JOIN or IN subquery instead



Syntax

```
-- PostgreSQL / SQL Server / Oracle:
SELECT column1 FROM table1
INTERSECT
SELECT column1 FROM table2;

-- MySQL alternative (no native INTERSECT):
SELECT column1 FROM table1
WHERE column1 IN (SELECT column1 FROM table2);

-- Or using INNER JOIN:
SELECT DISTINCT t1.column1
FROM table1 t1
INNER JOIN table2 t2 ON t1.column1 = t2.column1;
```



Examples

Example 1: Cities That Have BOTH Employees AND Customers

```
-- PostgreSQL / SQL Server:
SELECT city FROM employees
INTERSECT
SELECT city FROM customers;

-- MySQL equivalent:
SELECT DISTINCT e.city
FROM employees AS e
WHERE e.city IN (SELECT city FROM customers);
```

Output:

city
Delhi
Mumbai

Mumbai and Delhi appear in both employees and customers tables.

Example 2: Employees Who Are Also Listed as Customers (by email)

```
-- If the same person appears in both tables:
SELECT email FROM employees_contacts
INTERSECT
SELECT email FROM customers;
```

Example 3: MySQL Workaround for INTERSECT

```
-- Find product_ids ordered in BOTH January AND February:
SELECT DISTINCT product_id
FROM orders
WHERE MONTH(order_date) = 1
      AND product_id IN (
      SELECT product_id FROM orders WHERE MONTH(order_date) = 2
      );
```

TOPIC 4: EXCEPT

📘 Concept Explanation

EXCEPT (also called **MINUS** in Oracle) returns rows from the **first query that do NOT appear in the second query**.

Think of it as: Set A minus Set B

Database Support:

- PostgreSQL: **EXCEPT**
- SQL Server: **EXCEPT**
- Oracle: **MINUS**
- MySQL: **Does NOT support EXCEPT** — use NOT IN or LEFT JOIN + IS NULL

📖 Syntax

```
-- PostgreSQL / SQL Server:
SELECT column1 FROM table1
EXCEPT
SELECT column1 FROM table2;

-- Oracle:
SELECT column1 FROM table1
MINUS
SELECT column1 FROM table2;

-- MySQL alternative:
SELECT column1 FROM table1
WHERE column1 NOT IN (SELECT column1 FROM table2);

-- Or using LEFT JOIN:
SELECT DISTINCT t1.column1
FROM table1 t1
LEFT JOIN table2 t2 ON t1.column1 = t2.column1
WHERE t2.column1 IS NULL;
```

🔍 Examples

Example 1: Employee Cities NOT in Customer Cities

```
-- PostgreSQL / SQL Server:
SELECT city FROM employees
EXCEPT
SELECT city FROM customers;

-- MySQL alternative:
SELECT DISTINCT city
FROM employees
WHERE city NOT IN (SELECT city FROM customers);
```

Output:

city
Bangalore
Chennai
Pune

(Mumbai and Delhi appear in both, so they're excluded. Bangalore, Chennai, Pune are only in employees.)

Example 2: Customers Who Have NOT Ordered (Using EXCEPT)

```
-- PostgreSQL / SQL Server:
SELECT customer_id FROM customers
EXCEPT
```

```
SELECT DISTINCT customer_id FROM orders;

-- MySQL alternative (already seen in Phase 5):
SELECT customer_id FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

Output:

customer_id

7

(Carlos Ruiz)

Example 3: Products Never Ordered

```
-- PostgreSQL:
SELECT product_id, name FROM products
EXCEPT
SELECT DISTINCT p.product_id, p.name
FROM products p
WHERE p.product_id IN (SELECT product_id FROM orders);

-- Simpler MySQL approach:
SELECT product_id, name FROM products
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM orders);
```

Output:

product_id	name
5	Monitor

TOPIC 5: Set Operations Comparison

Complete Comparison Table

Operation	Returns	Duplicates	Speed
UNION	Rows from A + rows from B (combined unique)	Removed	Slower
UNION ALL	Rows from A + ALL rows from B (all incl. dupes)	Kept	Faster
INTERSECT	Only rows in BOTH A and B	Removed	Medium
EXCEPT	Rows in A that are NOT in B	Removed	Medium

Visual Summary:

```
Set A = {1, 2, 3, 4}    Set B = {3, 4, 5, 6}

UNION:      {1, 2, 3, 4, 5, 6}    -- All unique from both
UNION ALL:  {1, 2, 3, 4, 3, 4, 5, 6} -- All rows including dupes
INTERSECT:  {3, 4}                -- Only in BOTH
EXCEPT A-B: {1, 2}              -- In A but NOT in B
EXCEPT B-A: {5, 6}              -- In B but NOT in A
```

Practice Questions

Easy Level

- Q1. Write a UNION query to get all unique city names from both `employees` and `customers` tables.
- Q2. Write a UNION ALL query to get all names from `employees` (`first_name`) and `customers` (`name`) tables, keeping all duplicates.
- Q3. Using a subquery (MySQL-style INTERSECT), find all `city` values that appear in BOTH `employees` and `customers` tables.
- Q4. Write a query (MySQL-style EXCEPT) to find cities that appear in `employees` but NOT in `customers` .

Medium Level

Q5. Write a query to get a combined list of employee and customer contact information:

- Who is in the record (`first_name+last_name` for employees, `name` for customers)
- Their city
- A tag: 'Employee' or 'Customer'

Use UNION and sort by city.

Q6. Write a UNION ALL to get all order amounts plus a final "TOTAL" row added manually. (Hint: UNION ALL the orders with a SELECT that has a hardcoded total.)

Q7. Using INTERSECT (or MySQL workaround), find product_ids that were ordered in BOTH January 2024 AND February 2024.

Q8. Using EXCEPT (or MySQL NOT IN), find all employees whose city is NOT a city where any customer lives.

Hard / Interview Level

- Q9. Explain the difference between UNION and JOIN. When would you use one vs the other?
- Q10. Write a UNION-based query to simulate a summary report showing:
- Count of employees with salary above average

- Count of employees at or below average
Label them as 'Above Average' and 'Below Average'.

Q11. Write a UNION-based query to find all people (employees + customers) who are from `India`. Use UNION to avoid duplicates (in case someone is in both tables).

Q12. Using EXCEPT or NOT IN, find customers who have placed orders for Electronics but have NEVER placed an order for Furniture.

Solutions

Q1.

```
SELECT city FROM employees
UNION
SELECT city FROM customers
ORDER BY city;
```

Output: Bangalore, Bangalore (from customer?), Chennai, Delhi, London, Madrid, Mumbai, New York, Pune, Shanghai

(Note: UNION removes duplicates — Mumbai, Delhi appear in both but show only once)

Q2.

```
SELECT first_name AS name FROM employees
UNION ALL
SELECT name FROM customers;
```

Q3.

```
-- MySQL INTERSECT workaround:
SELECT DISTINCT city
FROM employees
WHERE city IN (SELECT city FROM customers)
ORDER BY city;

-- Output: Delhi, Mumbai
```

Q4.

```
-- MySQL EXCEPT workaround:
SELECT DISTINCT city
FROM employees
WHERE city NOT IN (SELECT city FROM customers)
ORDER BY city;

-- Output: Bangalore, Chennai, Pune
```

Q5.

```
SELECT
    CONCAT(first_name, ' ', last_name) AS person_name,
    city,
    'Employee' AS type
FROM employees

UNION

SELECT
    name,
    city,
    'Customer' AS type
FROM customers

ORDER BY city, person_name;
```

Q6.

```
SELECT order_id, amount FROM orders
UNION ALL
SELECT NULL AS order_id, SUM(amount) AS amount FROM orders;
```

Output: All individual orders + a final row with NULL order_id and total sum.

Q7.

```
-- Products ordered in BOTH January AND February 2024:
SELECT DISTINCT product_id
FROM orders
WHERE MONTH(order_date) = 1
    AND product_id IN (
    SELECT product_id FROM orders WHERE MONTH(order_date) = 2
);
```

Checking our data:

- January orders: product_ids = 1, 3, 2 (orders 101, 102, 103)

- February orders: product_ids = 4, 1, 2 (orders 104, 105, 106)
- Common: product_ids 1 (Laptop) and 2 (Smartphone)

Output:

product_id

1

2

Q8.

```
-- Employees in cities where NO customer lives:
SELECT DISTINCT city AS employee_only_city
FROM employees
WHERE city NOT IN (SELECT city FROM customers);

-- Output: Bangalore, Chennai, Pune
```

Q9.

```
UNION vs JOIN:

UNION:
- Combines ROWS from multiple queries VERTICALLY (stacking)
- Queries must have same number of columns with compatible types
- Used to merge data from different tables of similar structure
- Example: Combine employees + contractors into one people list

JOIN:
- Combines COLUMNS from multiple tables HORIZONTALLY (side by side)
- Creates wider rows by associating related records
- Used to combine data from related tables (via foreign keys)
- Example: Combine employee names with their order info

Rule of Thumb:
- Same structure, different sources → UNION
- Different structure, related data → JOIN

UNION Example:
SELECT name FROM employees UNION SELECT name FROM customers

JOIN Example:
SELECT c.name, o.amount FROM customers c JOIN orders o ON c.id = o.customer_id
```

Q10.

```
SELECT 'Above Average' AS category, COUNT(*) AS employee_count
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)

UNION ALL

SELECT 'Below Average or Equal', COUNT(*)
FROM employees
WHERE salary <= (SELECT AVG(salary) FROM employees);
```

Output:

category	employee_count
Above Average	4
Below Average or Equal	6

Q11.

```
SELECT CONCAT(first_name, ' ', last_name) AS name, 'Employee' AS type
FROM employees
WHERE city IN (SELECT city FROM customers WHERE country = 'India')

UNION

SELECT name, 'Customer' AS type
FROM customers
WHERE country = 'India';
```

Q12.

```
-- Customers who ordered Electronics but NOT Furniture:
SELECT DISTINCT o.customer_id
FROM orders AS o
JOIN products AS p ON o.product_id = p.product_id
WHERE p.category = 'Electronics'

AND o.customer_id NOT IN (
```

```
SELECT DISTINCT o2.customer_id
FROM orders AS o2
JOIN products AS p2 ON o2.product_id = p2.product_id
WHERE p2.category = 'Furniture'
);
```

Real-World Use Cases

Operation	Data Science Application
UNION	Merge customer records from multiple regional databases
UNION ALL	Consolidate transaction logs; combine historical + current data
INTERSECT	Find users active on BOTH mobile AND web platforms
EXCEPT	Find users who were active last month but NOT this month (churn!)

Churn Detection Pattern:

```
-- Users active last month but NOT this month = Churned!
SELECT user_id FROM activity WHERE month = 'January'
EXCEPT
SELECT user_id FROM activity WHERE month = 'February';
```

Phase 13: Data Cleaning in SQL

 **"Data Scientists spend 80% of their time cleaning data."**
SQL is one of the most powerful tools for data cleaning at scale — operating directly on the database without moving data to Python first.

TOPIC 1: Removing Duplicates

Concept Explanation

Duplicate records corrupt aggregations, inflate counts, and reduce model quality. Identifying and removing them is one of the most critical data cleaning steps.

Types of Duplicates:

- Exact duplicates** — Every column is identical
- Key duplicates** — Same primary key, different other columns (data entry errors)
- Fuzzy duplicates** — "Alice Kumar" vs "alice kumar" (same person, different format)

Examples

Example 1: Detect Duplicates with GROUP BY + HAVING

```
-- Find duplicate employee names (same first + last name):
SELECT
    first_name,
    last_name,
    COUNT(*) AS occurrences
FROM employees
GROUP BY first_name, last_name
HAVING COUNT(*) > 1;
```

If this returns rows, you have duplicates.

Example 2: Find All Duplicate Rows (Show Full Row)

```
-- Show all rows that have duplicates:
SELECT *
FROM employees
WHERE (first_name, last_name, department) IN (
    SELECT first_name, last_name, department
    FROM employees
    GROUP BY first_name, last_name, department
    HAVING COUNT(*) > 1
);
```

Example 3: ROW_NUMBER to Identify Duplicates (Best Method)

```
-- Add row number per duplicate group (row 1 = keep, row 2+ = delete):
SELECT
    emp_id,
    first_name,
    last_name,
    department,
    ROW_NUMBER() OVER (
        PARTITION BY first_name, last_name, department
        ORDER BY emp_id          -- Keep the record with smallest emp_id
    )
```



```
) AS rn
FROM employees;
```

Output in scenario with duplicates:

emp_id	first_name	last_name	department	rn
1	Alice	Kumar	Sales	1
11	Alice	Kumar	Sales	2

Example 4: DELETE Duplicates — Keep First Occurrence

```
-- Method 1: Using NOT IN + subquery with ROW_NUMBER (most portable):
DELETE FROM employees
WHERE emp_id NOT IN (
    SELECT emp_id
    FROM (
        SELECT emp_id,
               ROW_NUMBER() OVER (
                   PARTITION BY first_name, last_name, department
                   ORDER BY emp_id
               ) AS rn
        FROM employees
    ) AS deduped
    WHERE rn = 1
);

-- Method 2: Using a TEMP TABLE (safer for large tables):
CREATE TABLE employees_clean AS
SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY first_name, last_name, department
               ORDER BY emp_id
           ) AS rn
    FROM employees
) ranked
WHERE rn = 1;

DROP TABLE employees;
RENAME TABLE employees_clean TO employees;
```

Example 5: SELECT DISTINCT as Quick Deduplication

```
-- Create deduplicated view:
SELECT DISTINCT first_name, last_name, department, salary
FROM employees;

-- For inserting into a new clean table:
INSERT INTO employees_clean
SELECT DISTINCT first_name, last_name, department, salary, hire_date, city
FROM employees_raw;
```

Example 6: Count Duplicate Customers by Email

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY cnt DESC;
```

Example 7: Duplicate Orders (Same Customer, Same Product, Same Date)

```
SELECT customer_id, product_id, order_date, COUNT(*) AS dup_count
FROM orders
GROUP BY customer_id, product_id, order_date
HAVING COUNT(*) > 1;
```

TOPIC 2: Handling Missing Values

 Concept Explanation

Missing values (NULLs) in SQL can:

- Cause incorrect aggregations (AVG ignores NULLs, so the denominator is smaller)
- Break JOINS (NULLs never match)
- Crash ML models if not handled

Strategies:

1. **Detect** — Find which columns/rows have NULLs

- 2. **Replace** — Fill with defaults, means, or nearby values
- 3. **Remove** — Delete rows with critical NULLs
- 4. **Flag** — Add indicator column: `is_null_city = 1`

🔍 Examples

Example 1: Count NULLs Per Column

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(*) - COUNT(emp_id)      AS null_emp_id,
  COUNT(*) - COUNT(first_name)  AS null_first_name,
  COUNT(*) - COUNT(department) AS null_department,
  COUNT(*) - COUNT(salary)      AS null_salary,
  COUNT(*) - COUNT(manager_id) AS null_manager_id,
  COUNT(*) - COUNT(city)        AS null_city
FROM employees;
```

Output:

total_rows	null_emp_id	null_first_name	null_department	null_salary	null_manager_id	null_city
10	0	0	0	0	3	0

Example 2: Find All Rows with ANY NULL Column

```
SELECT *
FROM employees
WHERE
  emp_id      IS NULL OR
  first_name  IS NULL OR
  department  IS NULL OR
  salary      IS NULL OR
  city        IS NULL;
```

Example 3: Calculate NULL Percentage Per Column

```
SELECT
  'manager_id' AS column_name,
  COUNT(*) AS total,
  COUNT(*) - COUNT(manager_id) AS null_count,
  ROUND((COUNT(*) - COUNT(manager_id)) * 100.0 / COUNT(*), 2) AS null_pct
FROM employees
UNION ALL
SELECT
  'city',
  COUNT(*),
  COUNT(*) - COUNT(city),
  ROUND((COUNT(*) - COUNT(city)) * 100.0 / COUNT(*), 2)
FROM employees;
```

Output:

column_name	total	null_count	null_pct
manager_id	10	3	30.00
city	10	0	0.00

Example 4: Fill NULLs with COALESCE (No Update)

```
-- Show data with NULLs filled by defaults:
SELECT
  emp_id,
  first_name,
  COALESCE(department, 'Unknown')      AS department,
  COALESCE(salary, 0)                  AS salary,
  COALESCE(CAST(manager_id AS CHAR), 'No Manager') AS manager_id,
  COALESCE(city, 'Unknown City')       AS city
FROM employees;
```

Example 5: UPDATE — Fill NULLs Permanently

```
-- Fill NULL department with 'Unknown':
UPDATE employees
SET department = 'Unknown'
WHERE department IS NULL;

-- Fill NULL salary with department average:
UPDATE employees AS e
SET salary = (
  SELECT AVG(salary)
  FROM employees e2
  WHERE e2.department = e.department
```



```
        AND e2.salary IS NOT NULL
    )
WHERE salary IS NULL;
```

Example 6: Mean Imputation — Replace NULL with Column Average

```
-- Replace NULL salary with overall average (for ML feature imputation):
SELECT
    emp_id,
    first_name,
    COALESCE(salary, (SELECT AVG(salary) FROM employees WHERE salary IS NOT NULL)) AS salary_imputed
FROM employees;
```

Example 7: Mode Imputation — Replace NULL with Most Common Value

```
-- What is the most common city?
SELECT city, COUNT(*) AS cnt
FROM employees
WHERE city IS NOT NULL
GROUP BY city
ORDER BY cnt DESC
LIMIT 1;

-- Use that city to fill NULLs:
UPDATE employees
SET city = (
    SELECT city
    FROM (
        SELECT city, COUNT(*) AS cnt
        FROM employees
        WHERE city IS NOT NULL
        GROUP BY city
        ORDER BY cnt DESC
        LIMIT 1
    ) AS mode_city
)
WHERE city IS NULL;
```

Example 8: Add NULL Indicator Column (for ML)

```
-- Instead of removing NULLs, flag them (keeps information):
SELECT
    emp_id,
    salary,
    CASE WHEN salary IS NULL THEN 1 ELSE 0 END AS salary_is_missing,
    COALESCE(salary, 0) AS salary_filled
FROM employees;
```

Example 9: Delete Rows with Critical NULLs

```
-- If salary NULL is unacceptable:
DELETE FROM employees
WHERE salary IS NULL;

-- If any key column is NULL:
DELETE FROM orders
WHERE customer_id IS NULL OR amount IS NULL;
```

Example 10: Forward Fill (Using LAG for Time Series)

```
-- Fill NULL with the previous non-NULL value (forward fill):
SELECT
    order_date,
    amount,
    COALESCE(amount, LAG(amount) OVER (ORDER BY order_date)) AS amount_filled
FROM orders_with_gaps;
```

TOPIC 3: Data Formatting & Standardization

Concept Explanation

Raw data is often inconsistent — different cases, extra spaces, mixed formats. Standardization ensures data quality before analysis or ML training.

Common Issues:

- Inconsistent casing: 'Sales', 'SALES', 'sales'
- Extra whitespace: ' Alice ', 'Alice '
- Date format variations: '2024/01/15', '15-Jan-2024', '01-15-2024'
- Inconsistent categories: 'M', 'Male', 'male', 'm'

- Phone number formats: '9876543210', '+91-9876543210', '98-765-43210'

Examples

Example 1: Standardize Text Case

```
-- Standardize all department names to UPPERCASE:
UPDATE employees
SET department = UPPER(TRIM(department));

-- Standardize all city names to Proper Case (MySQL):
UPDATE employees
SET city = CONCAT(
    UPPER(LEFT(TRIM(city), 1)),
    LOWER(SUBSTRING(TRIM(city), 2))
);

-- View the standardized data:
SELECT department, city FROM employees;
```

Example 2: Remove Extra Whitespace

```
-- Find records with leading/trailing spaces:
SELECT first_name
FROM employees
WHERE first_name != TRIM(first_name);

-- Fix them:
UPDATE employees SET first_name = TRIM(first_name);
UPDATE employees SET last_name  = TRIM(last_name);
UPDATE employees SET city       = TRIM(city);
UPDATE customers  SET email     = TRIM(LOWER(email));
```

Example 3: Standardize Gender Column

```
-- Inconsistent: 'M', 'Male', 'male', 'MALE', 'm', '1'
-- Standardize all to 'M' and 'F':
UPDATE employees
SET gender = CASE
    WHEN LOWER(TRIM(gender)) IN ('m', 'male', '1', 'man') THEN 'M'
    WHEN LOWER(TRIM(gender)) IN ('f', 'female', '0', 'woman') THEN 'F'
    ELSE '0'      -- Other / Unknown
END;
```

Example 4: Standardize Phone Numbers

```
-- Remove all non-numeric characters from phone:
-- (MySQL doesn't have regex replace natively, but here's the pattern for PostgreSQL)

-- PostgreSQL:
UPDATE customers
SET phone = REGEXP_REPLACE(phone, '[^0-9]', '', 'g');
-- '98-765-43210' → '9876543210'
-- '+91-9876543210' → '919876543210'

-- MySQL workaround (for known patterns):
UPDATE customers
SET phone = REPLACE(REPLACE(REPLACE(REPLACE(phone, '-', ''), '+', ''), ' ', ''), '(', '');
```

Example 5: Standardize Date Formats

```
-- Convert messy date strings to proper DATE type:
SELECT STR_TO_DATE('15/01/2024', '%d/%m/%Y') AS clean_date;    -- 2024-01-15
SELECT STR_TO_DATE('January 15, 2024', '%M %d, %Y');          -- 2024-01-15
SELECT STR_TO_DATE('15-Jan-2024', '%d-%b-%Y');                -- 2024-01-15

-- Update table with correct date format:
UPDATE orders_raw
SET order_date = STR_TO_DATE(raw_date_string, '%d/%m/%Y')
WHERE raw_date_string IS NOT NULL;
```

Example 6: Find and Fix Typos / Inconsistent Values

```
-- Audit unique values in department column:
SELECT department, COUNT(*) AS cnt
FROM employees
GROUP BY department
ORDER BY cnt DESC;

-- Common typos in category columns:
```

```
SELECT DISTINCT category FROM products WHERE category LIKE '%ectron%';
-- Finds: 'Electronics', 'Electronnics', 'Electroniques' (typos)

-- Fix known typos:
UPDATE products
SET category = REPLACE(category, 'Electronnics', 'Electronics');
```

Example 7: Validate Email Format

```
-- Find invalid email addresses:
SELECT customer_id, name, email
FROM customers
WHERE
    email NOT LIKE '%@%'           -- No @ sign
    OR email NOT LIKE '%.%'       -- No dot
    OR email LIKE '@%'            -- Starts with @
    OR email LIKE '%@'            -- Ends with @
    OR LENGTH(email) < 6;         -- Too short
```

Example 8: Standardize Salary to Consistent Format

```
-- Remove commas from salary stored as text ('85,000' → 85000):
UPDATE employees_raw
SET salary = CAST(REPLACE(salary_raw, ',', '' ) AS DECIMAL(10,2))
WHERE salary_raw REGEXP '^[0-9,]+$';
```

Example 9: Comprehensive Data Cleaning Pipeline

```
-- Full data cleaning in one query (view/staging):
SELECT
    emp_id,
    TRIM(UPPER(first_name)) AS first_name,
    TRIM(UPPER(last_name)) AS last_name,
    CASE
        WHEN LOWER(TRIM(gender)) IN ('m', 'male') THEN 'M'
        WHEN LOWER(TRIM(gender)) IN ('f', 'female') THEN 'F'
        ELSE '0'
    END AS gender,
    TRIM(UPPER(department)) AS department,
    COALESCE(salary, 0) AS salary,
    COALESCE(TRIM(city), 'Unknown') AS city,
    hire_date AS hire_date
FROM employees_raw
WHERE emp_id IS NOT NULL      -- Remove rows with no ID
    AND first_name IS NOT NULL -- Remove rows with no name
ORDER BY emp_id;
```

Example 10: Outlier Detection with SQL

```
-- Find salary outliers (outside 3 standard deviations):
WITH stats AS (
    SELECT
        AVG(salary) AS mean_sal,
        STDDEV(salary) AS std_sal
    FROM employees
)
SELECT
    e.first_name,
    e.salary,
    s.mean_sal,
    s.std_sal,
    (e.salary - s.mean_sal) / NULLIF(s.std_sal, 0) AS z_score
FROM employees AS e, stats AS s
WHERE ABS((e.salary - s.mean_sal) / NULLIF(s.std_sal, 0)) > 3;

-- Find IQR-based outliers:
WITH quartiles AS (
    SELECT
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3
    FROM employees
)
SELECT e.first_name, e.salary
FROM employees e, quartiles q
WHERE e.salary < q.q1 - 1.5 * (q.q3 - q.q1)
    OR e.salary > q.q3 + 1.5 * (q.q3 - q.q1);
```

Practice Questions

● Easy Level

- Q1.** Write a query to find all customers whose email is NULL.
- Q2.** Write a query to count NULLs in the `manager_id` column of employees.
- Q3.** Write a query to display all employees with NULL replaced by 'N/A' in the `city` column.
- Q4.** Write a query to find all departments that appear MORE THAN ONCE in the employees table (used as a dedup check).
- Q5.** Write a query to display the `gender` column standardized to 'Male' / 'Female' / 'Other' using CASE.

Medium Level

- Q6.** Write a query that shows a null summary: for each column in a mini dataset (name, email, city), count total rows, null count, and null percentage.
- Q7.** Write a query using ROW_NUMBER() to identify duplicate employees (same first_name + last_name + department). Show all rows with rn > 1 (the duplicates).
- Q8.** Write a SELECT query to show all employees with "cleaned" data:
- Trim whitespace from first/last name
 - Standardize department to UPPER CASE
 - Replace NULL city with 'Unknown'
 - Replace NULL manager_id with 0
- Q9.** Write a data validation query to find all orders where:
- Amount is NULL or 0 or negative
 - customer_id is NULL
 - order_date is NULL or in the future
- Q10.** Write a query to find and count all distinct `status` values in the orders table (to identify any inconsistent formatting like 'delivered', 'DELIVERED', 'Delivered').

Hard / Interview Level

- Q11.** Given the orders table, write a SQL query to:
- Count total records
 - Count records with NULL amount
 - Count records with amount = 0
 - Count records with amount < 0
 - Count records outside the normal range (amount > 500000)

All in a single query.

- Q12.** Write a complete data profiling query for the `employees` table that shows:
- Column name
 - Count of non-null values
 - Count of null values
 - Null percentage
 - Count of distinct values

(Done via UNION ALL for each column)

- Q13.** Write a query to "flag" potential data quality issues in the customers table:
- `email_valid` : 0 if email doesn't contain '@', else 1
 - `age_valid` : 0 if age < 0 or age > 120, else 1
 - `city_missing` : 1 if city is NULL, else 0
- Q14.** You received a raw CSV dump with the following issues in the employees table. Write the UPDATE statements to fix each:
- Some `department` values are 'hr', 'HR', 'Hr' — standardize to 'HR'
 - Some `city` values have trailing spaces — fix them
 - NULL `salary` values — set to -1 as a sentinel value

Solutions

Q1.

```
SELECT customer_id, name, city
FROM customers
WHERE email IS NULL;
-- None in our clean dataset; would show real records in dirty data
```

Q2.

```
SELECT COUNT(*) - COUNT(manager_id) AS null_manager_count
FROM employees;
-- Output: 3
```

Q3.

```
SELECT
    first_name,
    COALESCE(city, 'N/A') AS city
FROM employees;
```

Q4.

```
SELECT department, COUNT(*) AS count
FROM employees
GROUP BY department
HAVING COUNT(*) > 1
ORDER BY count DESC;
```

Output:

department	count
Sales	3
IT	3
HR	2

(These are expected — same dept, multiple employees. Duplicate detection should use more columns.)

Q5.

```
SELECT
  first_name,
  gender,
  CASE
    WHEN gender = 'M' THEN 'Male'
    WHEN gender = 'F' THEN 'Female'
    ELSE 'Other'
  END AS gender_label
FROM employees;
```

Q6.

```
SELECT
  'name' AS column_name,
  COUNT(*) AS total,
  COUNT(*) - COUNT(name) AS null_count,
  ROUND((COUNT(*) - COUNT(name)) * 100.0 / COUNT(*), 1) AS null_pct
FROM customers
UNION ALL
SELECT 'email', COUNT(*), COUNT(*) - COUNT(email),
  ROUND((COUNT(*) - COUNT(email)) * 100.0 / COUNT(*), 1)
FROM customers
UNION ALL
SELECT 'city', COUNT(*), COUNT(*) - COUNT(city),
  ROUND((COUNT(*) - COUNT(city)) * 100.0 / COUNT(*), 1)
FROM customers;
```

Q7.

```
SELECT *
FROM (
  SELECT
    emp_id,
    first_name,
    last_name,
    department,
    ROW_NUMBER() OVER (
      PARTITION BY first_name, last_name, department
      ORDER BY emp_id
    ) AS rn
  FROM employees
) ranked
WHERE rn > 1;
```

Q8.

```
SELECT
  emp_id,
  TRIM(first_name) AS first_name,
  TRIM(last_name) AS last_name,
  UPPER(TRIM(department)) AS department,
  COALESCE(city, 'Unknown') AS city,
  COALESCE(manager_id, 0) AS manager_id
FROM employees;
```

Q9.

```
SELECT order_id, customer_id, amount, order_date
FROM orders
WHERE
  amount IS NULL OR amount <= 0
  OR customer_id IS NULL
  OR order_date IS NULL
  OR order_date > CURRENT_DATE;
```

Q10.

```
SELECT DISTINCT status, COUNT(*) AS cnt
FROM orders
GROUP BY status
ORDER BY status;

-- If you see 'delivered', 'Delivered', 'DELIVERED' – those are data quality issues
```

Q11.

```
SELECT
    COUNT(*)                AS total_records,
    COUNT(*) - COUNT(amount) AS null_amount,
    SUM(CASE WHEN amount = 0 THEN 1 ELSE 0 END) AS zero_amount,
    SUM(CASE WHEN amount < 0 THEN 1 ELSE 0 END) AS negative_amount,
    SUM(CASE WHEN amount > 500000 THEN 1 ELSE 0 END) AS outlier_amount
FROM orders;
```

Q12.

```
-- Data profiling for employees table (UNION ALL for each column):
SELECT 'emp_id'      AS col, COUNT(emp_id)      AS non_null, COUNT(*) - COUNT(emp_id)      AS nulls, ROUND((COUNT(*)-COUNT(e
mp_id))*100.0/COUNT(*),1), COUNT(DISTINCT emp_id)      AS distinct_vals FROM employees UNION ALL
SELECT 'first_name', COUNT(first_name), COUNT(*)-COUNT(first_name), ROUND((COUNT(*)-COUNT(first_name))*100.0/COUNT(*),1),
COUNT(DISTINCT first_name) FROM employees UNION ALL
SELECT 'department', COUNT(department), COUNT(*)-COUNT(department), ROUND((COUNT(*)-COUNT(department))*100.0/COUNT(*),1),
COUNT(DISTINCT department) FROM employees UNION ALL
SELECT 'salary',     COUNT(salary),     COUNT(*)-COUNT(salary),     ROUND((COUNT(*)-COUNT(salary))*100.0/COUNT(*),1),
COUNT(DISTINCT salary)     FROM employees UNION ALL
SELECT 'manager_id', COUNT(manager_id), COUNT(*)-COUNT(manager_id), ROUND((COUNT(*)-COUNT(manager_id))*100.0/COUNT(*),1),
COUNT(DISTINCT manager_id) FROM employees UNION ALL
SELECT 'city',        COUNT(city),        COUNT(*)-COUNT(city),        ROUND((COUNT(*)-COUNT(city))*100.0/COUNT(*),1),
COUNT(DISTINCT city)        FROM employees;
```

Q13.

```
SELECT
    customer_id,
    name,
    email,
    age,
    city,
    CASE WHEN email LIKE '%@%' THEN 1 ELSE 0 END AS email_valid,
    CASE WHEN age BETWEEN 0 AND 120 THEN 1 ELSE 0 END AS age_valid,
    CASE WHEN city IS NULL THEN 1 ELSE 0 END AS city_missing
FROM customers;
```

Q14.

```
-- Fix department casing:
UPDATE employees
SET department = UPPER(department)
WHERE LOWER(department) = 'hr' OR department = 'Hr';

-- Fix trailing spaces in city:
UPDATE employees
SET city = RTRIM(city)
WHERE city != RTRIM(city);

-- Fill NULL salary with sentinel -1:
UPDATE employees
SET salary = -1
WHERE salary IS NULL;
```

 Real-World Use Cases

Technique	Data Science Application
Duplicate detection	Prevent double-counting in metrics; clean training data
NULL detection	Data quality checks before feature engineering
NULL imputation (SQL)	Mean/mode fill at scale without loading into Python
Standardization	Consistent categorical encoding, case normalization
Outlier detection	Z-score / IQR flagging before model training
Data profiling	Automated per-column quality reports for pipelines

 Phase 14: Performance Basics

TOPIC 1: Indexes — Concept & Types

📘 Concept Explanation

An **index** is a data structure that allows the database to find rows **much faster**, without scanning the entire table.

Analogy: An index in SQL is like the index at the back of a textbook. Instead of reading every page to find "normalization," you jump directly to page 145. Without an index, the DB scans every row (full table scan).

How Indexes Work:

```
Without Index:
SELECT * FROM employees WHERE email = 'alice@company.com';

Database scans ROW 1 → check → no
                ROW 2 → check → no
                ROW 3 → check → no
                ...
                ROW 1,000,000 → found!

With Index on email:
Database jumps directly to ROW location → found in microseconds
```

Index tradeoffs:

Tradeoff	Detail
✅ Faster reads	SELECT queries on indexed columns are dramatically faster
❌ Slower writes	INSERT, UPDATE, DELETE must also update index
❌ More storage	Indexes consume disk space
❌ Memory usage	Indexes are loaded into memory (buffer pool)

Rule of thumb: Index columns that are frequently used in:

- `WHERE` clause filters
- `JOIN` ON conditions
- `ORDER BY` columns
- `GROUP BY` columns

📖 Syntax

```
-- Create a simple index:
CREATE INDEX idx_name ON table_name (column_name);

-- Create index with custom name:
CREATE INDEX idx_employee_dept ON employees (department);

-- Unique index (enforces uniqueness + faster lookups):
CREATE UNIQUE INDEX idx_email ON customers (email);

-- Composite index (multiple columns):
CREATE INDEX idx_dept_salary ON employees (department, salary);

-- Drop an index:
DROP INDEX idx_name ON table_name;      -- MySQL
DROP INDEX idx_name;                    -- PostgreSQL

-- View indexes on a table:
SHOW INDEX FROM employees;              -- MySQL
\d employees                            -- PostgreSQL
```

🔍 Types of Indexes

Type 1: Primary Key Index (Clustered)

```
-- Primary key automatically creates an index:
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,    -- Index created automatically
  name VARCHAR(100)
);
```

The table's physical row order matches the primary key order (clustered index in SQL Server/MySQL InnoDB).

Type 2: Simple Index (Non-Clustered)

```
-- Index on department for fast department-based lookups:
CREATE INDEX idx_department ON employees (department);

-- Now this query is fast:
SELECT * FROM employees WHERE department = 'IT';
```

Type 3: Unique Index

```
-- Ensures no duplicate emails + speeds up lookup:
CREATE UNIQUE INDEX idx_unique_email ON customers (email);

-- Both of these are now fast:
SELECT * FROM customers WHERE email = 'alice@company.com';
-- AND: prevents inserting a duplicate email
```

Type 4: Composite Index (Multi-Column)

```
-- Index on (department, salary) together:
CREATE INDEX idx_dept_sal ON employees (department, salary);

-- Uses index:
SELECT * FROM employees WHERE department = 'IT' AND salary > 80000;
SELECT * FROM employees WHERE department = 'IT'; -- Also uses (leftmost prefix)

-- Does NOT use composite index efficiently:
SELECT * FROM employees WHERE salary > 80000; -- Leftmost column (dept) not used
```

Key Rule: The **leftmost prefix rule** — composite index on (A, B, C) can be used for: A alone, A+B together, A+B+C together — but NOT B alone or C alone.

Type 5: Full-Text Index

```
-- For fast text/keyword search in large text columns:
CREATE FULLTEXT INDEX idx_ft_name ON products (name, description);

-- Use with MATCH...AGAINST instead of LIKE:
SELECT * FROM products
WHERE MATCH(name, description) AGAINST ('laptop gaming');
-- Much faster than: WHERE name LIKE '%laptop%'
```

When to Index:

- ✅ Index these:
 - Primary keys (auto-indexed)
 - Foreign keys (customer_id in orders)
 - Columns in WHERE clauses (department, status, city)
 - Columns in JOIN conditions
 - Columns in ORDER BY (for sorted scans)
- ❌ Don't index these:
 - Columns with very low cardinality (gender: only M/F)
 - Columns rarely used in queries
 - Small tables (< 1000 rows — full scan is fine)
 - Frequently updated columns (index maintenance overhead)

TOPIC 2: Query Optimization Basics

Concept Explanation

Query optimization is about writing SQL that the database executes as efficiently as possible — using indexes, minimizing data scanned, reducing memory usage.

Key Optimization Rules

Rule 1: SELECT Specific Columns — Not SELECT *

```
-- ❌ SLOW and wasteful (reads all columns, lots of data transfer):
SELECT * FROM employees;

-- ✅ FAST (reads only what you need):
SELECT first_name, salary, department FROM employees;
```

Rule 2: Filter Early with WHERE

```
-- ❌ SLOW (joins all rows, then filters):
SELECT e.first_name, o.amount
FROM employees e
JOIN orders o ON e.emp_id = o.customer_id; -- Doesn't exist but illustrates point

-- ✅ FAST (filter before joining):
SELECT e.first_name, o.amount
FROM employees e
JOIN orders o ON e.emp_id = o.customer_id
WHERE o.status = 'Delivered' AND e.department = 'Sales';
```

Rule 3: Use Indexes — Avoid Functions on Indexed Columns


```
-- ❌ SLOW – function on indexed column prevents index use:
SELECT * FROM employees WHERE YEAR(hire_date) = 2020;
SELECT * FROM employees WHERE UPPER(department) = 'SALES';
SELECT * FROM employees WHERE salary + 10000 > 90000;

-- ✅ FAST – rearrange to use index on raw column:
SELECT * FROM employees
WHERE hire_date BETWEEN '2020-01-01' AND '2020-12-31';

SELECT * FROM employees WHERE department = 'Sales';    -- If data is clean
SELECT * FROM employees WHERE salary > 80000;          -- No function
```

Rule 4: Use EXISTS Instead of COUNT for Existence Checks

```
-- ❌ SLOW – counts ALL matching rows even though you only care about existence:
SELECT * FROM customers c
WHERE (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) > 0;

-- ✅ FAST – EXISTS stops at the first match:
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Rule 5: Avoid SELECT DISTINCT Unless Necessary

```
-- ❌ SLOW – DISTINCT requires sorting all results to find unique values:
SELECT DISTINCT department FROM employees;    -- Still fast on small tables

-- ✅ FASTER alternative if you just need to know unique values:
SELECT department FROM employees GROUP BY department;

-- If you need unique records, use DISTINCT; just be aware of cost on large tables
```

Rule 6: LIMIT Early

```
-- ❌ Process all 1M rows, then return 10:
SELECT * FROM huge_table ORDER BY created_at DESC;    -- Show in app without limiting!

-- ✅ Limit immediately:
SELECT * FROM huge_table ORDER BY created_at DESC LIMIT 10;
```

Rule 7: Avoid OR on Indexed Columns — Use IN or UNION

```
-- ❌ OR prevents index use in some databases:
SELECT * FROM employees WHERE department = 'IT' OR department = 'Sales';

-- ✅ Use IN (index-friendly):
SELECT * FROM employees WHERE department IN ('IT', 'Sales');

-- ✅ Or UNION (when conditions are on different indexed columns):
SELECT * FROM employees WHERE department = 'IT'
UNION
SELECT * FROM employees WHERE city = 'Mumbai';
```

Rule 8: Avoid Leading Wildcards in LIKE

```
-- ❌ SLOW – leading % prevents index use (full table scan):
SELECT * FROM customers WHERE name LIKE '%Ali%';

-- ✅ FAST – trailing % uses index:
SELECT * FROM customers WHERE name LIKE 'Ali%';    -- Can use index!

-- For full-text search, use FULLTEXT index with MATCH AGAINST
```

Rule 9: Correlated Subqueries → Rewrite as JOINS

```
-- ❌ SLOW – Correlated subquery runs ONCE PER ROW:
SELECT first_name, salary
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department = e.department);

-- ✅ FAST – Pre-compute averages with JOIN:
SELECT e.first_name, e.salary
FROM employees e
JOIN (
  SELECT department, AVG(salary) AS avg_sal
  FROM employees
  GROUP BY department
) dept_avg ON e.department = dept_avg.department
WHERE e.salary > dept_avg.avg_sal;
```

Rule 10: Use CTEs or Subqueries Instead of Repeating Calculations

```
-- ❌ BAD – avg_salary computed TWICE:
SELECT first_name,
       salary - (SELECT AVG(salary) FROM employees) AS diff,
       (SELECT AVG(salary) FROM employees) AS avg
FROM employees;

-- ✅ BETTER – compute once in CTE:
WITH avg_cte AS (SELECT AVG(salary) AS avg_sal FROM employees)
SELECT
  e.first_name,
  e.salary - a.avg_sal AS diff,
  a.avg_sal AS avg
FROM employees e, avg_cte a;
```

📊 Query Optimization Checklist

```
Before running any query, check:
❑ Did I use SELECT * unnecessarily?
❑ Do I have a WHERE clause to filter rows early?
❑ Are the JOIN columns indexed?
❑ Am I applying functions on WHERE/JOIN columns (prevents index use)?
❑ Can I rewrite correlated subqueries as JOINS?
❑ Did I add LIMIT when exploring large tables?
❑ Am I using LIKE with a leading % on a large text column?
❑ Could an index help this query?
```

TOPIC 3: EXPLAIN / EXPLAIN ANALYZE

📘 Concept Explanation

EXPLAIN shows the **execution plan** — how the database plans to execute your query. This is how you identify:

- Full table scans (bad for large tables)
- Missing indexes
- Expensive sort operations

📄 Syntax

```
-- MySQL:
EXPLAIN SELECT * FROM employees WHERE department = 'IT';

-- MySQL (with actual execution stats):
EXPLAIN ANALYZE SELECT * FROM employees WHERE department = 'IT';

-- PostgreSQL:
EXPLAIN SELECT * FROM employees WHERE department = 'IT';
EXPLAIN ANALYZE SELECT * FROM employees WHERE department = 'IT';

-- SQL Server:
SET STATISTICS IO ON;
SELECT * FROM employees WHERE department = 'IT';
```

🔍 Examples

Example 1: EXPLAIN Output (MySQL)

Before creating index:

```
EXPLAIN SELECT * FROM employees WHERE department = 'IT';
```

Output:

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employees	ALL	NULL	10	Using where

`type = ALL` means **FULL TABLE SCAN** — reads every row. `key = NULL` means no index is being used. `rows = 10` — estimates 10 rows scanned (our full table).

After creating index:

```
CREATE INDEX idx_dept ON employees (department);
EXPLAIN SELECT * FROM employees WHERE department = 'IT';
```

Output:

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employees	ref	idx_dept	3	Using index condition

`type = ref` — uses the index (much better than ALL). `key = idx_dept` — confirms index is used. `rows = 3` — only 3 rows estimated (just IT employees).

Example 2: Reading EXPLAIN Output (Key Column Types)

type	Value	Meaning	Performance
system		Single row in table	Best ✓✓✓
const		Primary key lookup	Best ✓✓✓
eq_ref		Unique index per row	Great ✓✓
ref		Non-unique index used	Good ✓
range		Index range scan (BETWEEN, IN)	OK ⚠
index		Full INDEX scan	Slow ✗
ALL		Full TABLE scan	Worst ✗✗

Example 3: Finding Slow Queries in MySQL

```
-- Enable slow query log in MySQL:
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1.0;    -- Log queries slower than 1 second

-- Or: Check the slow query log file for recent slow queries
-- Location: /var/log/mysql/mysql-slow.log
```

Practice Questions

● Easy Level

- Q1.** Write the SQL to create an index named `idx_salary` on the `salary` column of the `employees` table.
- Q2.** Write the SQL to create a UNIQUE index on the `email` column of the `customers` table.
- Q3.** Which of the following queries would better use an index on `department` ? Why?

```
-- A:
SELECT * FROM employees WHERE UPPER(department) = 'SALES';
-- B:
SELECT * FROM employees WHERE department = 'Sales';
```

- Q4.** Write the SQL to drop the index `idx_salary` from the employees table (MySQL syntax).
- Q5.** Write the EXPLAIN command to check how the database executes:

```
SELECT * FROM employees WHERE department = 'IT' AND salary > 80000;
```

● Medium Level

- Q6.** You have a table with 10 million orders. A query runs slowly:

```
SELECT * FROM orders WHERE YEAR(order_date) = 2024;
```

Rewrite this query to use an index on `order_date` .

- Q7.** Which columns in the `orders` table would you index, and why? Consider: `order_id`, `customer_id`, `product_id`, `amount`, `order_date`, `status`.
- Q8.** Rewrite this slow correlated subquery as a faster JOIN:

```
SELECT first_name FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department = e.department);
```

- Q9.** A query uses `LIKE '%Kumar'` . Why is this slow, and what alternatives exist?
- Q10.** What is a composite index? Create a composite index on `employees` for queries that filter by `department` and then sort by `salary` .

● Hard / Interview Level

- Q11.** Explain the difference between a clustered index and a non-clustered index. Which does MySQL InnoDB use for PRIMARY KEY?
- Q12.** A table has 100M rows. This query takes 30 seconds:

```
SELECT customer_id, SUM(amount)
FROM orders
WHERE status = 'Delivered'
GROUP BY customer_id;
```

What indexes would you create? Rewrite the query if needed.

- Q13.** What is index cardinality? Why should you NOT create an index on a low-cardinality column like `gender` (only values: M, F)?
- Q14.** Read the following EXPLAIN output and identify the problem:

```
type = ALL, key = NULL, rows = 5000000, Extra = Using filesort
```

What two problems do you see? How would you fix them?

Solutions

Q1.

```
CREATE INDEX idx_salary ON employees (salary);
```

Q2.

```
CREATE UNIQUE INDEX idx_unique_email ON customers (email);
```

Q3.

Query B (WHERE department = 'Sales') would better use the index.

Query A has UPPER(department) – a function applied to the column. This transforms the indexed value before comparison, so the index on raw `department` values cannot be used. The DB falls back to a full table scan.

Fix for A: Standardize data to always UPPERCASE first, then query without function:
WHERE department = 'SALES' -- if data is stored in UPPER already

Q4.

```
DROP INDEX idx_salary ON employees; -- MySQL syntax
```

Q5.

```
EXPLAIN SELECT * FROM employees WHERE department = 'IT' AND salary > 80000;
```

Q6.

```
-- ❌ Original (cannot use index on order_date):  
SELECT * FROM orders WHERE YEAR(order_date) = 2024;  
  
-- ✅ Rewritten (uses index on order_date – BETWEEN range scan):  
SELECT * FROM orders  
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Q7.

Column	Index?	Reason
order_id	YES (PK)	Already a primary key – auto-indexed
customer_id	YES	Frequently used in JOINS with customers table
product_id	YES	Frequently used in JOINS with products table
status	YES	Often filtered (WHERE status = 'Delivered')
order_date	YES	Date range queries, ORDER BY, time-series analysis
amount	MAYBE	Only if frequently filtered or sorted on amount alone

Composite index: (customer_id, order_date) for queries like:
SELECT * FROM orders WHERE customer_id = 1 ORDER BY order_date;

Q8.

```
-- Rewritten as JOIN:  
SELECT e.first_name  
FROM employees e  
JOIN (  
    SELECT department, AVG(salary) AS dept_avg  
    FROM employees  
    GROUP BY department  
) dept ON e.department = dept.department  
WHERE e.salary > dept.dept_avg;
```

Q9.

LIKE '%Kumar' – leading wildcard is the problem.

Reason: Index B-Tree is sorted from left to right. When you put % at the START, the DB doesn't know where to start in the index – it must scan every entry. This defeats the index.

Alternatives:

- Reverse the string and store as computed column, then search:
WHERE REVERSE(last_name) LIKE REVERSE('%Kumar') -- still not great
- Use FULLTEXT index with MATCH...AGAINST for text search
- If searching last_name ending with 'Kumar', store last_name separately:
WHERE last_name = 'Kumar' -- exact match – uses index perfectly
- Use a search engine (Elasticsearch) for heavy text search requirements

Q10.

```
CREATE INDEX idx_dept_salary ON employees (department, salary);  
  
-- This index helps:  
SELECT * FROM employees WHERE department = 'IT' ORDER BY salary DESC;
```

```
SELECT * FROM employees WHERE department = 'IT' AND salary > 80000;
SELECT * FROM employees WHERE department = 'IT'; -- Leftmost column works alone

-- Does NOT help:
SELECT * FROM employees WHERE salary > 80000; -- No leading department filter
```

Q11.

Clustered Index:

- Determines the PHYSICAL ORDER of rows on disk
- Table data IS the index (data and index are one structure)
- MySQL InnoDB uses the PRIMARY KEY as the clustered index
- Only ONE clustered index per table (since data has one physical order)
- Lookup by PK is extremely fast (data is right there)

Non-Clustered Index:

- Separate data structure that POINTS to the actual row location
- Contains index key + a pointer/reference to the actual data row
- Multiple non-clustered indexes allowed per table
- Slightly slower than clustered (requires a "bookmark lookup" to fetch data)

MySQL InnoDB:

- PRIMARY KEY = clustered index (data ordered by PK)
- All other indexes are non-clustered (they store PK as the row reference)

Q12.

```
-- Both of these would help:

-- Index 1: Filter by status quickly:
CREATE INDEX idx_status ON orders (status);

-- Index 2: Composite for filtering + grouping:
CREATE INDEX idx_status_customer ON orders (status, customer_id);

-- Index 3: Covering index (avoids going back to table row at all):
CREATE INDEX idx_covering ON orders (status, customer_id, amount);

-- The original query is already well-written. Just needs the index.
-- If still slow: Consider partitioning by status or date.
```

Q13.

Cardinality = Number of unique values in a column.

High cardinality (email, customer_id): 1M unique values → Index is VERY useful
Medium cardinality (city, department): 10-100 unique values → Index may help
Low cardinality (gender: M/F, is_active: 0/1): 2 unique values → Index rarely helps

Why NOT index gender (M/F)?

- If 50% of rows are 'M' and 50% are 'F', an index returns half the table
- The DB optimizer decides: "If I use the index, I still have to fetch 5M rows. It's faster to just do a full table scan."
- The overhead of maintaining the index outweighs the benefit

Rule: Index columns with cardinality ≥ 100+ unique values,
or where filtered results are ≤ 5-10% of total rows.

Q14.

```
EXPLAIN output: type = ALL, key = NULL, rows = 5000000, Extra = Using filesort
```

Problem 1: type = ALL → FULL TABLE SCAN

- No index is being used (key = NULL)
- Database is reading ALL 5 million rows
- Fix: Create an index on the filter column (WHERE clause column)

Problem 2: Extra = Using filesort → EXTERNAL SORT

- The results are being sorted but no index supports the sort
- Database is sorting 5M rows in memory/disk (very slow)
- Fix: Create an index on the ORDER BY column
- Or: Create a composite index on (WHERE_column, ORDER_BY_column)

Combined Fix:

```
CREATE INDEX idx_fix ON table (where_column, order_by_column);
```

Real-World Use Cases

Concept	Data Science / Engineering Application
Index on FK	Fast JOINS between large fact and dimension tables
Composite Index	(user_id, event_date) index for time-series user event queries
EXPLAIN	Profile slow feature extraction queries before ML pipeline runs

Concept	Data Science / Engineering Application
Avoid func in WHERE	Pushdown predicates to storage — critical for BigQuery/Snowflake
Covering Index	Serve analytical queries entirely from index — no table scan needed
Partitioning	Partition large tables by date — scan only the month you need

Final Phase: Combination Practice & Interview-Level Problems 🔥🔥🔥

OVERVIEW

This final phase is the "Boss Level." Here, we combine everything learned from Phase 1 to Phase 14. These problems are designed to simulate **real-world Data Science tasks** and **Top-Tier Tech Interview questions** (Amazon, Google, Meta, etc.).

Concepts to Combine:

- CTEs & Subqueries
- Window Functions (Rank, Lead, Lag, Sum Over)
- Complex Joins (Self-Joins, Multiple Joins)
- Data Cleaning & Transformation (CASE, COALESCE)
- Date & String Manipulation

THE CHALLENGES

Challenge 1: Customer Churn Analysis (Recency)

Scenario: A "Churned" customer is someone who hasn't placed an order in the last 60 days.

Task: Write a query to identify churned customers. For each churned customer, show:

- Customer Name
- Date of their last order
- Days since last order
- Total they spent in their lifetime

```
WITH customer_last_order AS (
  SELECT
    customer_id,
    MAX(order_date) AS last_order_date,
    SUM(amount) AS lifetime_value
  FROM orders
  GROUP BY customer_id
)
SELECT
  c.name,
  clo.last_order_date,
  DATEDIFF(CURRENT_DATE, clo.last_order_date) AS days_since_last,
  clo.lifetime_value
FROM customers AS c
INNER JOIN customer_last_order AS clo ON c.customer_id = clo.customer_id
WHERE DATEDIFF(CURRENT_DATE, clo.last_order_date) > 60;
```

Challenge 2: Cohort Analysis (Monthly Retention)

Scenario: We want to see how much revenue each "signup month" cohort generates over time.

Task: Find the total revenue generated by customers grouped by the month they placed their **first ever order**.

```
WITH first_orders AS (
  -- Find the first order month for each customer
  SELECT
    customer_id,
    MIN(order_date) AS first_order_date
  FROM orders
  GROUP BY customer_id
),
cohorts AS (
  -- Assign each customer to a cohort month
  SELECT
    customer_id,
    DATE_FORMAT(first_order_date, '%Y-%m') AS cohort_month
  FROM first_orders
)
SELECT
  ch.cohort_month,
  DATE_FORMAT(o.order_date, '%Y-%m') AS order_month,
  SUM(o.amount) AS revenue
FROM orders AS o
JOIN cohorts AS ch ON o.customer_id = ch.customer_id
GROUP BY ch.cohort_month, order_month
ORDER BY ch.cohort_month, order_month;
```


Challenge 3: RFM Scoring (Data Science Classic)

Scenario: Segment customers using Recency (R), Frequency (F), and Monetary (M).

Task: Assign a score from 1-4 for each category (using NTILE) and calculate a total RFM score.

```
WITH rfm_raw AS (
  SELECT
    customer_id,
    DATEDIFF(CURRENT_DATE, MAX(order_date)) AS recency,
    COUNT(order_id) AS frequency,
    SUM(amount) AS monetary
  FROM orders
  GROUP BY customer_id
),
rfm_scores AS (
  SELECT
    customer_id,
    NTILE(4) OVER (ORDER BY recency DESC) AS r_score,      -- Lower recency is better
    NTILE(4) OVER (ORDER BY frequency ASC) AS f_score,     -- Higher frequency is better
    NTILE(4) OVER (ORDER BY monetary ASC) AS m_score      -- Higher monetary is better
  FROM rfm_raw
)
SELECT
  c.name,
  r_score, f_score, m_score,
  (r_score + f_score + m_score) AS total_rfm_score
FROM rfm_scores rs
JOIN customers c ON rs.customer_id = c.customer_id
ORDER BY total_rfm_score DESC;
```

Challenge 4: Top 2 Trending Products per Category

Scenario: We need to find the most popular products in each category based on sales count.

Task: List the top 2 products (by order count) for each category. Handle ties using `DENSE_RANK()`.

```
WITH product_sales AS (
  SELECT
    p.category,
    p.name AS product_name,
    COUNT(o.order_id) AS sales_count
  FROM products p
  JOIN orders o ON p.product_id = o.product_id
  GROUP BY p.category, p.name
),
ranked_sales AS (
  SELECT
    *,
    DENSE_RANK() OVER (PARTITION BY category ORDER BY sales_count DESC) AS rnk
  FROM product_sales
)
SELECT category, product_name, sales_count
FROM ranked_sales
WHERE rnk <= 2;
```

Challenge 5: Year-over-Year (YoY) Growth %

Scenario: Compare this month's revenue to the same month last year.

Task: Calculate the YoY growth percentage for each month.

```
WITH monthly_rev AS (
  SELECT
    YEAR(order_date) AS yr,
    MONTH(order_date) AS mth,
    SUM(amount) AS revenue
  FROM orders
  GROUP BY YEAR(order_date), MONTH(order_date)
),
yoy_prep AS (
  SELECT
    yr,
    mth,
    revenue,
    LAG(revenue) OVER (PARTITION BY mth ORDER BY yr) AS last_year_revenue
  FROM monthly_rev
)
SELECT
  yr,
  mth,
  revenue,
  last_year_revenue,
  ROUND((revenue - last_year_revenue) * 100.0 / last_year_revenue, 2) AS yoy_growth_pct
FROM yoy_prep
WHERE last_year_revenue IS NOT NULL;
```


Challenge 6: The "Gap" Analysis

Scenario: Identify customers who placed an order in January but **NOT** in February.

Task: Use a Set Operation (EXCEPT/MINUS) or a JOIN/Subquery workaround.

```
-- Using EXCEPT (PostgreSQL/SQL Server)
SELECT customer_id FROM orders WHERE MONTH(order_date) = 1
EXCEPT
SELECT customer_id FROM orders WHERE MONTH(order_date) = 2;

-- MySQL Workaround:
SELECT DISTINCT customer_id
FROM orders
WHERE MONTH(order_date) = 1
AND customer_id NOT IN (
    SELECT customer_id FROM orders WHERE MONTH(order_date) = 2
);
```

Challenge 7: Running Percentage of Total

Scenario: In the `employees` table, find the contribution of each employee's salary to the total department budget, and show a running total of those percentages.

Task: Use Window Functions for both the total and the running sum.

```
WITH dept_data AS (
    SELECT
        first_name,
        department,
        salary,
        SUM(salary) OVER (PARTITION BY department) AS dept_total
    FROM employees
)
SELECT
    first_name,
    department,
    salary,
    ROUND(salary * 100.0 / dept_total, 2) AS pct_of_dept,
    ROUND(SUM(salary * 100.0 / dept_total) OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ), 2) AS running_pct
FROM dept_data;
```

Challenge 8: Identifying Overlapping Tenures (Self-Join)

Scenario: Find if any employee was hired on the same exact day as another employee in the same city.

Task: Use a Self-Join to find pairs.

```
SELECT
    e1.first_name AS emp1,
    e2.first_name AS emp2,
    e1.hire_date,
    e1.city
FROM employees e1
JOIN employees e2 ON e1.hire_date = e2.hire_date
    AND e1.city = e2.city
    AND e1.emp_id < e2.emp_id; -- Avoid self-pairing and duplicate pairs
```

Challenge 9: The "No-Order" Deadweight

Scenario: We want to find customers who signed up but never placed an order, AND employees who have no subordinates (leaf nodes in hierarchy).

Task: Combine these two lists into one "Audit" table using UNION.

```
SELECT name AS entity_name, 'Idle Customer' AS status
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders)

UNION ALL

SELECT first_name, 'No Subordinates'
FROM employees
WHERE emp_id NOT IN (SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL);
```

Challenge 10: ABC Analysis (Inventory Management)

Scenario: Classify products by revenue:

- Top 70% of revenue → 'A' (Core products)
 - Next 20% → 'B'
 - Bottom 10% → 'C'
- Task:** Use a running total percentage to classify.

```
WITH product_revenue AS (
    SELECT
```



```
        p.name,
        SUM(o.amount) AS total_rev
    FROM products p
    JOIN orders o ON p.product_id = o.product_id
    GROUP BY p.name
),
running_revenue AS (
    SELECT
        *,
        SUM(total_rev) OVER (ORDER BY total_rev DESC) AS cum_rev,
        SUM(total_rev) OVER () AS total_rev_global
    FROM product_revenue
),
pct_calc AS (
    SELECT
        *,
        (cum_rev * 100.0 / total_rev_global) AS running_pct
    FROM running_revenue
)
SELECT
    name,
    total_rev,
    CASE
        WHEN running_pct <= 70 THEN 'A'
        WHEN running_pct <= 90 THEN 'B'
        ELSE 'C'
    END AS abc_class
FROM pct_calc;
```

💡 INTERVIEW TIPS FOR DATA SCIENCE

1. **Ask Clarifying Questions:** Before writing SQL, ask: "How do we handle ties?", "Should we include customers with zero orders?", "Do we use 365 or 365.25 for age?"
2. **Think in CTEs:** Even if a subquery works, use CTEs in interviews. It shows you care about **readability and maintenance**.
3. **Explain the "Why":** While writing `DENSE_RANK()`, explain *why* you chose it over `RANK()` (no gaps).
4. **Performance Matters:** If the interviewer asks "How to make this faster?", talk about **Indexes**, **early filtering**, and avoiding `**SELECT **`.
5. **Data Cleaning is Key:** Always use `COALESCE` or `IFNULL` for monetary sums to handle cases where there are no joined records.

🏁 CONGRATULATIONS!

You have completed the entire **SQL Mastery for Data Science** curriculum.

What's Next?

1. **Practice on LeetCode/HackerRank:** Focus on the "Hard" SQL problems.
2. **Build a Portfolio:** Upload these notes to GitHub.
3. **Real Datasets:** Download datasets from Kaggle and explore them using these advanced techniques.